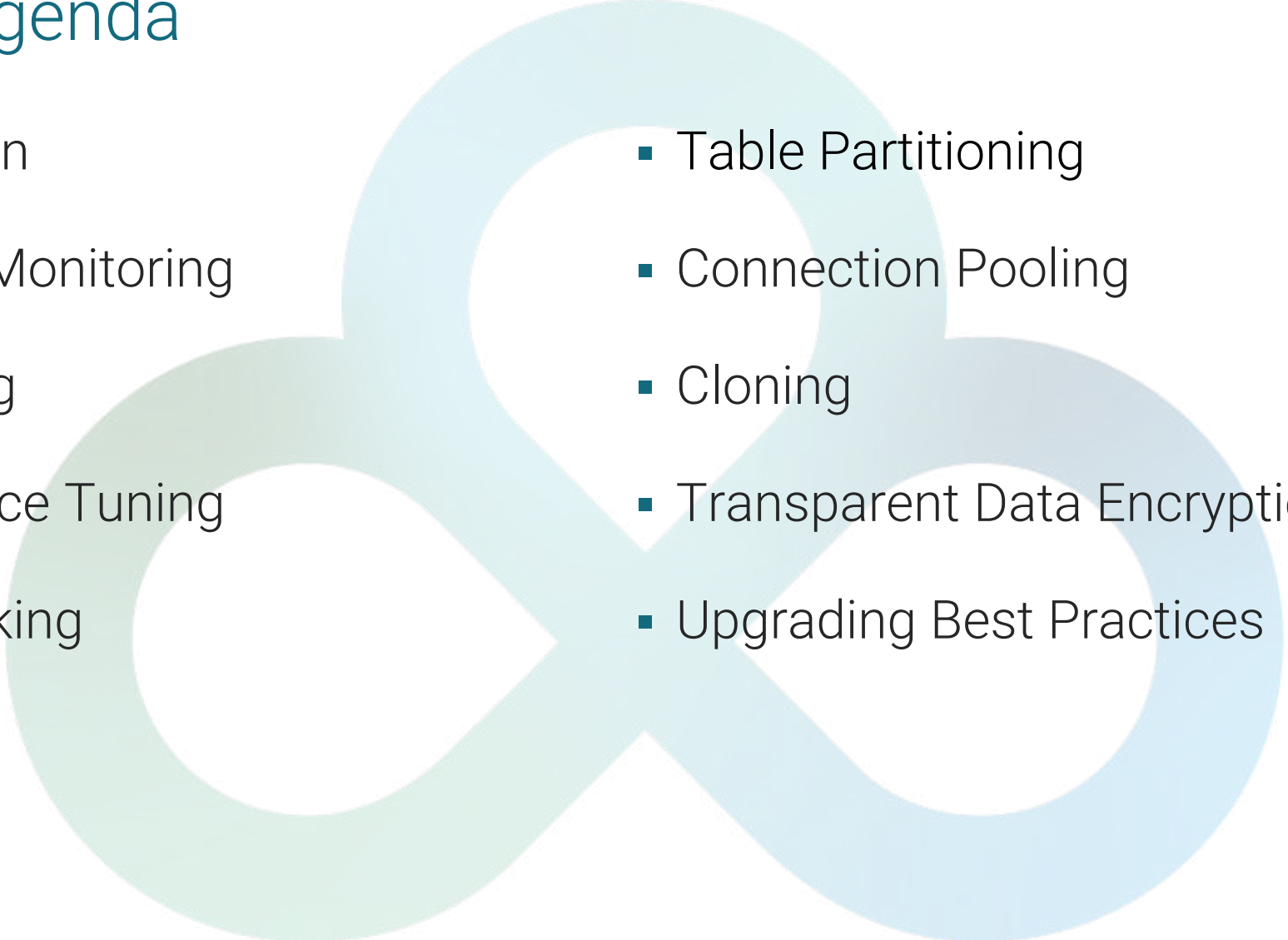




EDB Postgres AI Database Advanced v17

Course Agenda

- 
- Introduction
 - Database Monitoring
 - SQL Tuning
 - Performance Tuning
 - Benchmarking
 - Extensions
 - Table Partitioning
 - Connection Pooling
 - Cloning
 - Transparent Data Encryption
 - Upgrading Best Practices



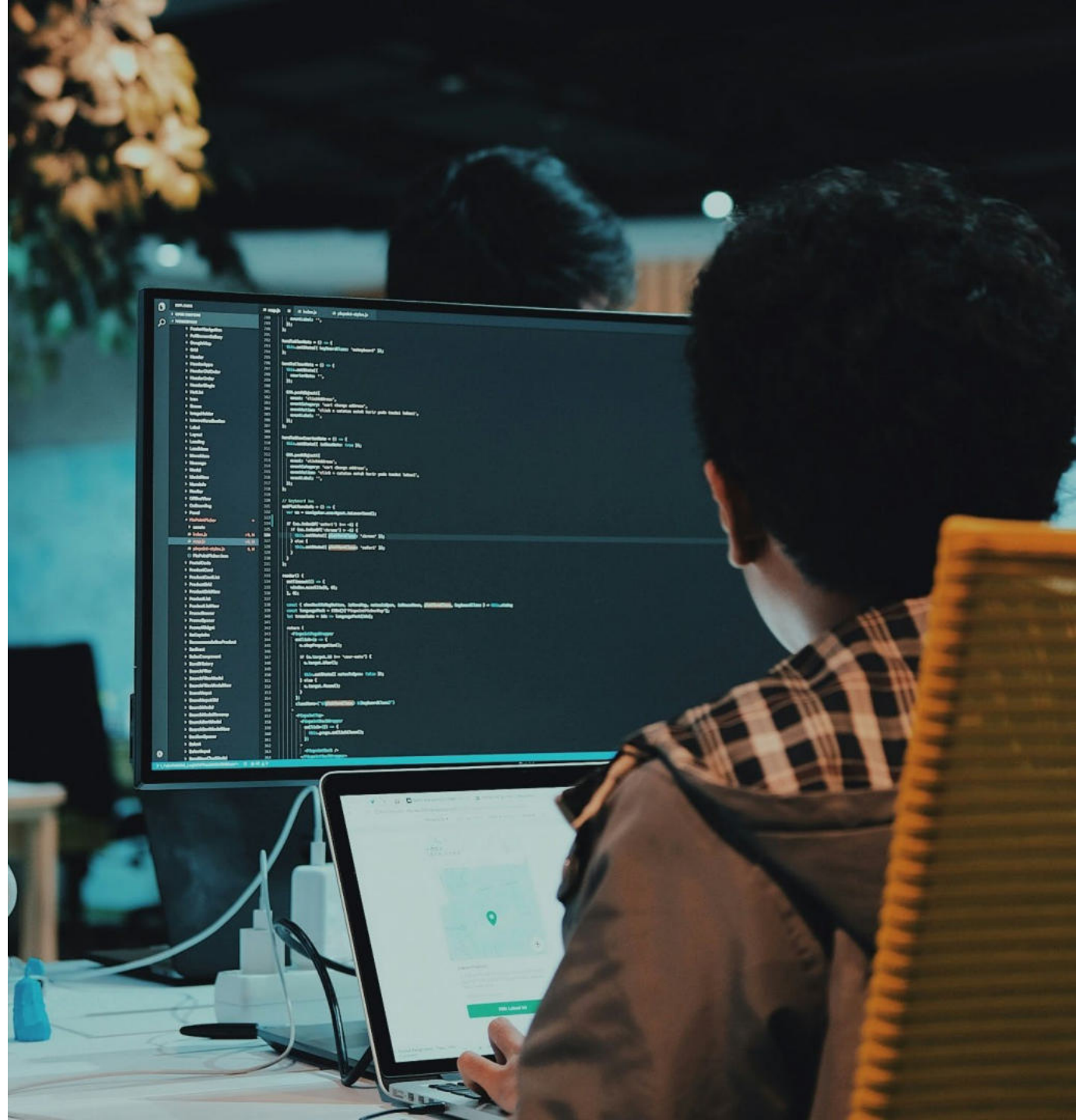


Introduction



Module Objectives

- EDB Postgres AI Platform
- Facts about PostgreSQL and Enterprise Postgres
- Lab Setup - Prepare a Sample Database



Operate Efficiently

- Cloud agility in any environment
- Unified management boosts productivity 30%
- 90% better value than cloud databases

Innovate Faster

- Accelerated time-to-market
- Secure open source flexibility
- AI-driven assistance and automation

Lead Your Market

- Fastest path to launching AI applications
- Competitive edge
- Ethical AI practices

Grow Revenue

- Always-on applications with up to 99.999% HA
- Business growth
- Customer experience

EDB Postgres AI



EDB POSTGRES® AI

YOUR SOVEREIGN DATA AND AI PLATFORM



BUILD WITHOUT BOUNDARIES

A single, sovereign data platform for any workload



ACCELERATE AI INITIATIVES AND GROW REVENUE

Secure, rapid, low-code, fully integrated AI Factory



GET REAL-TIME INSIGHTS WITHOUT COMPLEXITY

Rapid analytics and open ecosystem integrations



DRIVE OPERATIONAL EFFICIENCY WITH HYBRID MANAGEMENT

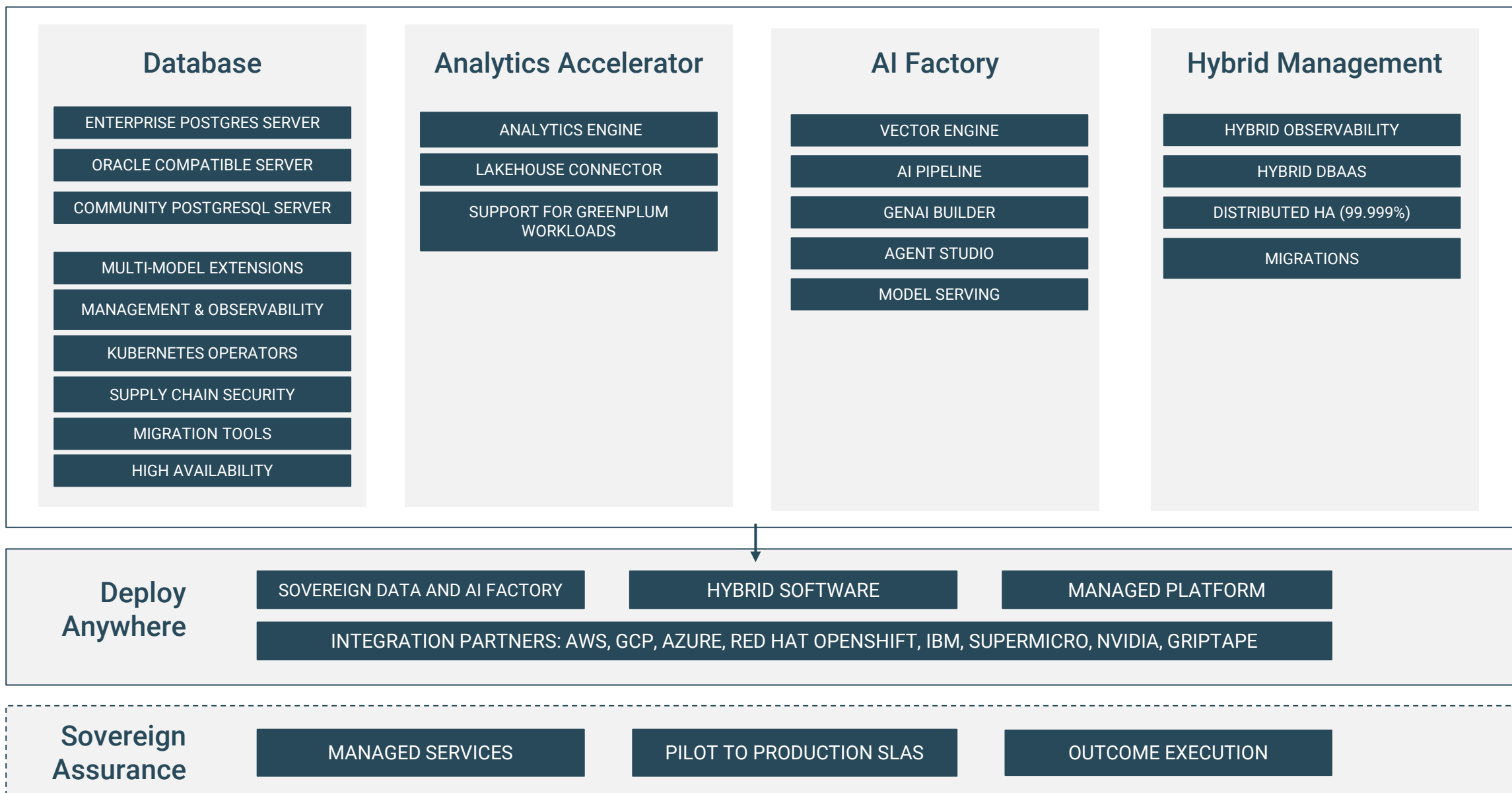
Cloud-native automation and deep observability across deployments from a single pane of glass



DELIVER CONTROLLED OUTCOMES WITH COMPLETE SOVEREIGNTY

Flexible deployment across hybrid software, sovereign data and AI factory, and managed platform





PostgreSQL

The open-source database of choice



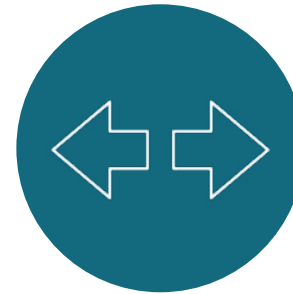
Performance

Handles enterprise workloads with 50% improvement in the last 4 years



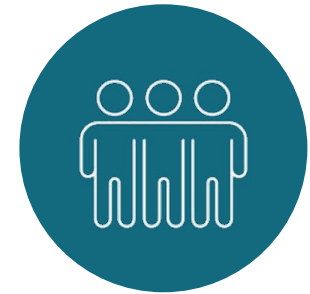
Scalability

Multiple technical options for operating Postgres at scale



Extensibility

Supported by a wide array of extensions plus multiple SQL and NoSQL data models



Community-driven

Multiple companies and individuals contribute to the project and drive innovation



Enterprise Postgres (Oracle Compatible)



Enterprise Postgres

- **Oracle Compatibility** - Compatibility for schemas, data types, indexes, users, roles, partitioning, packages, views, PL/SQL triggers, stored procedures, functions, and utilities
- **Additional Security** - Password policy management, session tag auditing, data redaction, SQL injection protection, and procedural language code obfuscation
- **Developer Productivity** - Over 200 pre-packaged utility functions, user-defined object types, autonomous transactions, nested tables, synonyms, advanced queueing
- **DBA Productivity** - Throttle CPU and I/O at the process level, over 55 extended catalog views to profile all the objects and processing that occurs in the database
- **Performance** - Query optimizer hints, SQL session/system wait diagnostics
- **Replication Enhancements** - Enables EDB Postgres Distributed functionality such as Group Commit, Commit at Most Once and Eager all-node synchronous replication, timestamp-based snapshots, estimates for replication catch-up times, selective backup of a single database, hold back freezing to assist resolution of UPDATE/DELETE conflicts, multi-node PITR



Lab Setup Guidelines

All the instructor demos and labs are based on Linux. Here are the machine requirements:

- Rocky Linux 9x with 2 GB RAM and 40 GB storage space is recommended.
- Latest version of Enterprise Postgres (Oracle Compatible) must be installed
- superuser access to Database and Operating System
- EDB credentials for EDB Repos
- Internet access for downloading EDB packages



Prepare a Sample Database

- In the training materials provided by EDB Postgres AI, there is a script file [edbstore.sql](#) that can be executed using `psql` to create a sample `edbstore` database.
- Here are the steps:
 - Download the [edbstore.sql](#) file and place in a directory which is accessible to the `enterprisedb` user
 - Login as `enterprisedb` OS user
 - Run the `edb-psql` command with the `-f` option to execute the [edbstore.sql](#) file and install all the sample objects required for this training

```
$ psql -p 5444 -f edbstore.sql -d edb -U enterprisedb
```

- Note - The above command will prompt for `edbuser` password. The password is `edbuser`



Prepare a Sample Database (Continued)

- Connect to edbstore database using edbuser
 - `psql -p 5444 -d edbstore -U edbuser`
- Verify the existence of the objects in psql by running `\d`

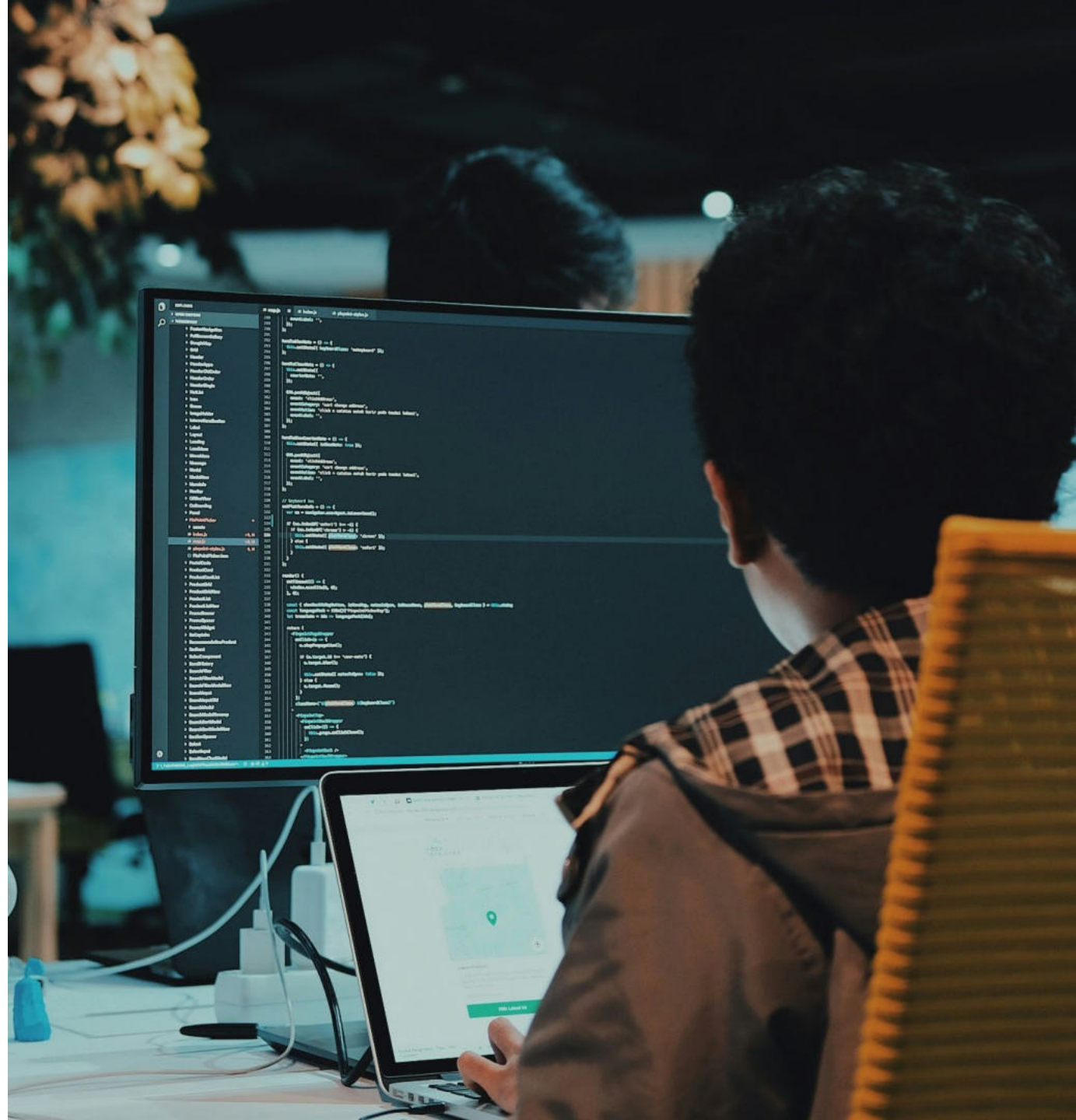
```
edbstore=> \d
```

List of relations			
Schema	Name	Type	Owner
edbuser	categories	table	edbuser
edbuser	categories_category_seq	sequence	edbuser
edbuser	cust_hist	table	edbuser
edbuser	customers	table	edbuser
edbuser	customers_customerid_seq	sequence	edbuser
edbuser	dept	table	edbuser
edbuser	emp	table	edbuser
edbuser	inventory	table	edbuser
edbuser	job_grd	table	edbuser
edbuser	jobhist	table	edbuser
edbuser	locations	table	edbuser
edbuser	next_empno	sequence	edbuser
edbuser	orderlines	table	edbuser
edbuser	orders	table	edbuser
edbuser	orders_orderid_seq	sequence	edbuser
edbuser	products	table	edbuser
edbuser	products_prod_id_seq	sequence	edbuser
edbuser	reorder	table	edbuser
edbuser	salesemp	view	edbuser
(19 rows)			



Module Summary

- EDB Postgres AI Platform
- Facts about PostgreSQL and Enterprise Postgres
- Lab Setup - Prepare a Sample Database





Database Monitoring



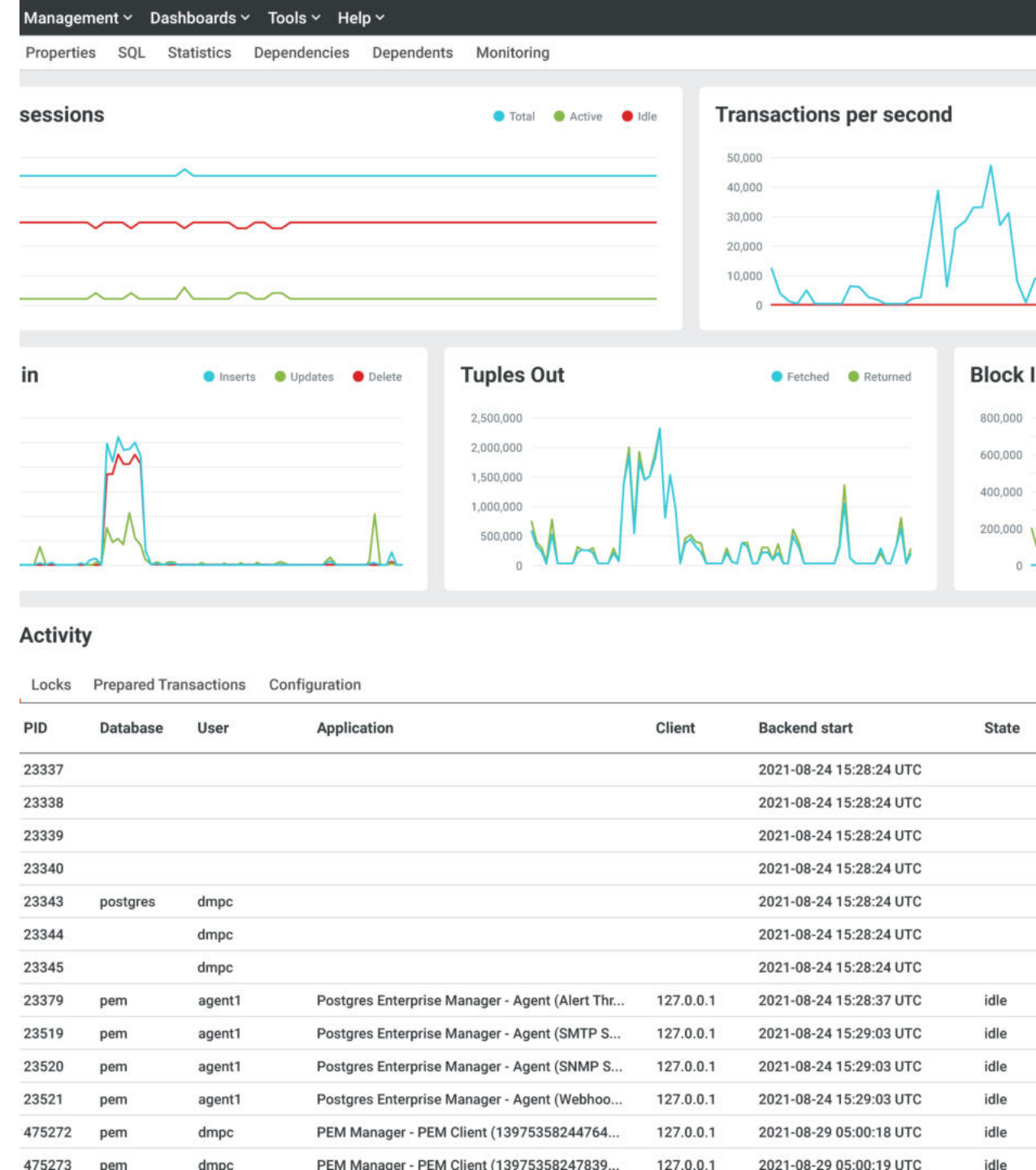
Module Objectives

- What is Database Monitoring?
- Postgres Stats Tables and Stats Collector Process
- Which Monitoring Tool to use?
- Check Sessions Activity and Locking
- Use pg_stat_statements to track planning and execution statistics
- How to Manage and Monitor Postgres using Postgres Enterprise Manager



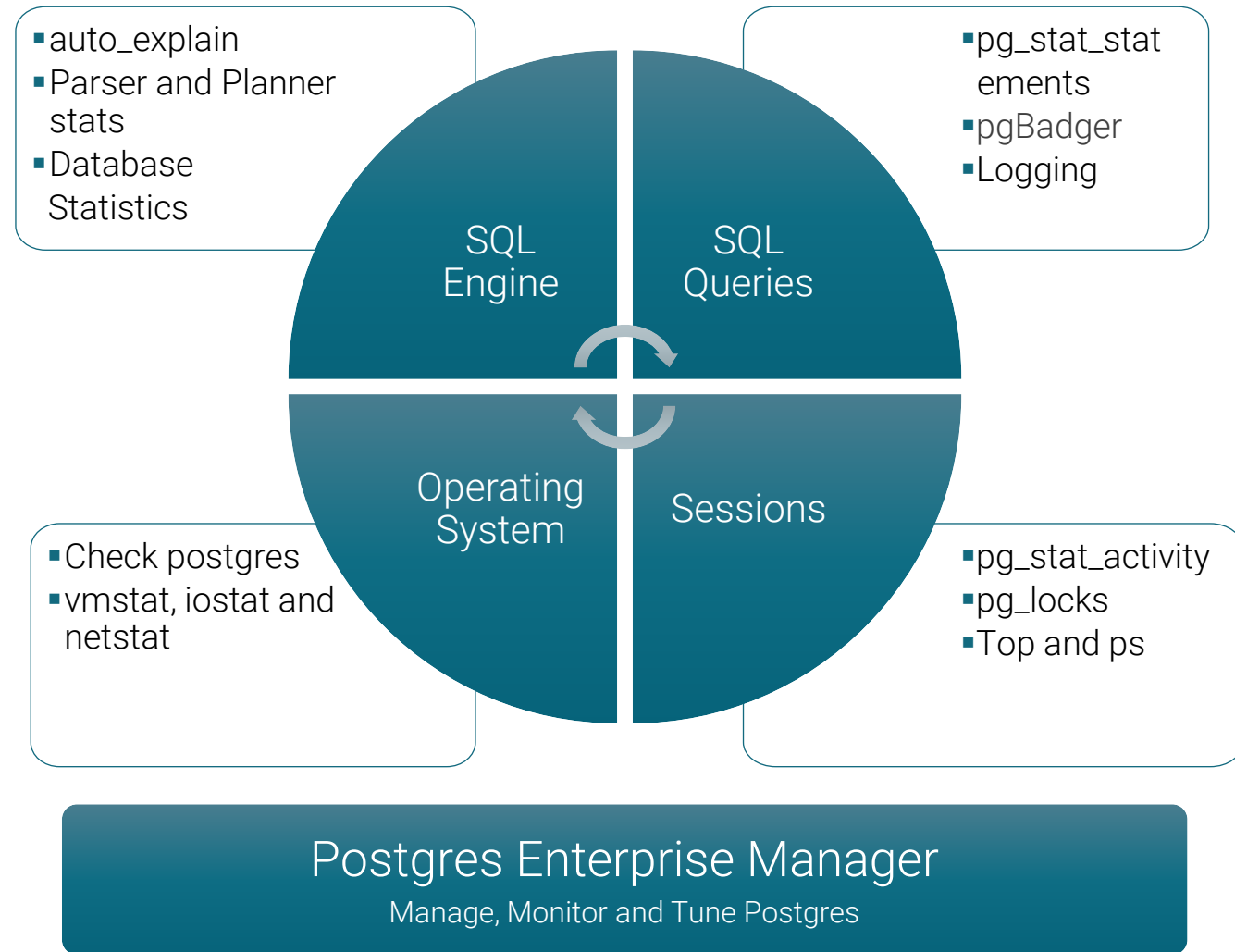
Database Monitoring

- Capturing and recording the database events
- Detect, identify and fix potential database performance issues
- Database statistics make it easy for you to monitor the health and performance of PostgreSQL databases



Monitoring Utilities

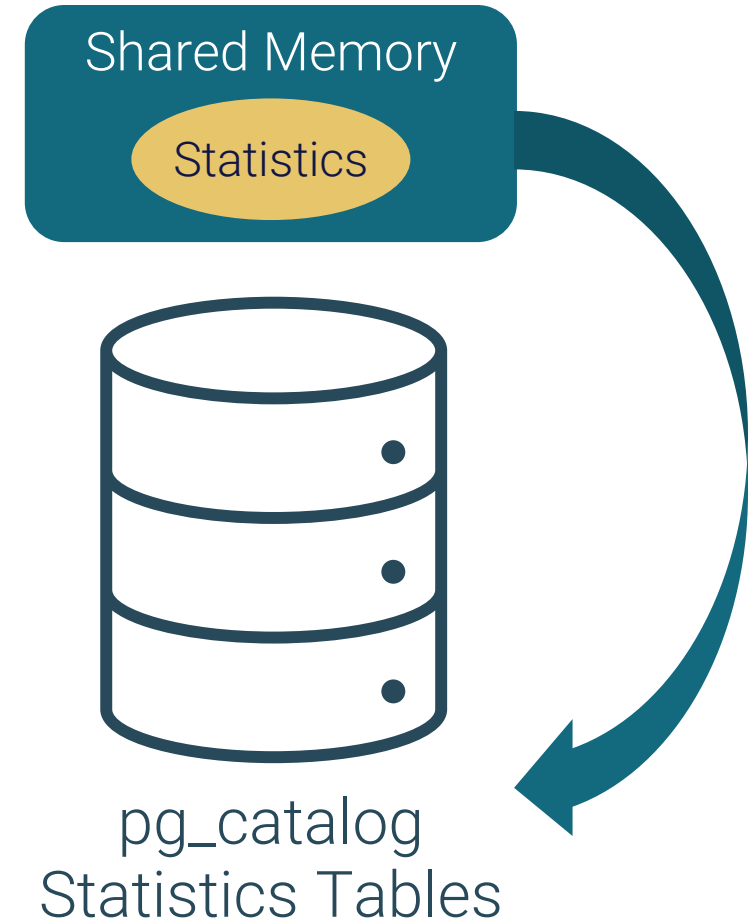
- Why is my database running slower?
- How to monitor slow running queries, locking, errors and on-going maintenance activities.
- Open-source tools like pgBadger can complement PostgreSQL Logging
- Complete Monitoring - Postgres Enterprise Manager



Database Statistics

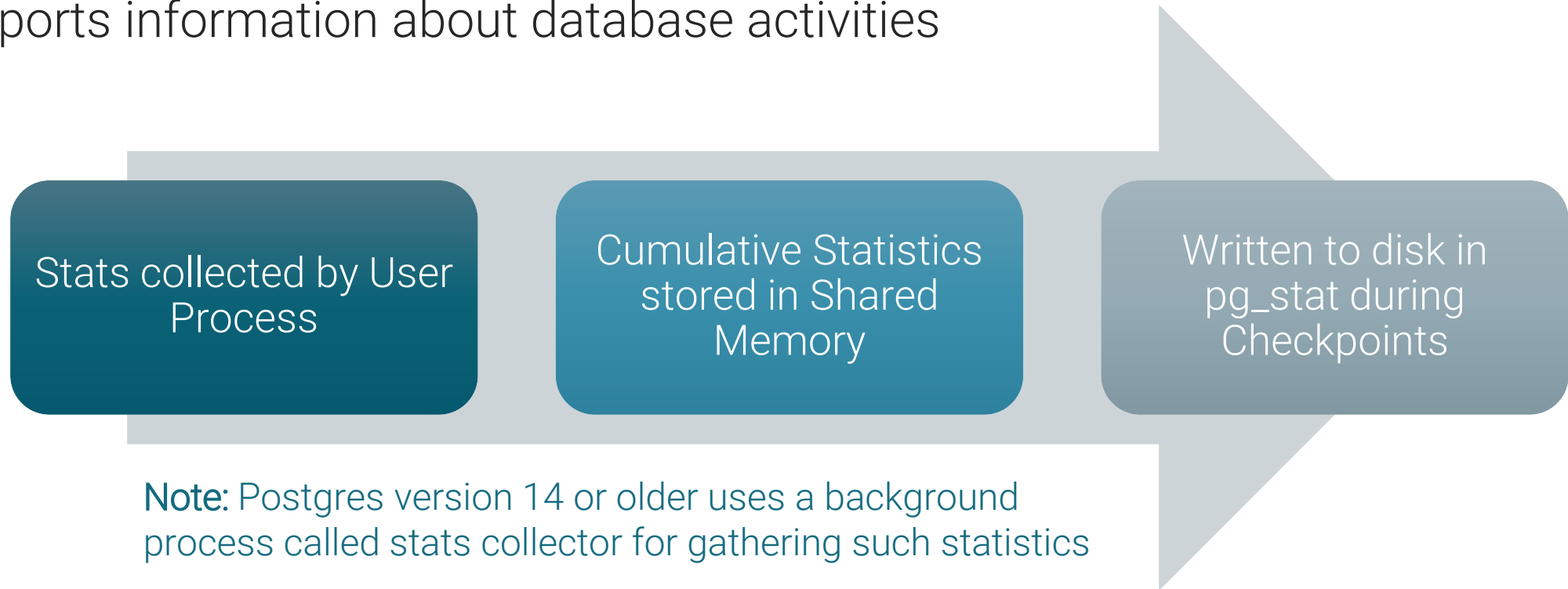
Database statistics catalog tables store database activity information:

- Current running sessions and SQL queries
- Locks
- DML counts
- Row counts
- Index usage



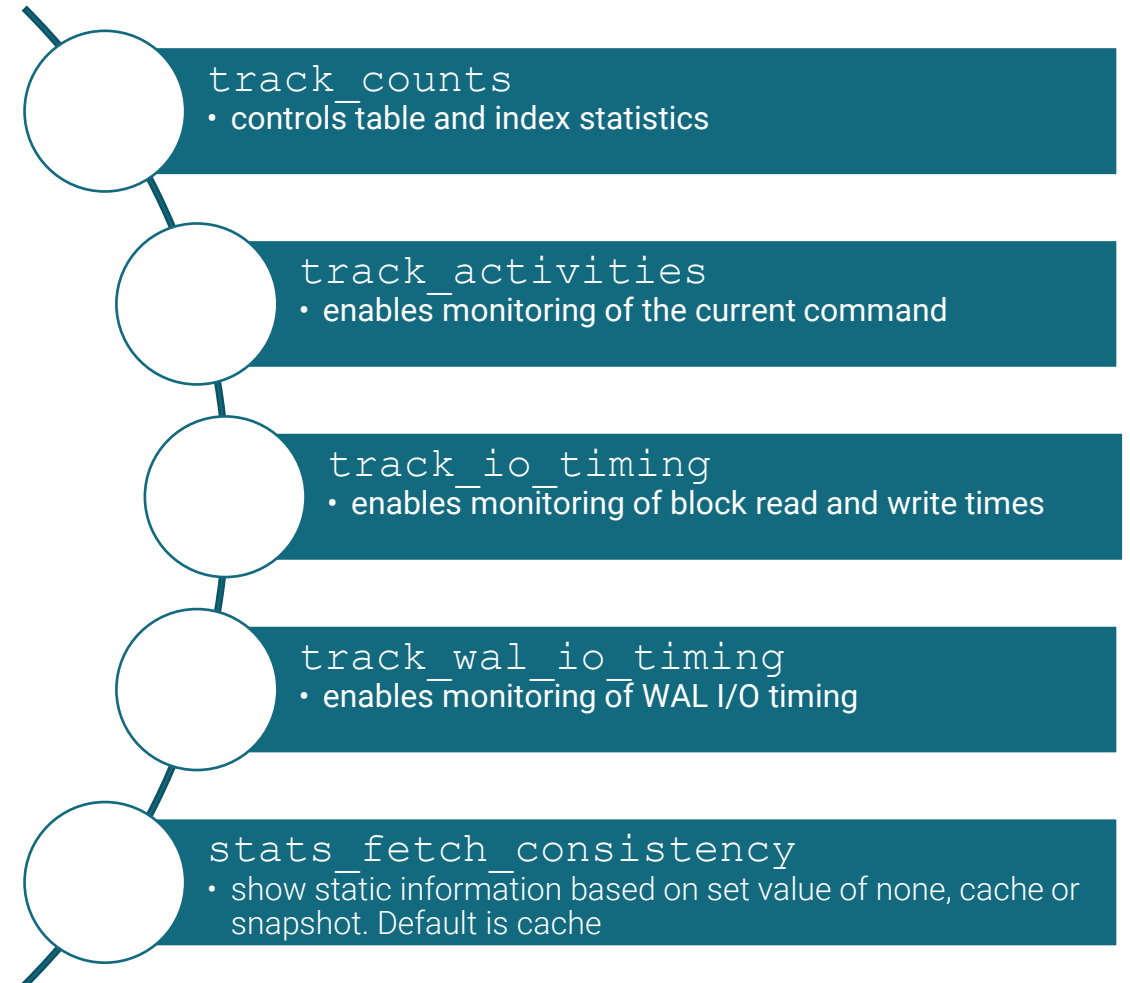
The Cumulative Statistics System

- The Cumulative Statistics System collects and reports information about database activities



Statistics System Configuration

- Statistics system adds some overhead to query execution
- Parameters can be configured at instance, database, user or session level
- Access to cumulative and dynamic statistics is restricted to superusers and users with `pg_read_all_stats` role
- Cumulative statistics is not the same as running `ANALYZE` to gather row sampling data



Database Statistic Tables

Table or View	Description
<code>pg_class</code> and <code>pg_stats</code>	Catalog tables which stores all statistics
<code>pg_stat_database</code>	System view can be used to view stats information about a database and temporary file counts and sizes
<code>pg_stat_bgwriter</code>	Shows background writer stats and checkpoint timing information
<code>pg_stat_user_tables</code>	Shows information about activities on a table like inserts, updates, deletes, vacuum, autovacuum etc.
<code>pg_stat_user_indexes</code>	Shows information about index usage for all user tables
<code>pg_stat_progress_vacuum</code>	Shows one row for each backend including autovacuum worker processes that is currently vacuuming
<code>pg_stat_progress_create_index</code>	Shows progress of <code>CREATE INDEX</code> and <code>REINDEX</code> operations
<code>pg_stat_progress_cluster</code>	Shows progress of <code>CLUSTER</code> and <code>VACUUM FULL</code>
<code>pg_backend_memory_contexts</code>	View memory context for the for current server process



Monitoring I/O

- The pg_stat_io view Cluster-wide I/O statistics
- The View represents a combination of backend type, target I/O object, and I/O context
- High evictions count may suggest a need to increase shared buffers
- Large numbers of fsyncs by client backends could indicate misconfiguration of shared buffers or the checkpointer

Backend types

- background worker
- autovacuum workers
- checkpointer and more

I/O operations

- | | |
|-----------|-------------|
| ▪ Reads | ▪ Evictions |
| ▪ Writes | ▪ Reuses |
| ▪ Extends | ▪ Fsyncs |
| ▪ Hits | |

I/O contexts

- Normal
- Vacuum
- Bulkread
- Bulkwrite



Monitoring Views

- Check wait events using `pg_wait_events`
- Checkpointer process statistics can be viewed using `pg_stat_checkpoint`
- Index processing during vacuuming can now be checked using `indexes_total` and `indexes_processed` of `pg_stat_progress_vacuum` view

```
pgdb=# SELECT * FROM pg_wait_events ;
type           | Activity
name           | ArchiverMain
description    | Waiting in main loop of archiver process
```

```
pgdb=# SELECT * FROM pg_stat_checkpoint;
num_timed          | 257
num_requested      | 4
restartpoints_timed | 0
restartpoints_req  | 0
restartpoints_done | 0
write_time         | 291771
sync_time          | 24
buffers_written    | 2910
stats_reset        | 2024-10-23 17:50:04.307163+00
```



Sessions Activity and Locking

Check Current Session Activity

- `pg_stat_activity`

Check Transaction Duration

- `pg_stat_activity`

Current Locks in Lock Manager

- `pg_locks`

Blocking Session

- `pg_stat_activity` and `pg_blocking_pids()`

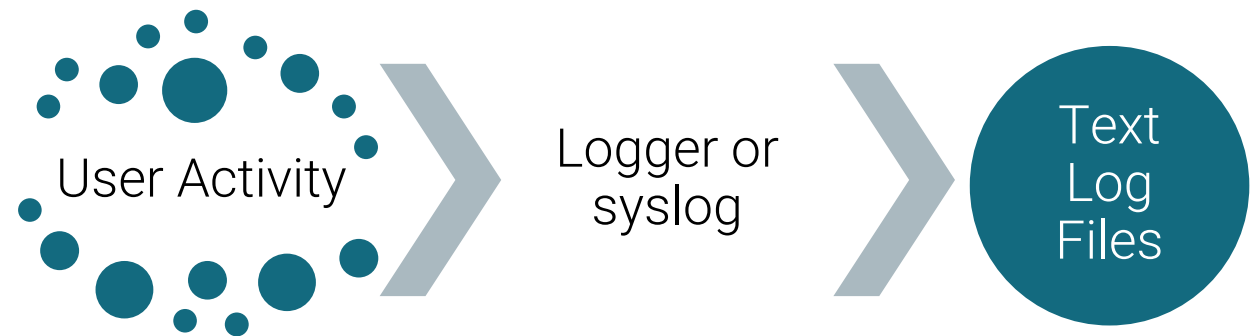
Lock Waits

- `pg_stat_activity` and `pg_locks`



Error Reporting and Logging

- Monitor and track what's happening using the in-built logging features of Postgres
- Logs are text files containing errors, info and various other messages
- SQL queries can be written to logs
- Logs can be shipped to a central location and parsed using various tools



Logging Configuration: What, When and Where to Log



Logging Parameters

Configure where, when and what to log in postgresql.conf file



Location

log_destination
logging_collector
log_directory
log_filename



Events

log_min_duration_statement
log_min_duration_sample
log_statement_sample_rate



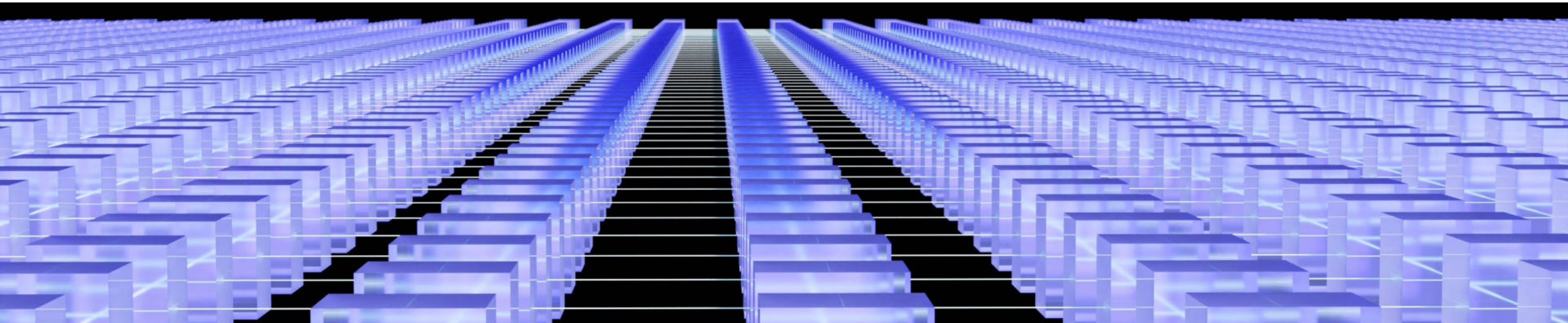
Information Written to Logs

log_line_prefix
log_statement
log_temp_files
log_autovacuum_min_duration
log_checkpoints
log_connections
log_lock_waits



Tracking Execution Statistics

- Track query execution statistics using `pg_stat_statements` extension
- `pg_stat_statements` tracks statistics across all databases of a cluster
- `pg_stat_statements` extension add view, functions and configuration parameters
- View - `pg_stat_statements` to access the query statistics
- Functions - `pg_stat_statements_reset` to reset the statistics



pg_stat_statements Setup

- Add `pg_stat_statements` to `shared_preload_libraries` parameter in `postgresql.conf`
- Configurable parameters:
 - `pg_stat_statements.max` -Maximum number of tracked statements, default 5000
 - `pg_stat_statements.track` - Which statements are counted, `top`, `all` or `none`. Default is `top`
 - `pg_stat_statements.track_utility` - Track commands other than `SELECT`, `INSERT`, `UPDATE` and `DELETE`. Default is `ON`
 - `pg_stat_statements.track_planning` - Track planning operations and duration
 - `pg_stat_statements.save` - Whether save statement statistics across server shutdowns. Default is `ON`
- Restart the Database Cluster
- Connect with a database and enable `pg_stat_statements` extension



Postgres Enterprise Manager

Manage, monitor, and tune Postgres at scale



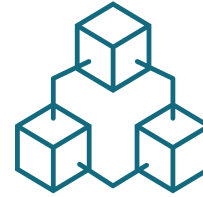
Manage from one interface

One place to visualize and manage everything



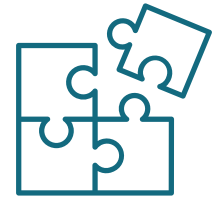
Optimize database performance

In-depth diagnostics for database reports and tuning



Monitor system health

Built-in dashboards and customizable alert thresholds



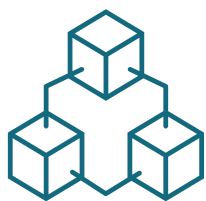
Integrate with other tools

APIs and webhooks to fetch data, send alerts, and manage servers



PEM - Features

Manage, Monitor and Tune PostgreSQL and Enterprise Postgres running on multiple Platforms



Management

- Integrated SQL IDE
- Built-in query debugger
- User/group access management
- Schema Diff
- Session profiling
- Job scheduling
- Backup and failover management



Monitoring

- Customizable charts and dashboards
- Predefined and custom alerts via email or SNMP
- User-defined metrics log analysis
- Database and OS level monitoring
- Web hooks and REST API for integrations



Tuning

- Detailed performance diagnostics
- SQL profiler
- Capacity management
- Log manager
- Expert wizards for configuration setup



PEM Components

What can be monitored using PEM?

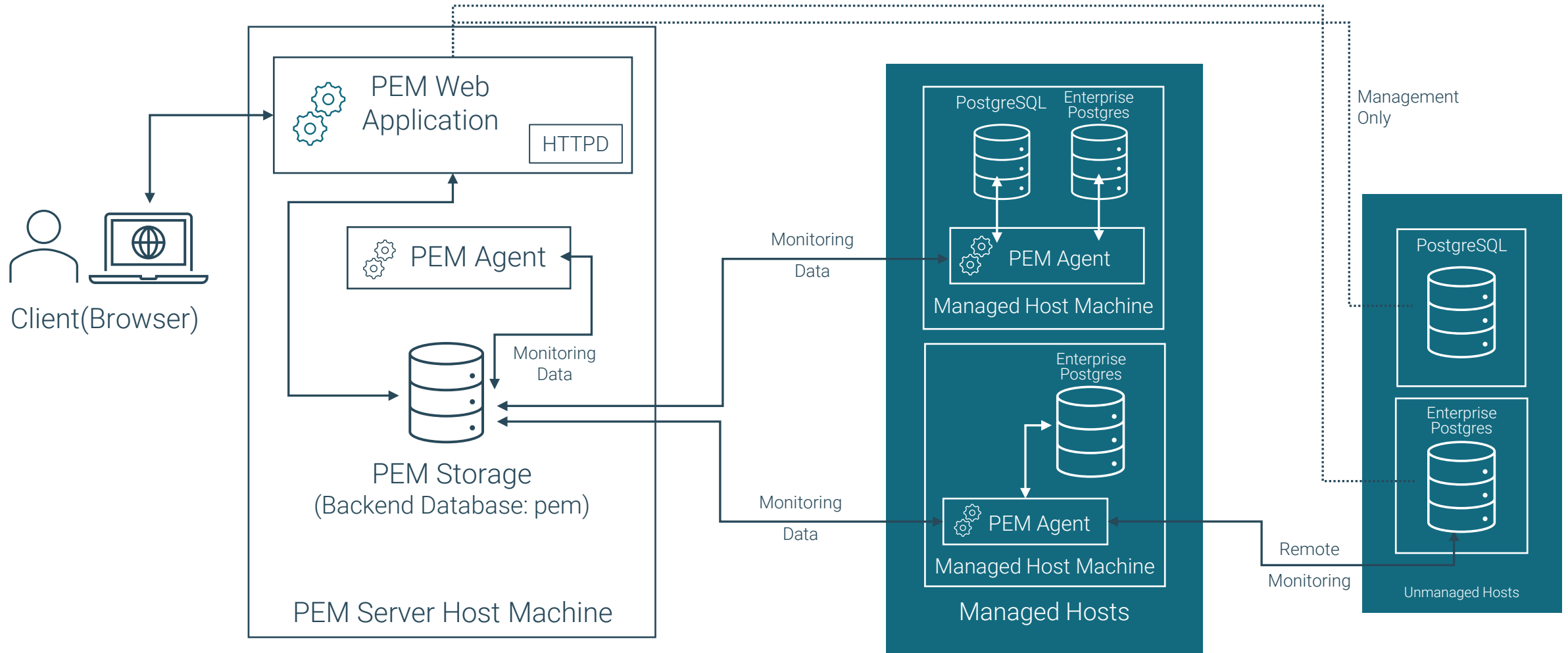
- Hosts – Operating System
- Servers – Postgres Instances
- Databases – Databases and Schema Objects

Individual Software Components:

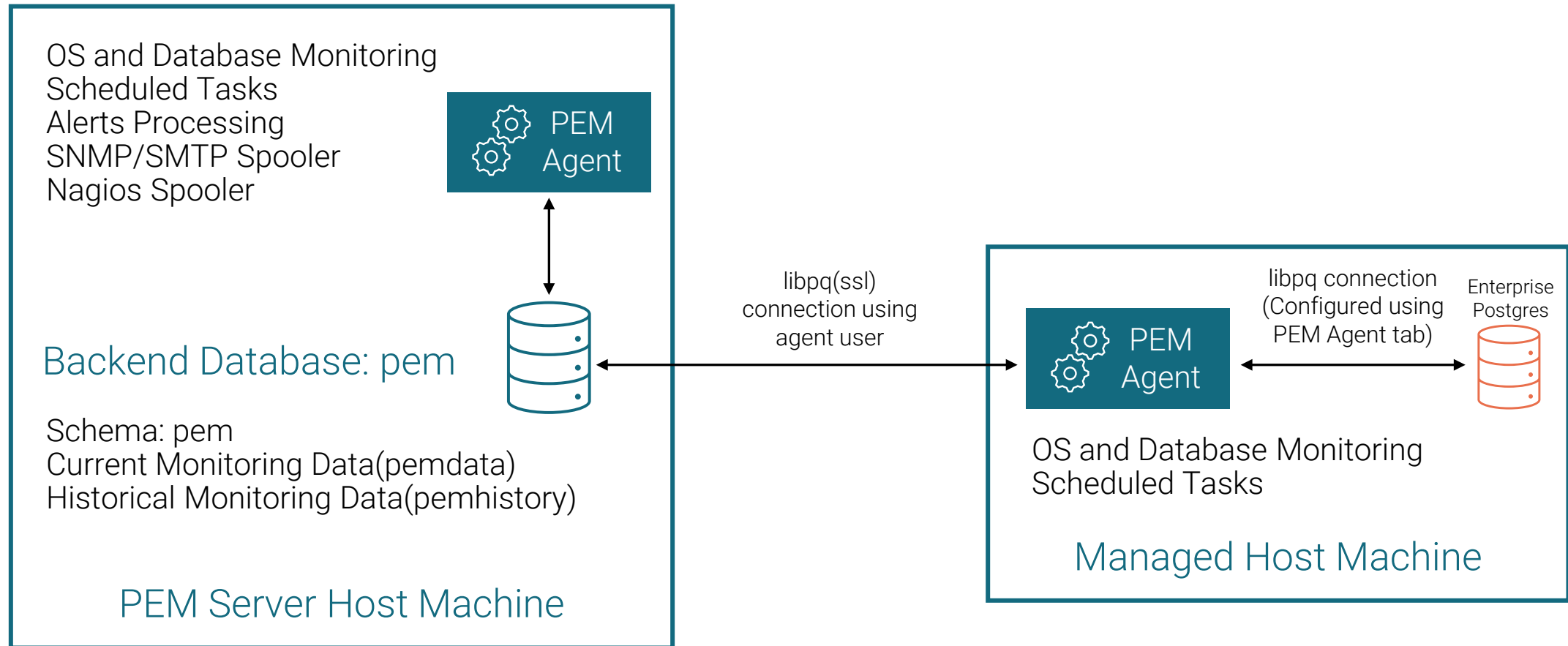
- PEM Server
- PEM Web Client
- SQL Profiler
- PEM Agent



PEM Architecture



PEM Server and Agent Communication



PEM- Supported CPU Architecture and OS

- PEM Server can be installed on various flavors of Linux including Rocky, RHEL and Ubuntu
- PEM Server can be installed on Linux x86-64 (amd64), Linux on IBM Power and Windows x86-64
- PEM Web Client can be accessed using Chrome, Firefox, Microsoft Edge and Safari browser.
- PEM Agent and SQL Profiler is supported on any platform where PostgreSQL or Enterprise Postgres version 9.6 or higher is installed.
- Check latest platform support:

<https://www.enterprisedb.com/product-compatibility#pem>



Installation - Prerequisites

- PEM server requires a backend database for storing monitoring data.
- Backend database can be Enterprise Postgres or PostgreSQL. Recommended version is 12 or higher.
- It is recommended to install PEM server on a system with at least 4 CPU cores, 8GB of RAM and 100GB of Storage.
- Additional disk space is required based on number of monitored instances and retention period for monitoring data.
- pg_hba.conf entries are required for PEM server PEM agent and PEM-HTTPD service connection to managed or unmanaged instances.



Postgres Enterprise Manager Download

- Go to www.enterprisedb.com and sign in or sign up for a free account
- Go to Resources->Downloads
- Look for Postgres Enterprise Manager and Click on Server
- Click on Access Repo button to activate your edb.repo access

Postgres Enterprise Manager
Manage, monitor, and optimize PostgreSQL
[Docs](#)
[Product Information](#)
[Release notes](#)

9.x Server 9.x Agent SQL Profiler

	Linux ARM64 DEB	Access repo
	Linux on Power RPM	Access repo
	Linux x86-64 RPM	Access repo
	Linux x86-64 DEB	Access repo
	Windows x86-64 Interactive Installer	Download



EDB Repo Access Details

EDB repositories

SIGNED IN AS ARUN GAVHANE

EDB Repos 2.0

Access to EDB's improved download experience

Currently available for **EDB Enterprise**, **EDB Standard**, **Trusted Postgres Architect**, **EDB Postgres Distributed 5**, **EDB Postgres for Kubernetes**, and **EDB Postgres Distributed for Kubernetes**.

Token-based authentication

Single, simple URL for all download types

Currently supporting **Red Hat Enterprise Linux (RHEL)**, **Oracle Linux (OL)**, **CentOS**, **Rocky Linux**, **AlmaLinux**, **Debian**, **Ubuntu**, and **SUSE Linux Enterprise Server (SLES)**

Repo Token:

[Access EDB Repos 2.0](#)



Installation – EDB Repos 2.0 Configuration for Linux

- On EDB Repositories page, select platform and search for Postgres Enterprise Manager
- Select latest version of PEM to view the installation instructions
- Follow Auto or Manual installation steps to configure enterprisedb repo
- Example: Auto Mode

Select Platform:

Rocky/Alma/Oracle Linux 9 x86_64 ▼

EDB Postgres Enterprise Manager

✓ 9.x.x

```
curl -sLf 'https://downloads.enterprisedb.com/<token>/enterprise/setup.rpm.sh'  
| sudo -E bash
```



Preparing PEM Repo Database

- Install Enterprise Postgres version 12 or later
 - An existing install of Enterprise Postgres or PostgreSQL can be used.
- Setup a database cluster and start the database service
 - Docs: <https://www.enterprisedb.com/docs/epas/latest/installing/>
- Gather connection information
 - Hostname, Port, User, Password
 - Database superuser access is required by PEM installer
- Install sslutils and contrib packages if using PostgreSQL
 - `dnf install sslutils_<x> postgresql<X>-contrib`



Install PEM server on Linux

- Install the PEM server on Rocky Linux 9 or RHEL:
 - `dnf -y install edb-pem`
 - PEM Agent (`edb-pem-agent`) is installed by default
- PEM Agent can be installed on managed hosts using:
 - `dnf -y install edb-pem-agent`



Post Install PEM Server Configuration

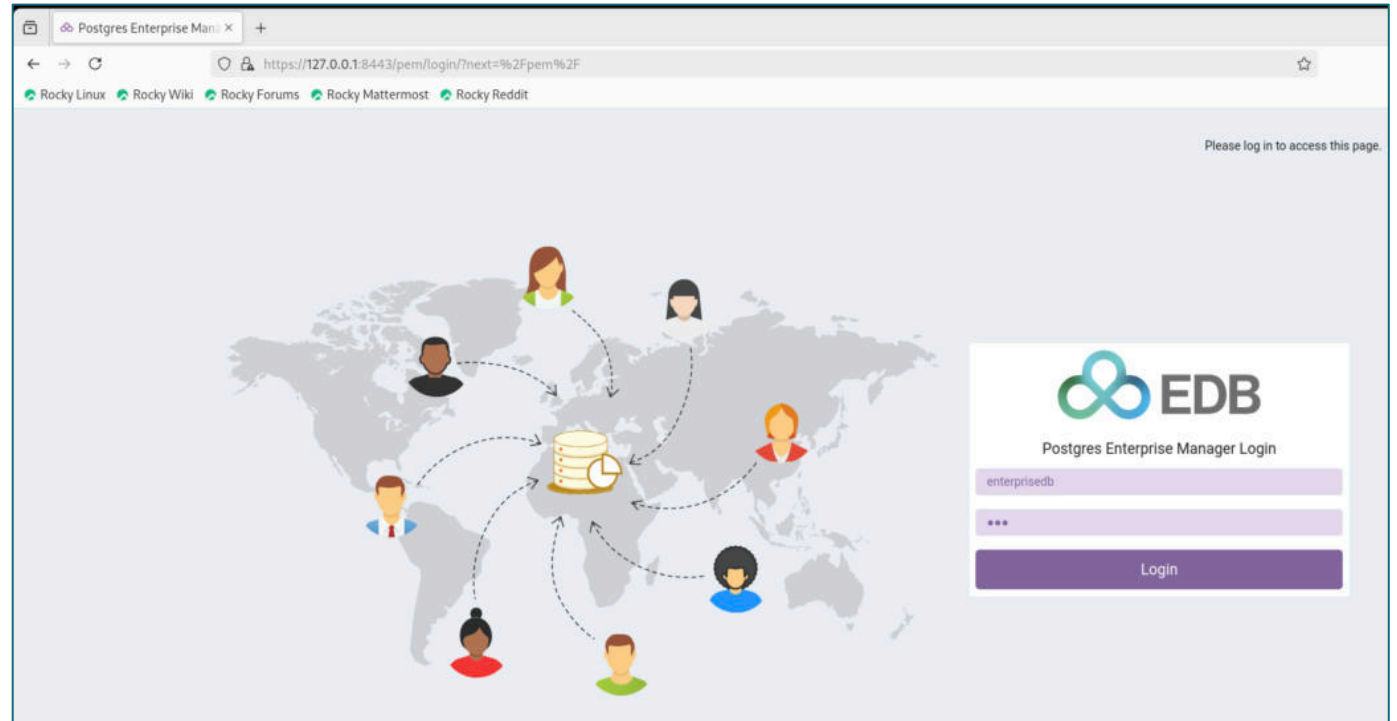
- The PEM server installation includes a configuration script that helps in:
 - Repo database (pem) setup
 - PEM Web Services setup
 - Local PEM Agent registration/binding
- Script must be run as root or sudo user:
 - `/usr/edb/pem/bin/configure-pem-server.sh`
- Configuration options and help:
 - `/usr/edb/pem/bin/configure-pem-server.sh --help`



PEM Web Client

- Open PEM Web Client using the URL provided at the end of installation:

```
--> [Info] --> [Info]  
Configured the webservice for EDB  
Postgres Enterprise Manager (PEM)  
Server on port '8443'.  
--> [Info] --> [Info] PEM  
server can be accessed at  
https://127.0.0.1:8443/pem  
at your browser
```



PEM Web Client - First Look

The screenshot displays the PEM Web Client interface. The top navigation bar includes a logo, a search bar, and menu items: File, Object, Management, Dashboards, Tools, and Help. The user is logged in as 'enterprisedb'. The left sidebar shows a 'Browser' section with links to BART Servers, Barman Servers, PEM Agents, and PEM Server Directory. The main content area is titled 'Monitoring' and features a 'Global Overview' section with four summary cards: Object Type (System), Status (N/A), Generated On (08/01/2025, 15:24:17), and No. of alerts (4 (Acknowledged : 0)). Below this is the 'Enterprise Dashboard' section, which contains a 'Status' bar chart and two tables: 'Agent Status' and 'Server Status'.

Global Overview

- Object Type: System
- Status: N/A
- Generated On: 08/01/2025, 15:24:17
- No. of alerts: 4 (Acknowledged : 0)

Enterprise Dashboard

Status

Agent Status

Blackout	Status	Name	Alerts	Version	Processes	Threads	CPU Utilization (%)	Memory Utilization (%)	Swap Utilization (%)	Disk Utilization
☐	Up	Postgres Enterprise Manager Host	1	9.8.0	390	1085	3.11	71.10	14.16	47.24

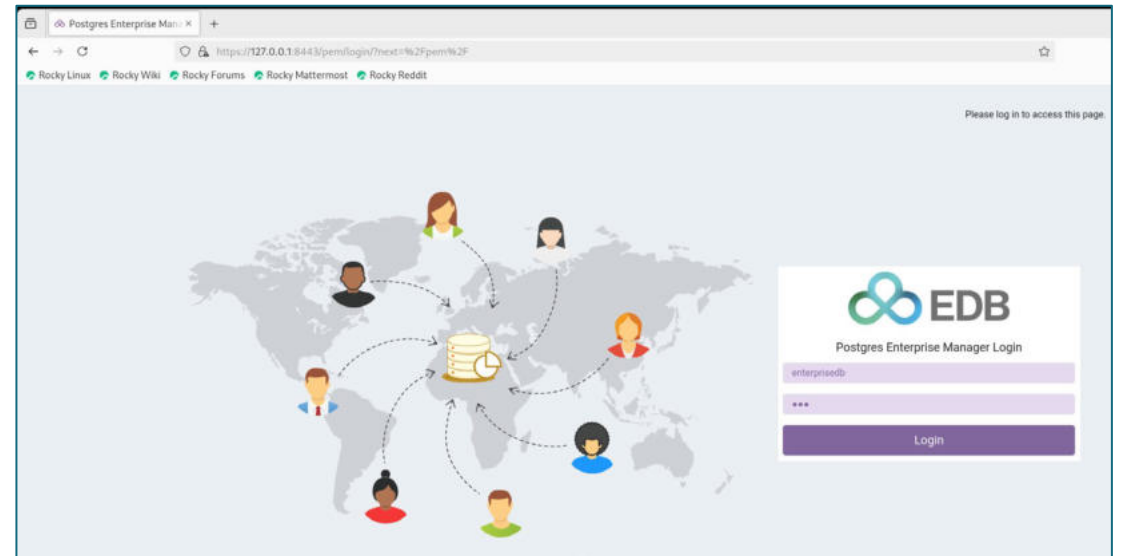
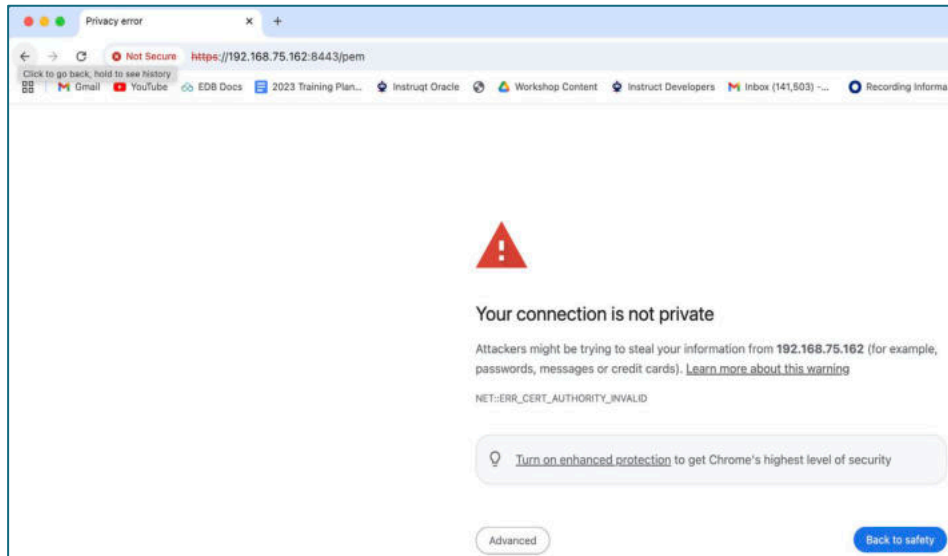
Server Status

Blackout	Status	Name	Connections	Alerts	Version	Remotely Monitored?
☐	Up	Postgres Enterprise Manager Server	11	3	PostgreSQL 17.2 (EnterpriseDB Advanced Server 17.2.0) on x86_64-pc-linux-gnu, compiled by gcc (GCC) 11.5.0 20240719 (Red Hat 11.5.0-2), 64-bit	No



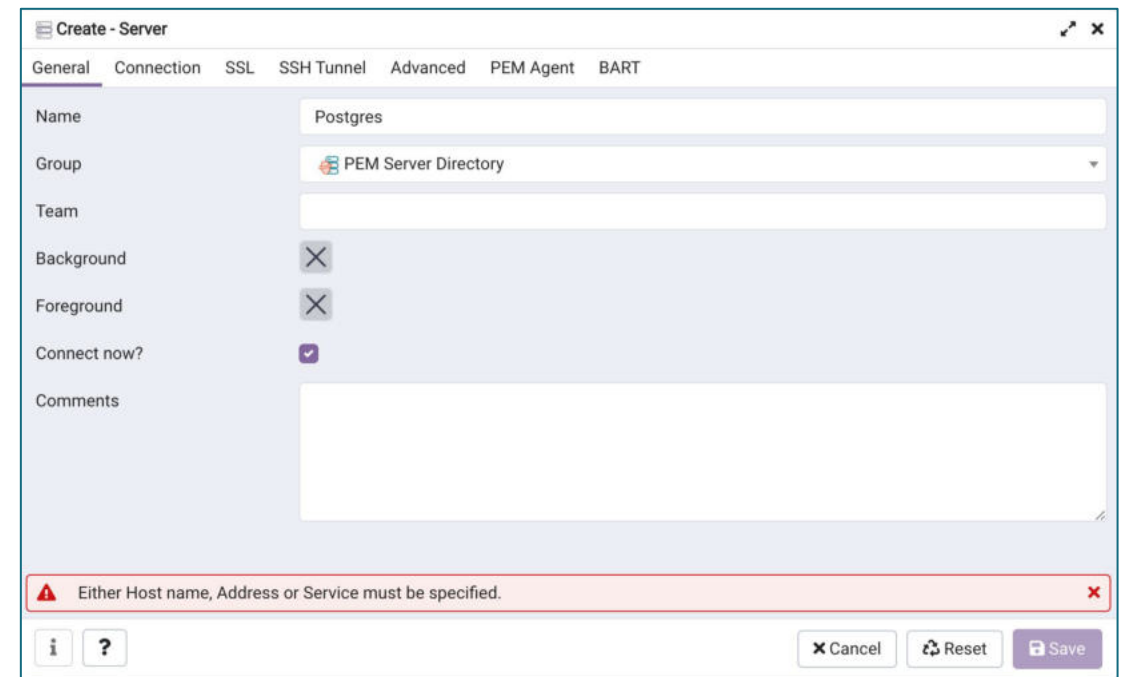
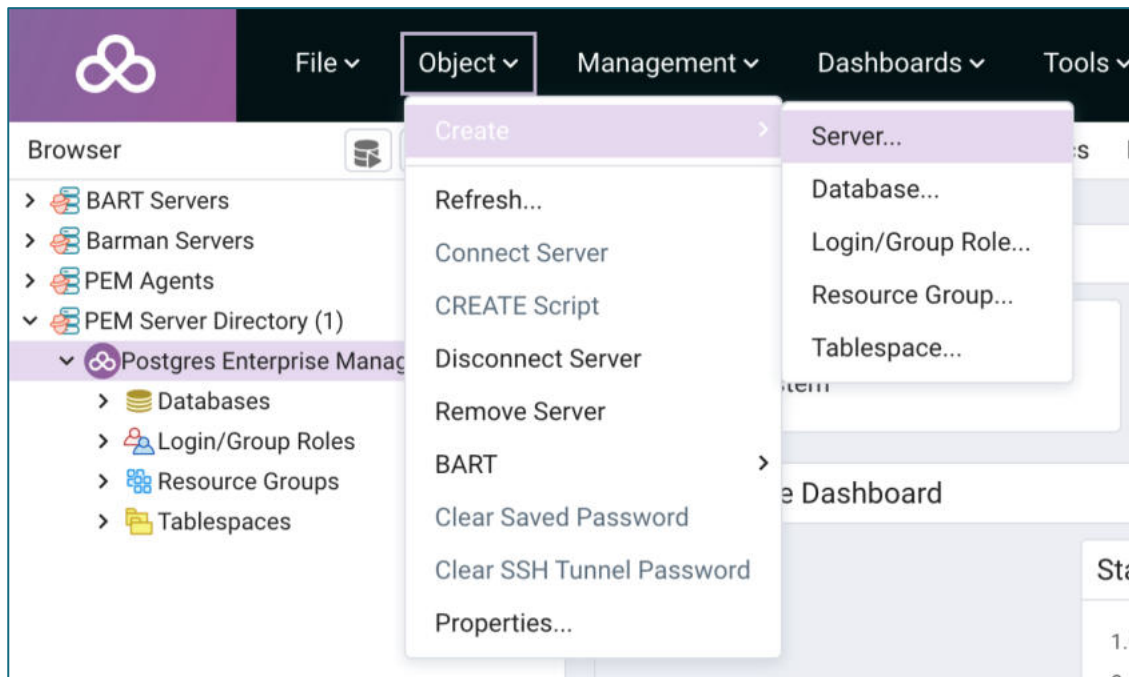
Accessing PEM Web Interface

- PEM client can be accessed using a browser
- Your browser may require a security exception to access PEM login page
- Default port for PEM client is 8443
- Open PEM client using:
`https://<pem_host_ip>:8443/pem`



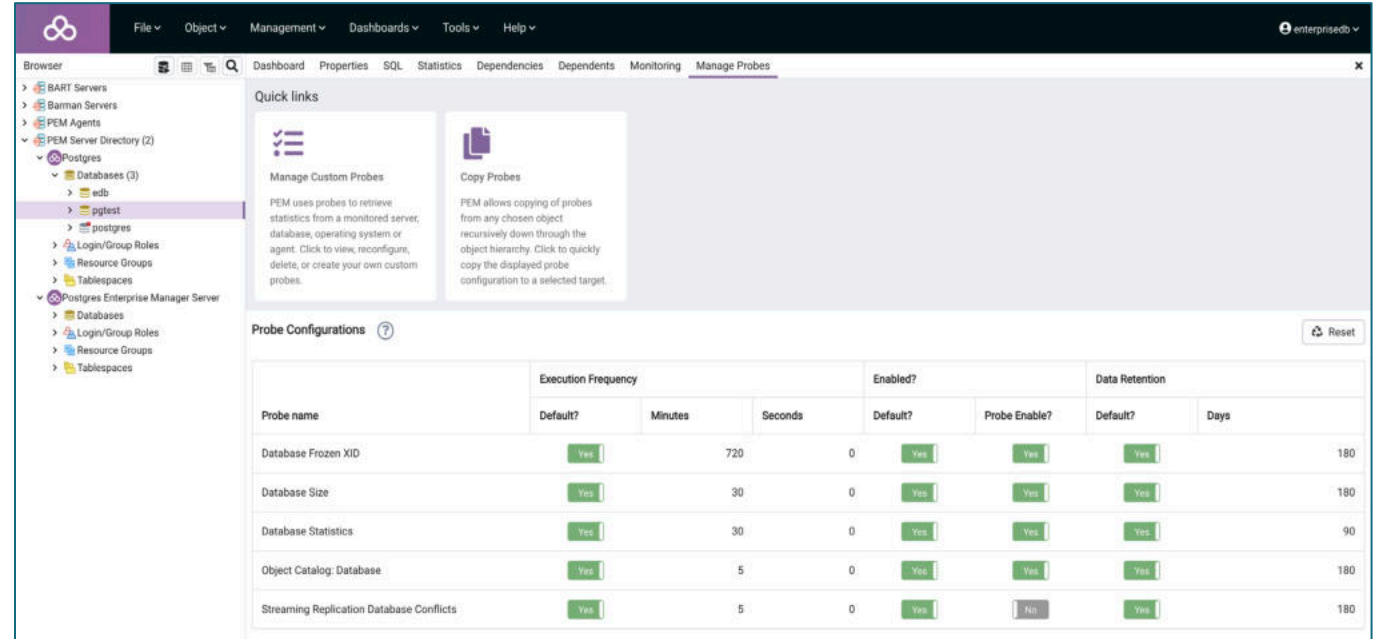
Register a Server

- Use the Create - Server dialog to register a new server connection



Managing Probes

- The **Manage Probes** tab provides a set of Quick Links that you can use to create and manage probes.
- You can modify the properties of an existing probe and create custom probes using **Manage Probes** tab.



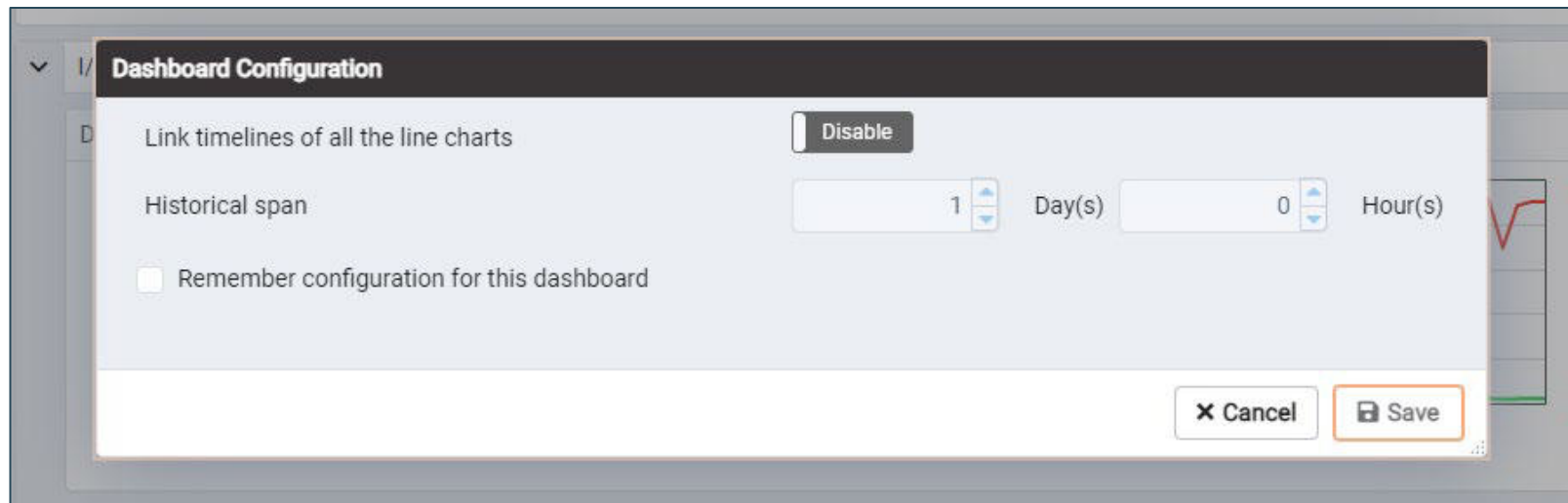
The screenshot shows the PostgreSQL Enterprise Manager (PEM) interface with the 'Manage Probes' tab selected. The left sidebar shows a tree view of the database structure, including BART Servers, Barman Servers, PEM Agents, and a PostgreSQL instance with various databases and tablespaces. The main content area has a 'Quick links' section with two cards: 'Manage Custom Probes' and 'Copy Probes'. Below this is a 'Probe Configurations' table with a 'Reset' button.

Probe name	Execution Frequency			Enabled?		Data Retention	
	Default?	Minutes	Seconds	Default?	Probe Enable?	Default?	Days
Database Frozen XID	☒	720	0	☒	☒	☒	180
Database Size	☒	30	0	☒	☒	☒	180
Database Statistics	☒	30	0	☒	☒	☒	90
Object Catalog: Database	☒	5	0	☒	☒	☒	180
Streaming Replication Database Conflicts	☒	5	0	☒	☐	☒	180



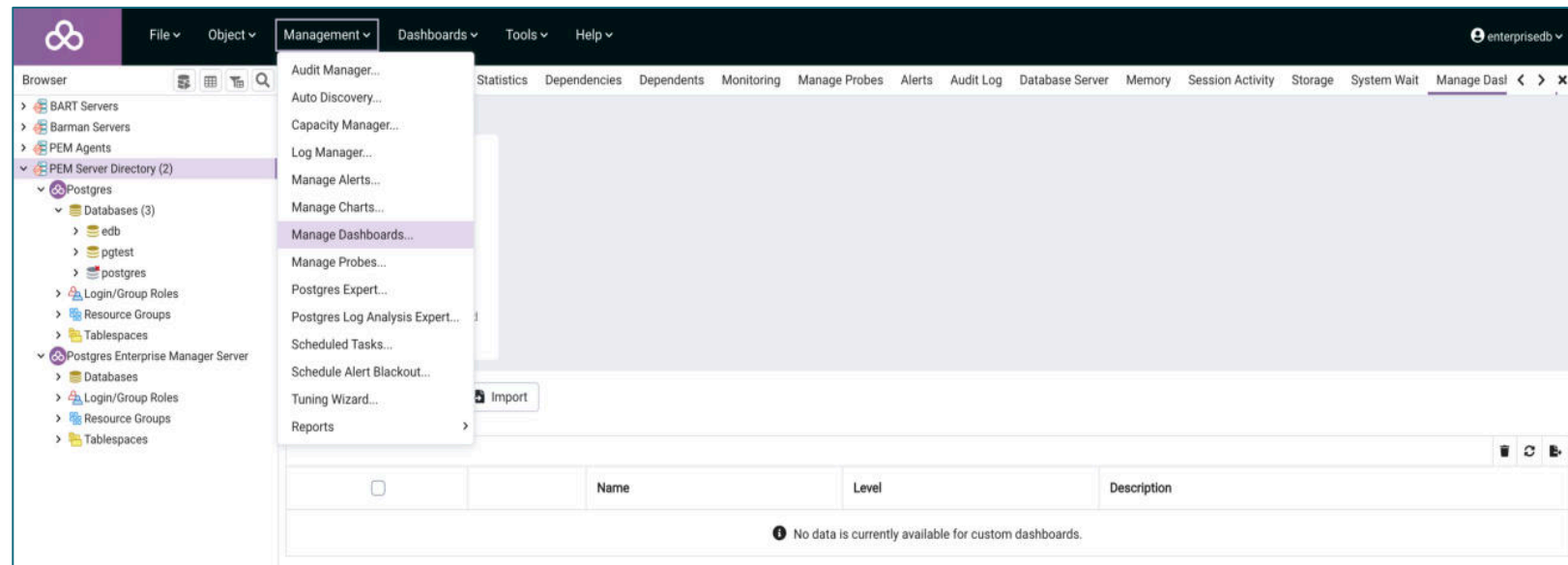
PEM Dashboards

- Postgres Enterprise Manager uses metrics (retrieved by probes) to generate the statistical information displayed on the dashboards.
- Dashboards use Tables, Pie charts, Bar and Line graphs to display statistical data.
- Dashboards Configuration

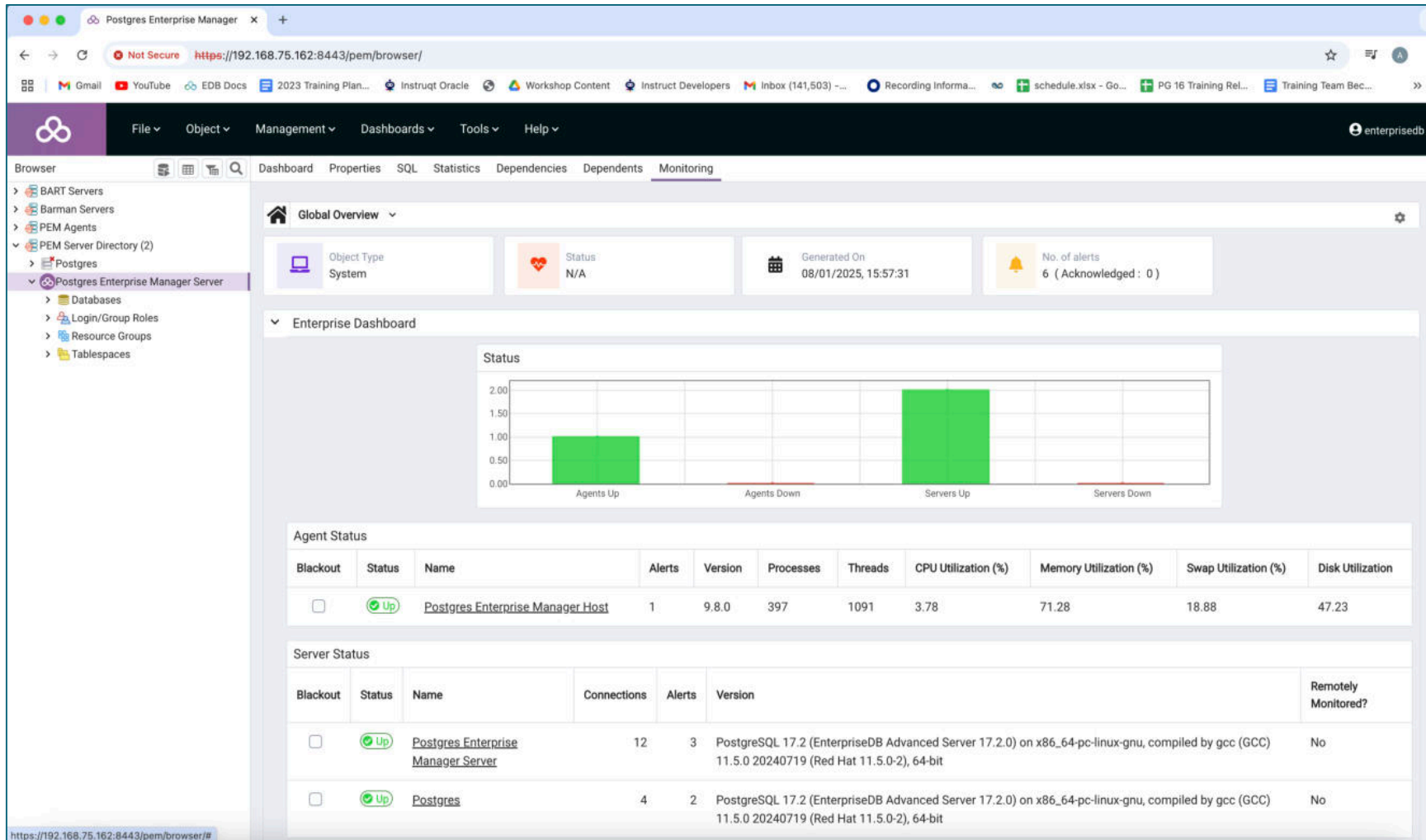


Opening Dashboard

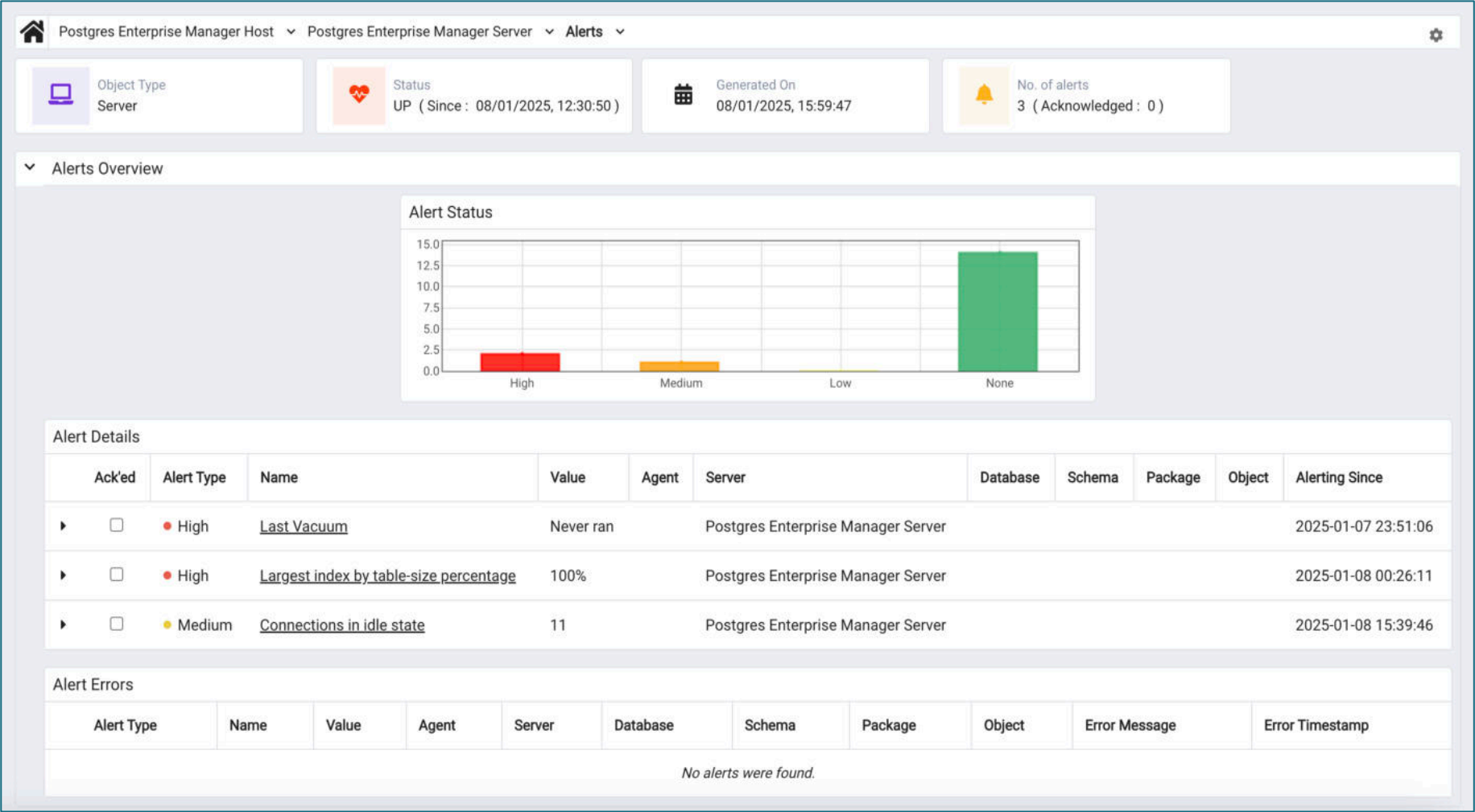
- PEM Dashboard menu bar option can be used to open available dashboard
- Dashboard menu automatically display available dashboard based on selection in Browser tree control
- Use Manage Dashboard dialog from Management menu to create a custom dashboard



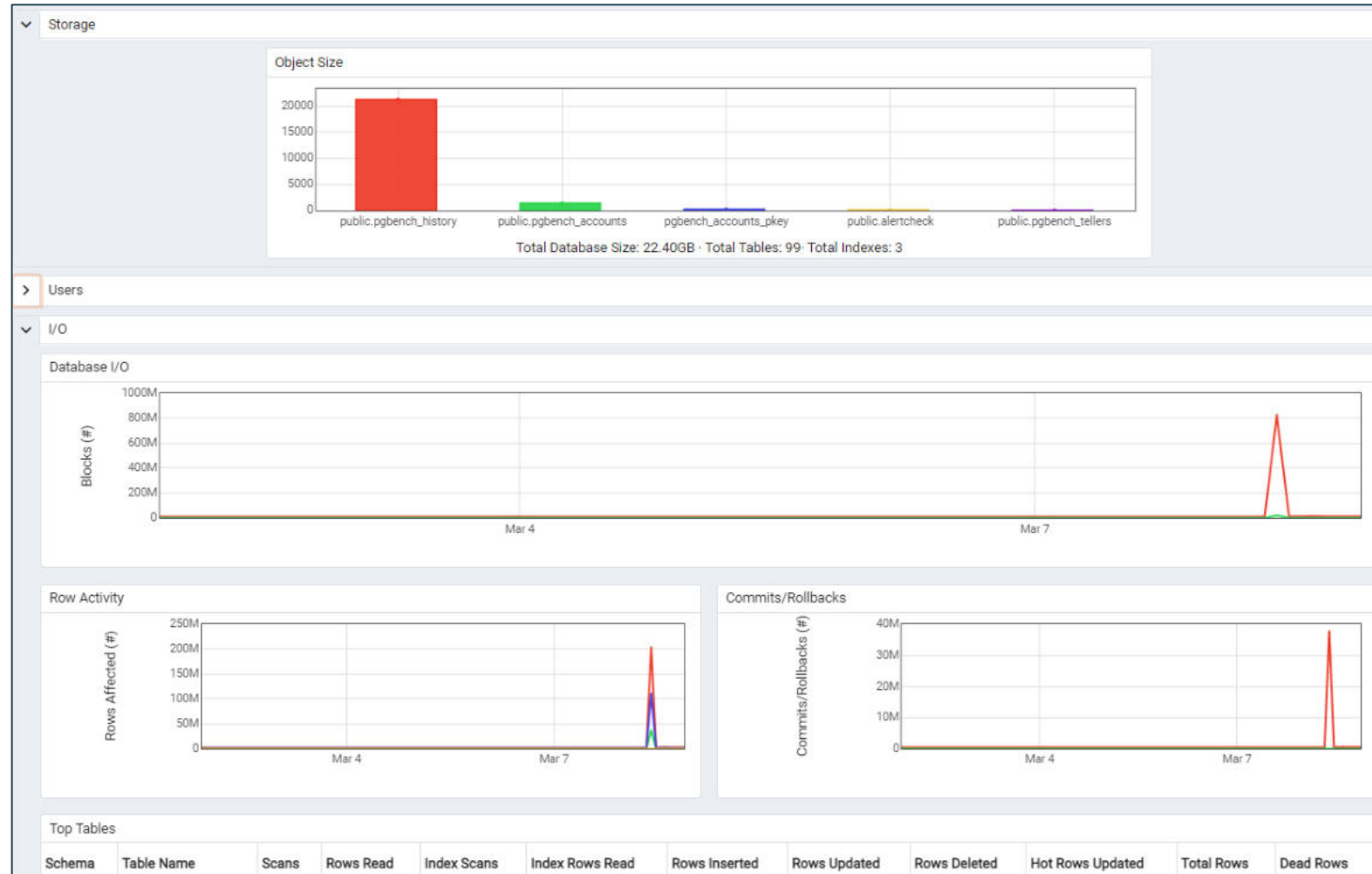
Global Overview Dashboard



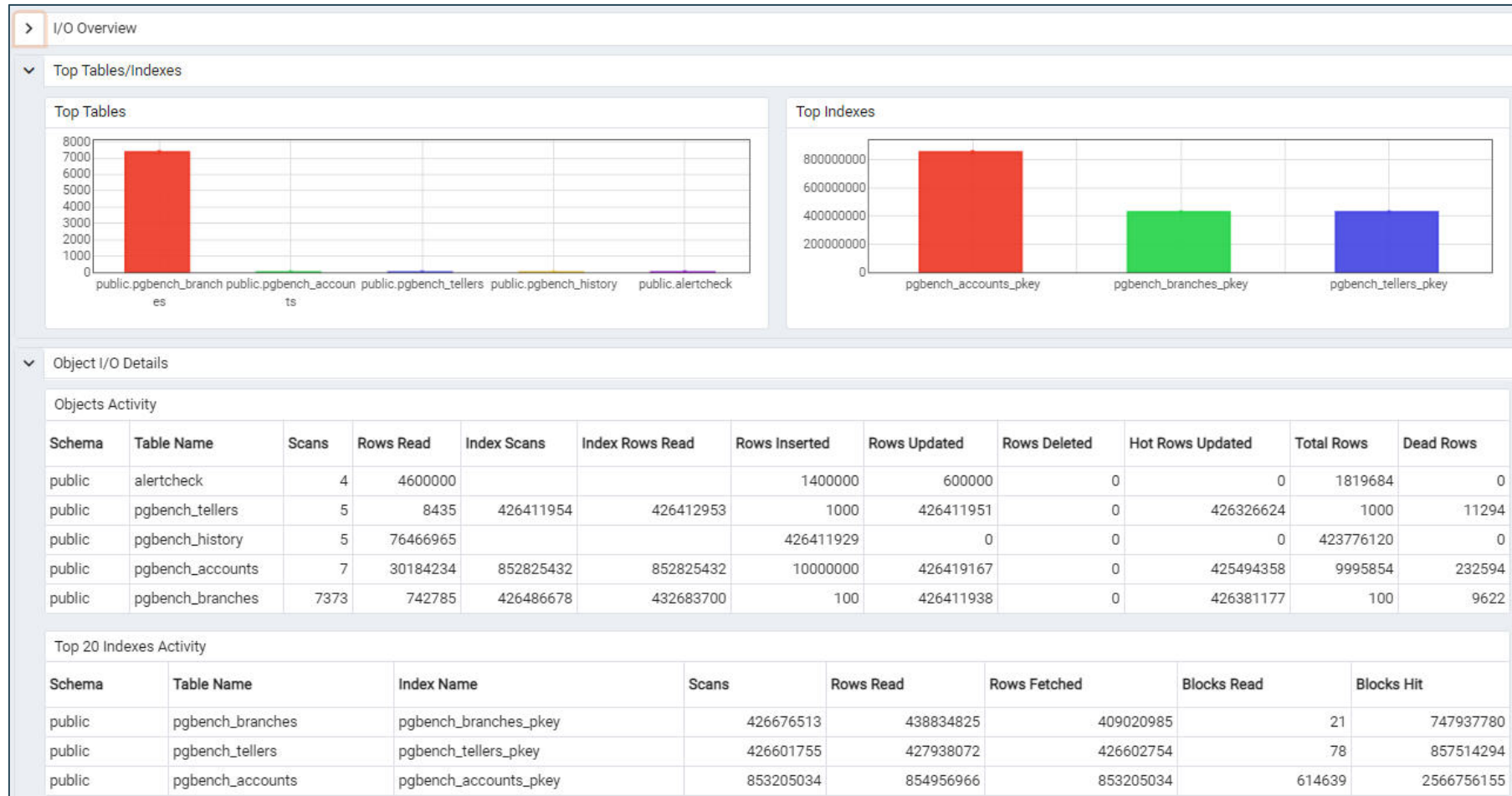
Alerts Dashboard



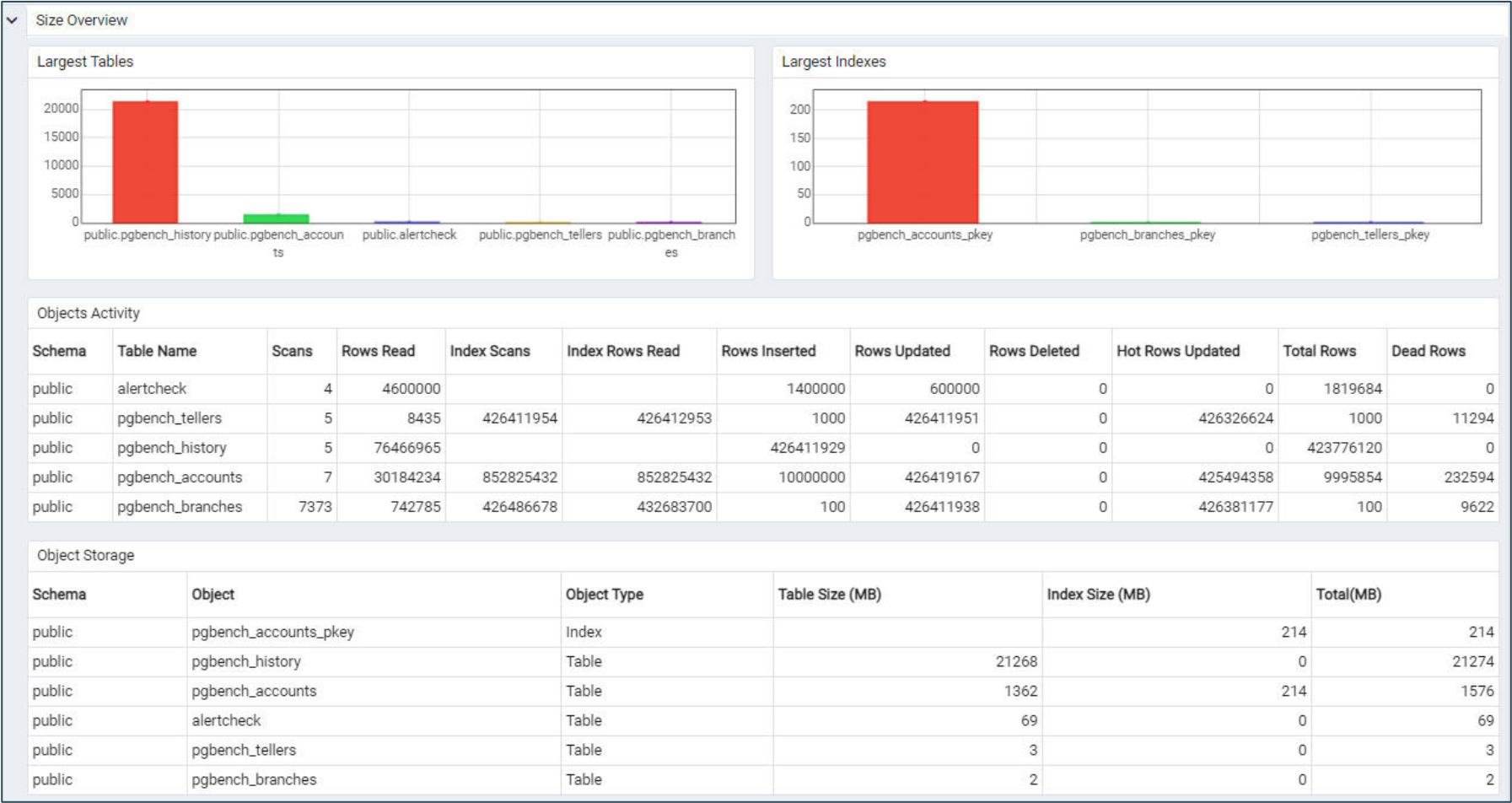
Database Analysis Dashboard



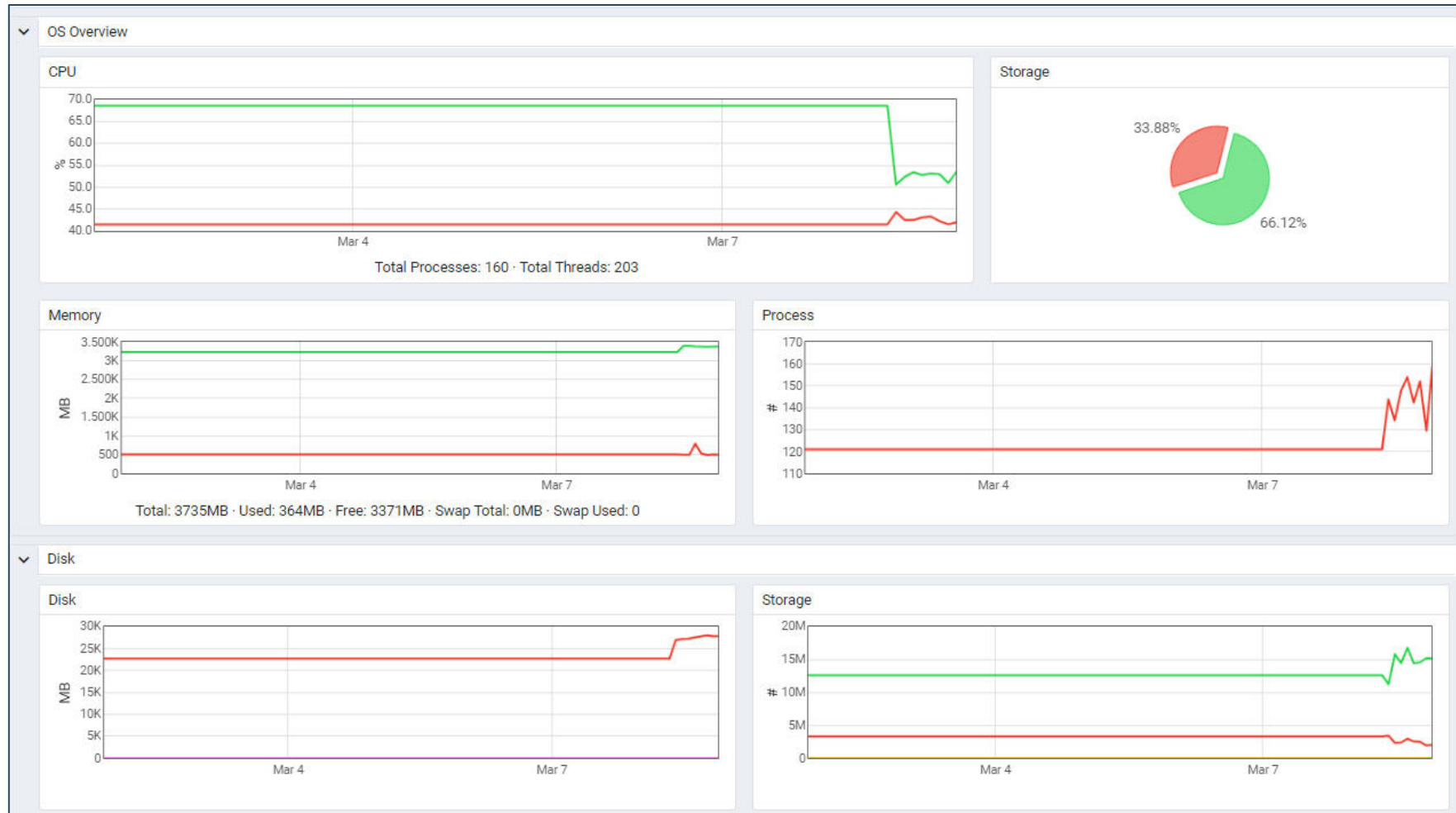
I/O Analysis Dashboard



Object Activity Analysis Dashboard



Operating System Analysis Dashboard



Server Log Analysis Dashboard

Postgres Enterprise Manager Host

EPAS-17

Server Log

Object Type

Server

Status

UP (Since : 14/01/2025, 16:17:01)

Generated On

14/01/2025, 21:00:31

No. of alerts

3 (Acknowledged : 0)

Server Logs

id	Timestamp	User Name	Database Name	Process ID	Session ID	Transaction ID	Connection From	Command	Message
80496	14/01/2025, 21:00:07	enterprisedb	postgres	143535	678682ff.230af	0	127.0.0.1:35764	authentication	connection authorized: user=enterprisedb database=postgres application_name=Postgres Enterprise Manager - Agent Monitoring
80495	14/01/2025, 21:00:07	enterprisedb	postgres	143535	678682ff.230af	0	127.0.0.1:35764	authentication	connection authenticated: user="enterprisedb" method=trust (/home/enterprisedb/data17/pg_hba.conf: 119)
80494	14/01/2025, 21:00:07			143535	678682ff.230af	0	127.0.0.1:35764		connection received: host=127.0.0.1 port=35764
80493	14/01/2025, 21:00:07	enterprisedb	postgres	143532	678682ff.230ac	0	127.0.0.1:35758	authentication	connection authorized: user=enterprisedb database=postgres application_name=Postgres Enterprise Manager - Agent Monitoring

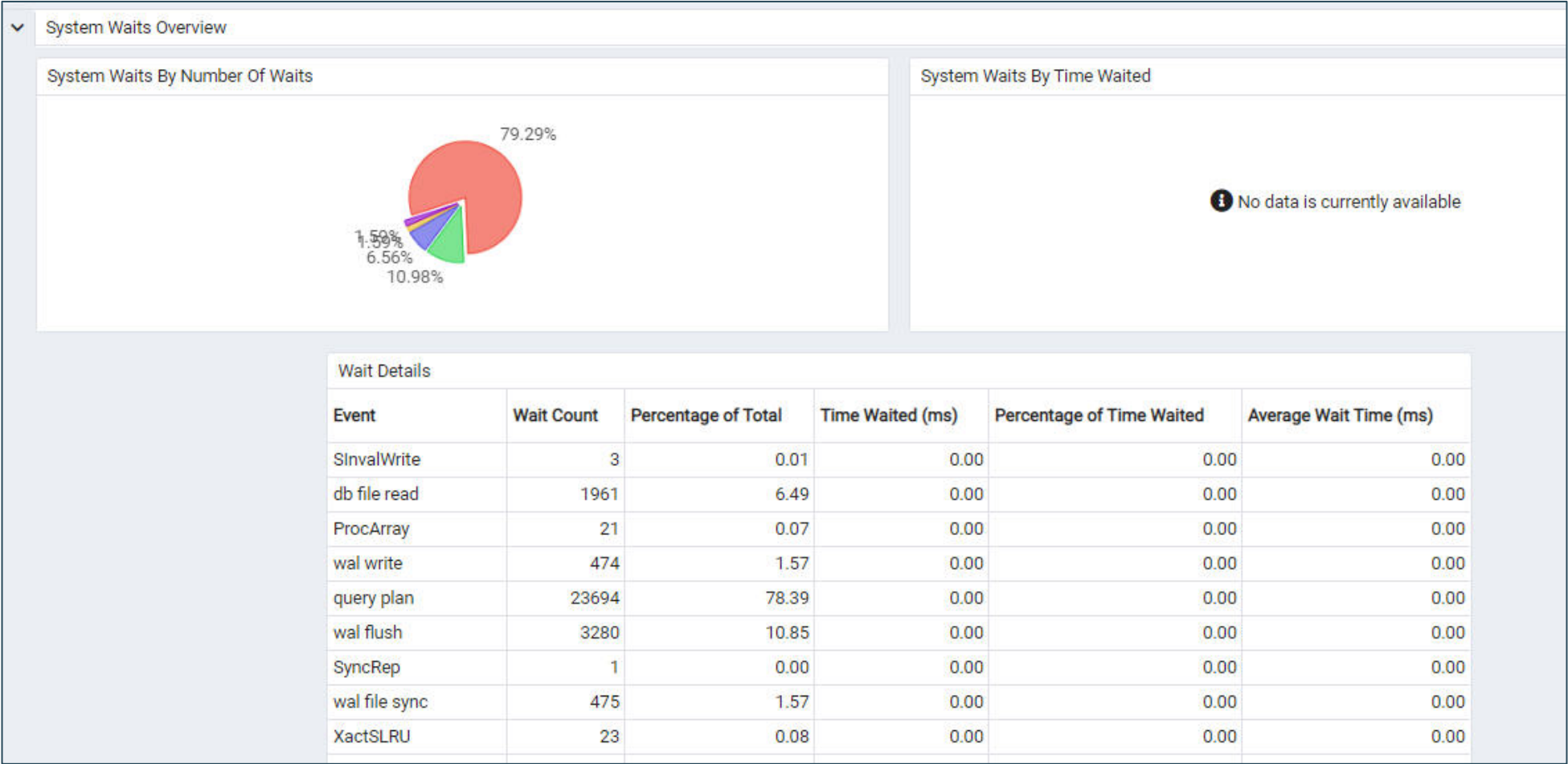


Session Activity Analysis Dashboard

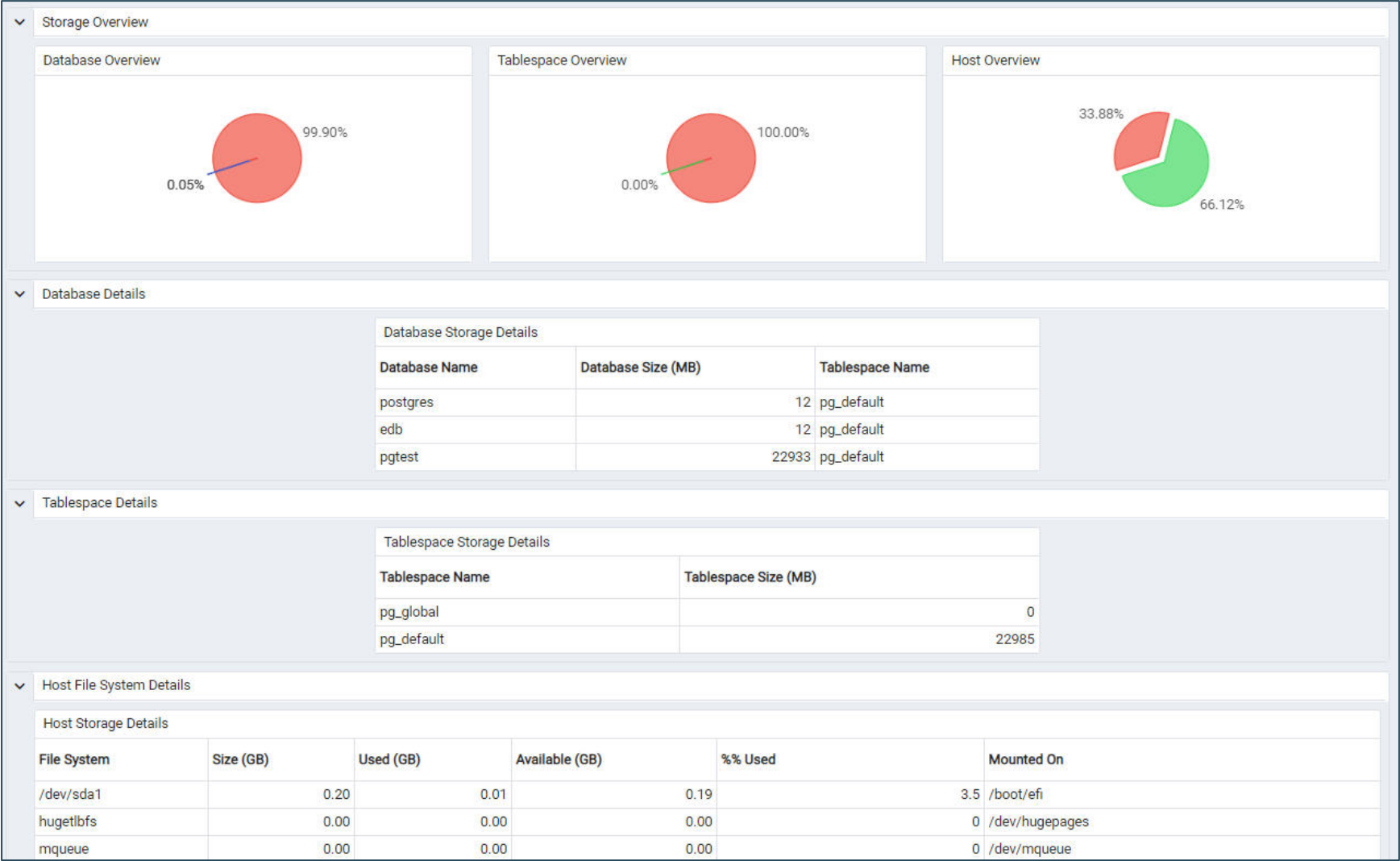
Home srv1 > Postgres > Session Activity >												
Object Type Server Status UP (Since : 3/8/2021, 9:10:08 AM) Generated On 3/8/2021, 9:45:40 PM No. of alerts 7 (Acknowledged : 0)												
Session Activity												
Work Load												
Session ID	User Name	Source	Database Name	Waiting?	Backend Start	Transaction Start	Query Start	Memory Usage	Swap Usage	CPU Usage	IO Reads (# bytes)	IO Writes (# bytes)
149176	enterprisedb	10.138.15.206:56974	edb	Yes	2021-03-08 21:36:10.619379-07		2021-03-08 21:39:08.805362-07	42.824	0.000	0.000	10942642.000	945.000
149757	enterprisedb	10.138.15.206:57122	pgtest	Yes	2021-03-08 21:38:41.475127-07		2021-03-08 21:42:41.536573-07	45.410	0.000	0.000	14930666.000	13239.000
147403	enterprisedb	10.138.15.207:41834	edb	Yes	2021-03-08 21:28:00.23231-07		2021-03-08 21:42:23.551698-07	15.492	0.000	0.000	227496.000	1000.000
150575	enterprisedb	10.138.15.207:43078	edb	No	2021-03-08 21:42:42.068399-07	2021-03-08 21:42:42.114325-07	2021-03-08 21:42:42.114325-07	20.621	0.000	0.000	227495.000	998.000
150576	enterprisedb	10.138.15.207:43080	pgtest	No	2021-03-08 21:42:42.074958-07			0.000	0.000	0.000	0.000	0.000
959				Yes	2021-03-08 09:10:09.036414-07			21.203	0.000	0.000	2777.000	525789913.000
962	enterprisedb			Yes	2021-03-08 09:10:09.037253-07			8.355	0.000	0.000	539.000	258.000
963	enterprisedb		postgres	Yes	2021-03-08 09:10:09.037569-07			17.738	0.000	0.000	615427.000	9895029.000



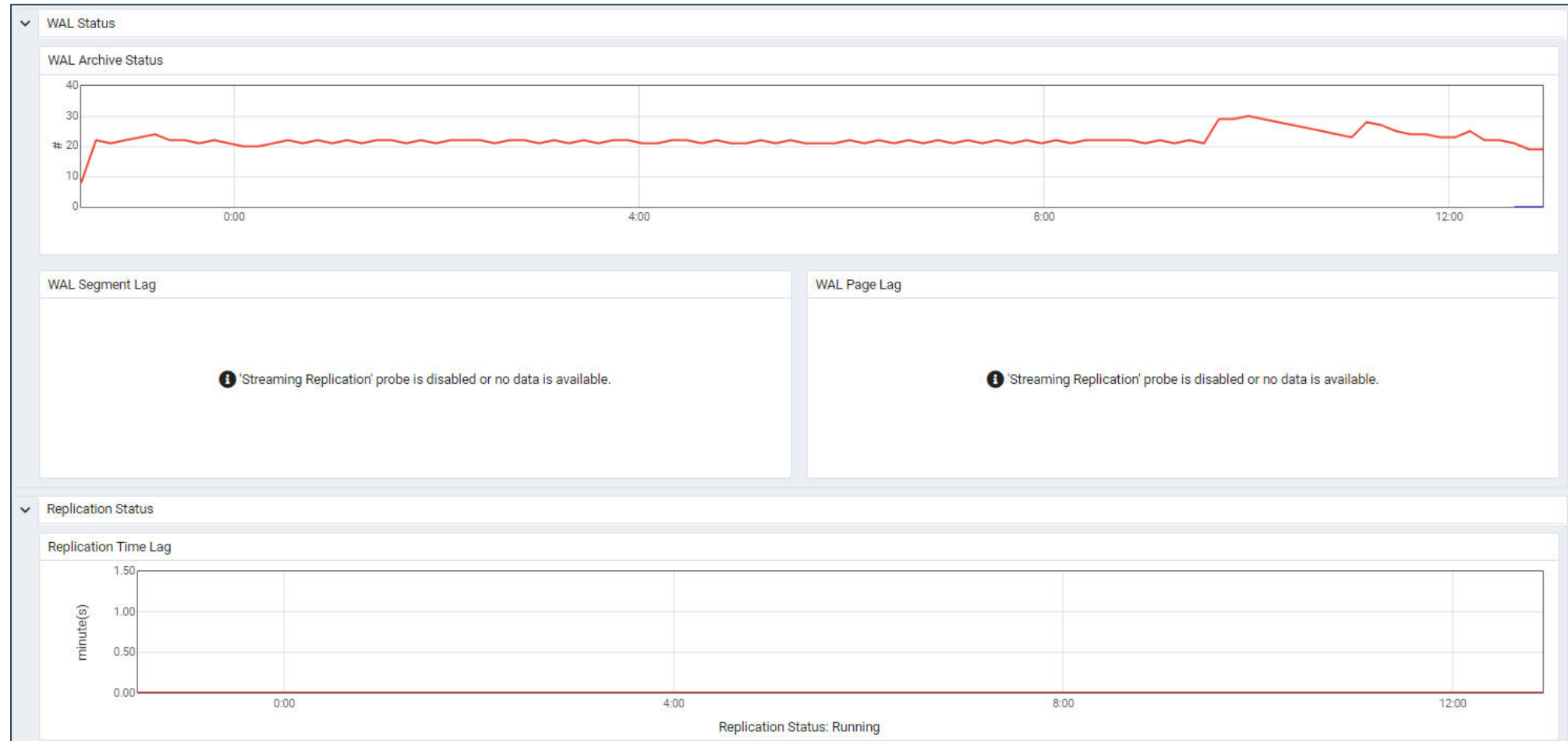
System Wait Analysis Dashboard



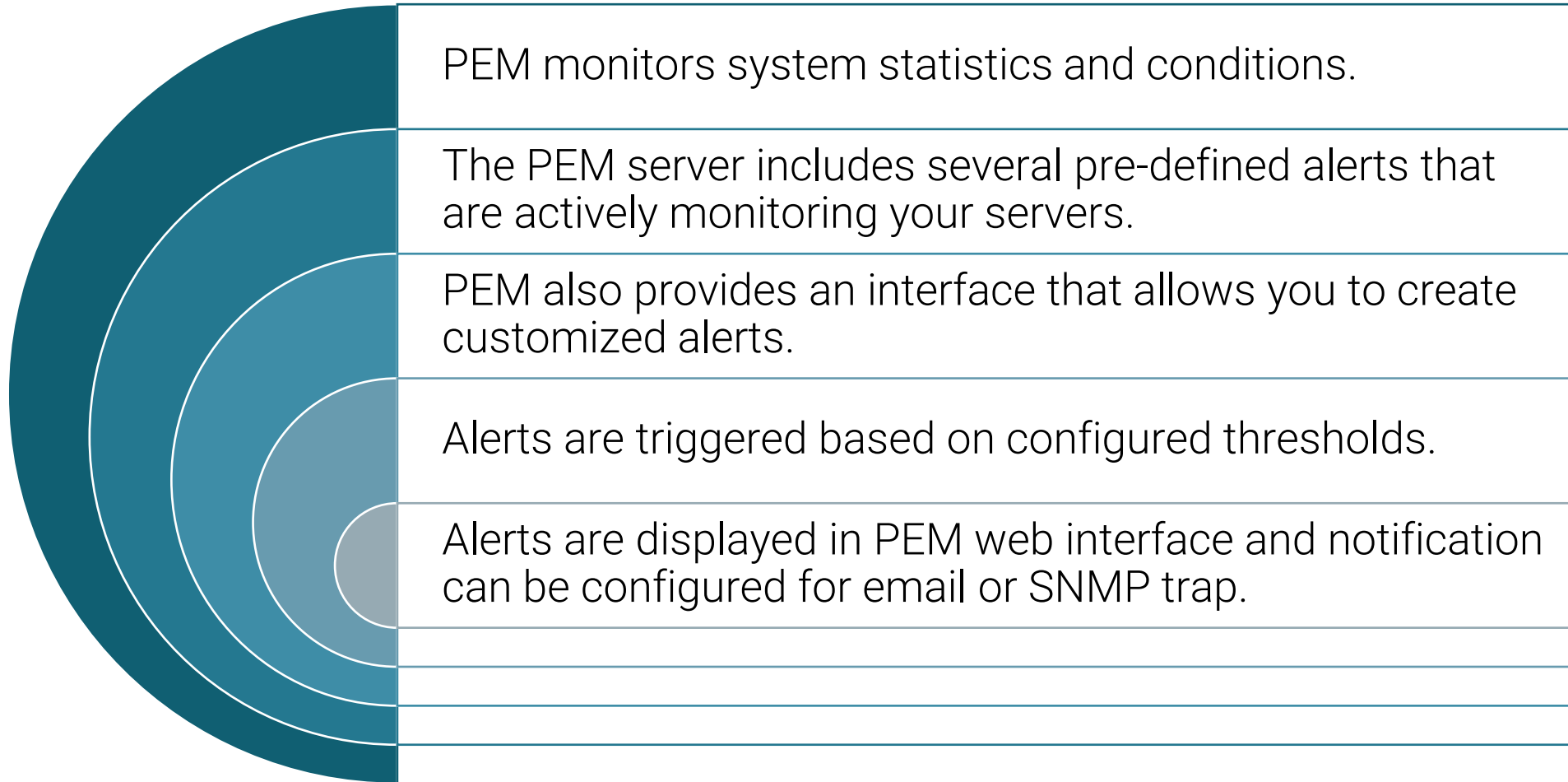
Storage Analysis Dashboard



Streaming Replication Analysis Dashboard



Postgres Enterprise Manager Alerting



Managing Alerts

- Manage Alerts tab allows you to define custom alerts or modify existing alerts.
- Click on Manage Alerts... from the Management menu
- Click any PEM agent or database server in Browser Tree to view default alerts.

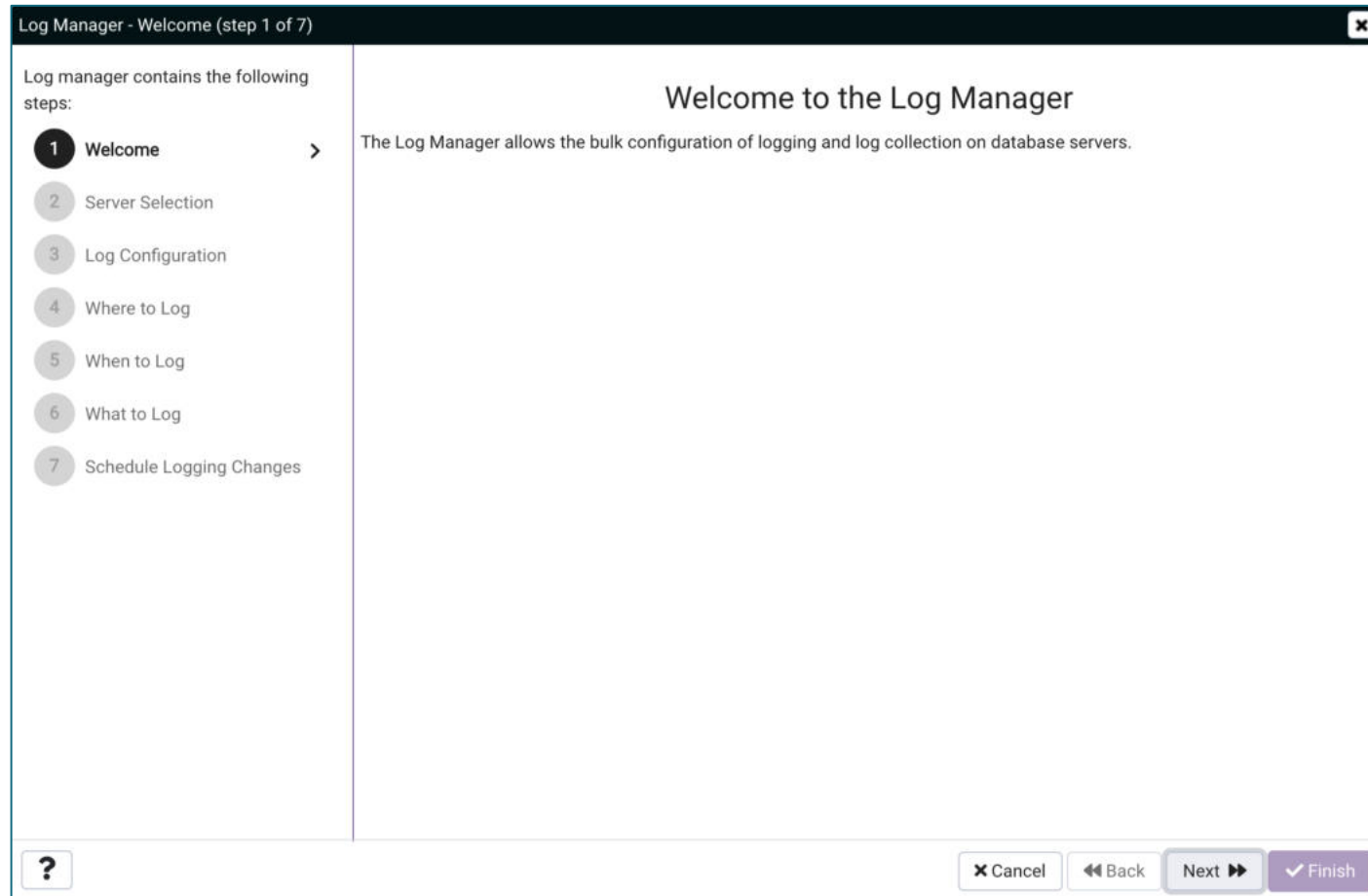
The screenshot shows the PostgreSQL Enterprise Manager (PEM) interface. On the left is a 'Browser' tree with a hierarchy: BART Servers, Barman Servers, PEM Agents, PEM Server Directory (2), CEPAS-17, Databases, Login/Group Roles, Resource Groups, Tablespaces, and PostgreSQL Enterprise Manager Server. The main panel is titled 'Manage Alerts' and contains a 'Quick links' section with icons for Copy Alerts, Alert Templates, Email Templates, Email Groups, Webhooks, and Server Configuration. Below this is a table of configured alerts.

Name	Auto created?	Template	Enable?	Interval		History retention		
				Default?	Minutes	Default?	Days	
☒ Audit config mismatch	☒ Yes	Audit config mismatch	☒ Yes	☒ Yes		1	☒ Yes	30
☒ Conflicting replication slots	☒ Yes	Conflicting replication slots	☒ Yes	☒ Yes		1	☒ Yes	30
☒ Connections in idle-in-transaction state	☒ Yes	Connections in idle-in-transaction state	☒ Yes	☒ Yes		1	☒ Yes	30
☒ Connections in idle-in-transaction state, as a ...	☒ Yes	Connections in idle-in-transaction sta...	☒ Yes	☒ Yes		1	☒ Yes	30
☒ Connections in idle state	☒ Yes	Connections in idle state	☒ Yes	☒ Yes		1	☒ Yes	30
☒ Inactive replication slots	☒ Yes	Inactive replication slots	☒ Yes	☒ Yes		1	☒ Yes	30



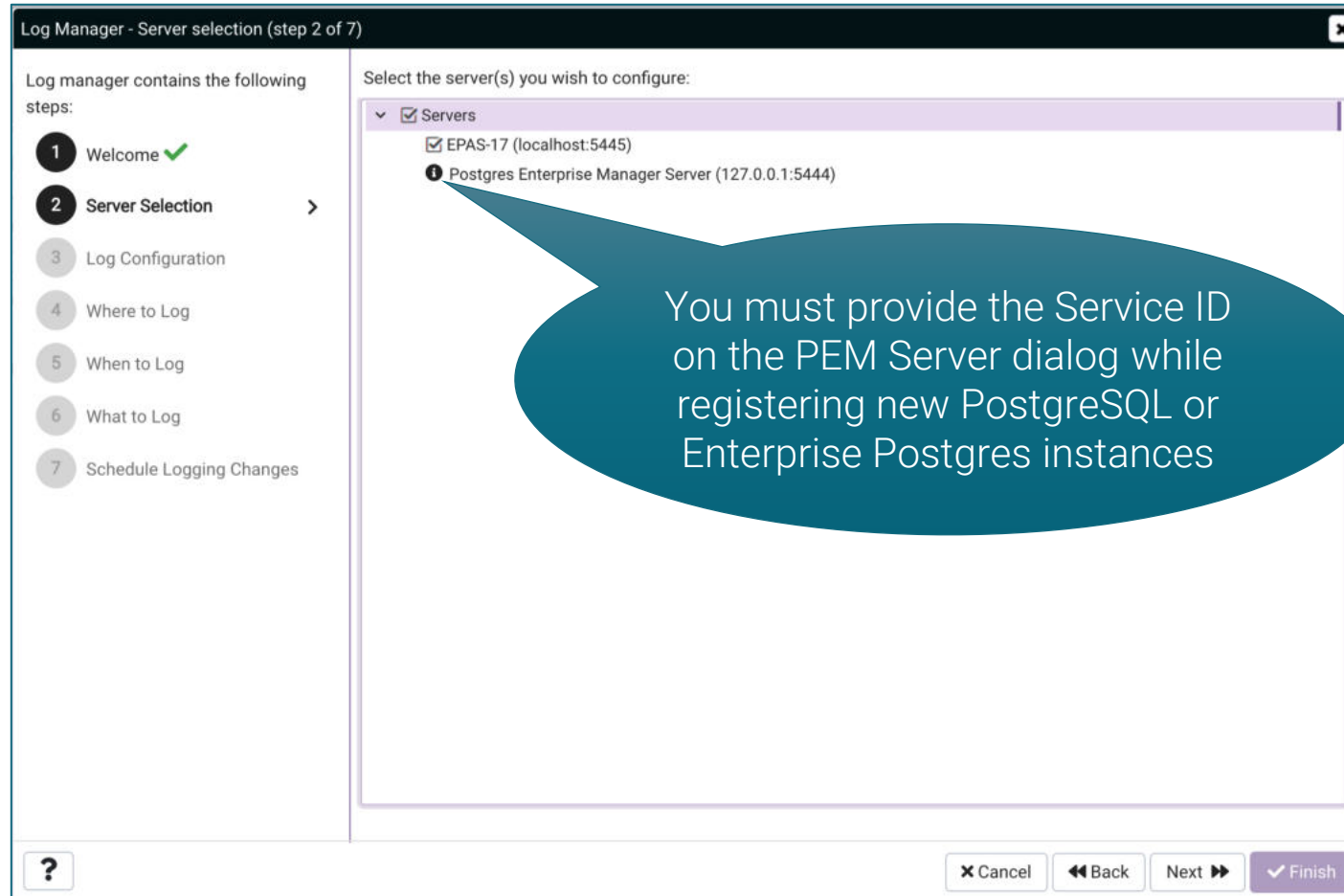
Log Manager

PEM Log manager can be used to configure, enable, and disable error logging for Postgres instances



Log Manager – Select Servers

Postgres Instances can be selected for configuring logging



Log Manager – Log Settings

Specify configuration details for each server

Log Manager - Log configuration (step 3 of 7)

Log manager contains the following steps:

- 1 Welcome ✓
- 2 Server Selection ✓
- 3 Log Configuration >
- 4 Where to Log
- 5 When to Log
- 6 What to Log
- 7 Schedule Logging Changes

Import logs

Import logs to PEM? ☒ Yes

Import frequency 12 Hours

Log rotation configuration

Rotation size (MB) 20
Automatic rotation of log files will happen after that much log output. 0 disables.

Rotation time (days) 1
Automatic rotation of log files will happen after that time. 0 disables.

Truncate on rotation? ☒ Yes
If on, an existing log file with the same name as the new log file will be truncated rather than appended to. Such truncation only occurs on time-driven rotation.

?



Log Manager – Where, When and What to Log

Configure log settings for each server

Log Manager - Where to log (step 4 of 7)

Log manager contains the following steps:

- Welcome ✓
- Server Selection ✓
- Log Configuration ✓
- Where to Log ✓
- When to Log
- What to Log
- Schedule Logging Changes

Log destination

stderr? ☒ Yes

syslog? ☐ No

csvlog? ☒ Yes

eventlog? ☐ No

Log collection

Log collector ☒ Enable

Log silent mode ☐ Disable

Log collector parameter requires server restart

Log directory

Change log directory for selected servers? ☐ No

Directory name

Log file name

Syslog facility

Syslog ident

When logging to syslog is enabled, this parameter determines the syslog "facility" to be used.

When logging to syslog is enabled, this parameter determines the program name used to identify PostgreSQL messages in syslog logs.

Cancel Back Next Finish

Log Manager - When to log (step 5 of 7)

Log manager contains the following steps:

- Welcome ✓
- Server Selection ✓
- Log Configuration ✓
- Where to Log ✓
- When to Log ✓
- What to Log
- Schedule Logging Changes

Client min messages

Message levels sent to the client:

Log min messages

Message levels written to the server log:

Log min error statement

SQL statements that cause an error condition are recorded in the server log.

Log min duration statement (ms)

Logs only statements running at least this number of milliseconds. -1: disabled, 0 logs all with durations.

Log temp files (kilobytes)

Log temporary files equal or larger than the specified size; -1 disables, 0 logs all temp files.

Log AutoVacuum min duration (ms)

If auto-vacuuming exceeds the length of time specified, the activity will be logged. The default value is -1.

Cancel Back Next Finish

Log Manager - What to log (step 6 of 7)

Log manager contains the following steps:

- Welcome ✓
- Server Selection ✓
- Log Configuration ✓
- Where to Log ✓
- When to Log ✓
- What to Log ✓
- Schedule Logging Changes

Debug options

Parse tree? ☐ No

Execution plan? ☐ No

Reewriter output? ☐ No

Indent debug options output in log? ☒ Yes

General options

Checkpoints? ☒ Yes

Connections? ☒ Yes

Disconnections? ☐ No

Duration? ☐ No

Hostname? ☐ No

Lock waits? ☒ Yes

Error verbosity

Level of message detail written to the server log.

Prefix string

Use the Prefix String field to specify a printf-style string that is written at the beginning of each log file entry.

Statements

Controls which SQL statements are logged.

Cancel Back Next Finish



Log Manager – Apply Changes

Configure schedule or apply changes now

Log Manager - Schedule logging changes (step 7 of 7)

Log manager contains the following steps:

- 1 Welcome ✓
- 2 Server Selection ✓
- 3 Log Configuration ✓
- 4 Where to Log ✓
- 5 When to Log ✓
- 6 What to Log ✓
- 7 Schedule Logging Changes >

Configure logging now? ☒ Yes

Time

Note: The selected servers will be restarted to complete log configuration.

?



Server Log Dashboard

Use Server Log dashboard to view logged information

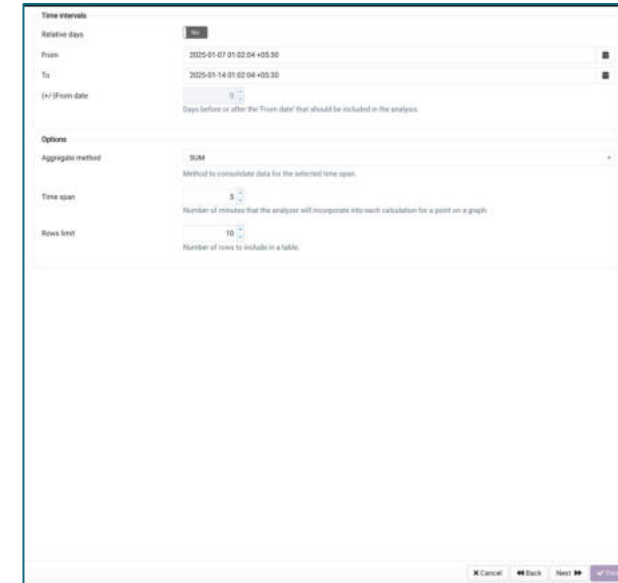
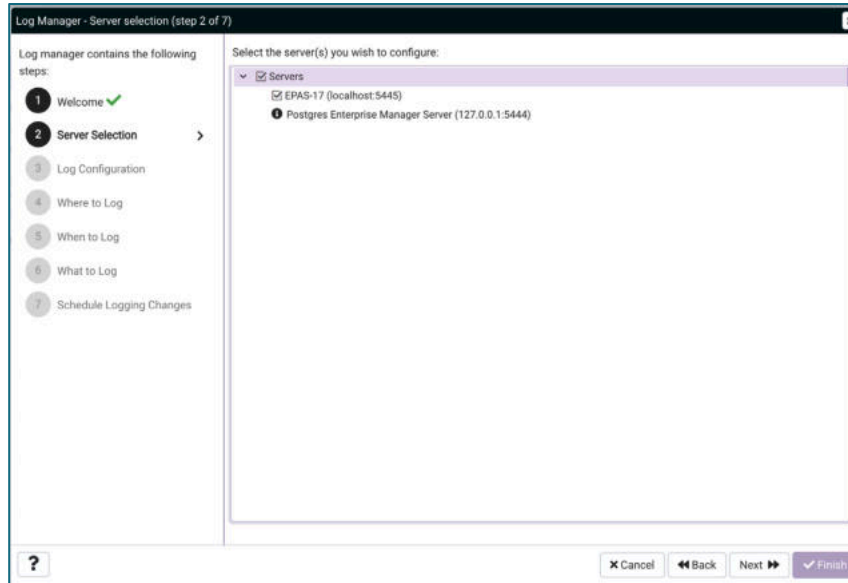
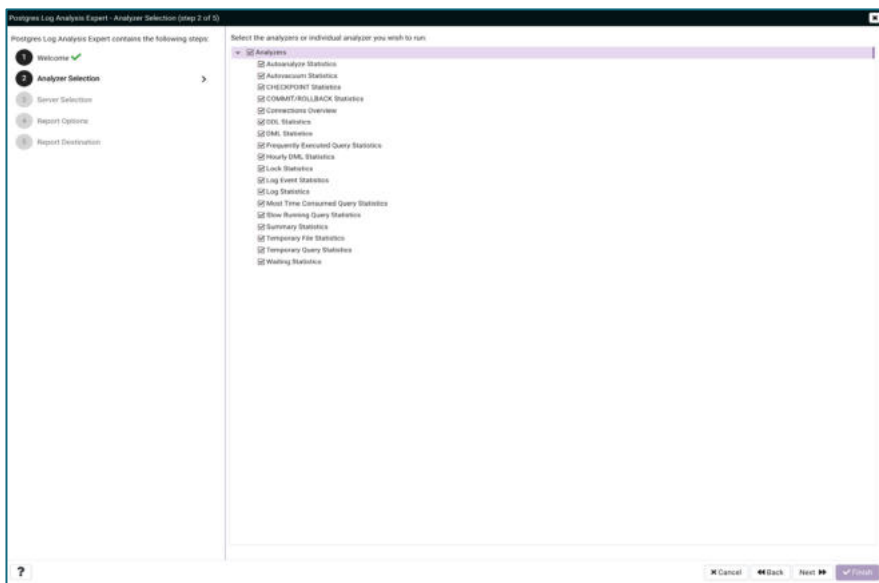
Server Logs

id	Timestamp	User Name	Database Name	Process ID	Session ID	Transaction ID	Connection From	Command	Message
10006	14/01/2025, 01:15:03	enterprisedb	edb	111740	67856d3f.1b47c	0	127.0.0.1:35228	authentication	connection authorized: user=enterprisedb database=edb application_name=Postgres Enterprise Manager - Agent Monitoring
10005	14/01/2025, 01:15:03	enterprisedb	edb	111740	67856d3f.1b47c	0	127.0.0.1:35228	authentication	connection authenticated: user='enterprisedb' method=trust (/home/enterprisedb/data17/ pg_hba.conf:119)
10004	14/01/2025, 01:15:03			111740	67856d3f.1b47c	0	127.0.0.1:35228		connection received: host=127.0.0.1 port=35228
10003	14/01/2025, 01:15:03	enterprisedb	edb	111738	67856d3f.1b47a	0	127.0.0.1:35222	authentication	connection authorized: user=enterprisedb database=edb application_name=Postgres Enterprise Manager - Agent Monitoring
10002	14/01/2025, 01:15:03	enterprisedb	edb	111738	67856d3f.1b47a	0	127.0.0.1:35222	authentication	connection authenticated: user='enterprisedb' method=trust (/home/enterprisedb/data17/ pg_hba.conf:119)
10001	14/01/2025, 01:15:03			111738	67856d3f.1b47a	0	127.0.0.1:35222		connection received: host=127.0.0.1 port=35222
10000	14/01/2025, 01:15:03	enterprisedb	edb	111736	67856d3f.1b478	0	127.0.0.1:35218	authentication	connection authorized: user=enterprisedb database=edb application_name=Postgres Enterprise Manager - Agent Monitoring



Postgres Log Analysis Expert

The Postgres Log Analysis Expert analyzes the log files of servers that are registered with PEM.



Sample Log Analysis Report

Postgres Log Analysis Expert

Interval: 2025-1-07 15:47:40 - 2025-1-14 15:47:40

Generated: 2025-01-14 15:52:30

Span: 5 Minutes

Aggregate: SUM

Go to:

EPAS-17

EPAS-17(localhost:5445)

Summary Statistics

Settings	Values
Number of unique queries	83
Total queries	83
Total queries duration	
First query	14/01/2025 15:26:17.019 IST
Last query	14/01/2025 15:45:13.813 IST
Queries peak time	14/01/2025 15:27:06 IST queries 10
Number of events	83
Number of unique events	1
Total number of sessions	74
Total duration of sessions	
Average sessions duration	(ms)
Total number of connections	6404
Total number of databases	0

Hourly DML Statistics

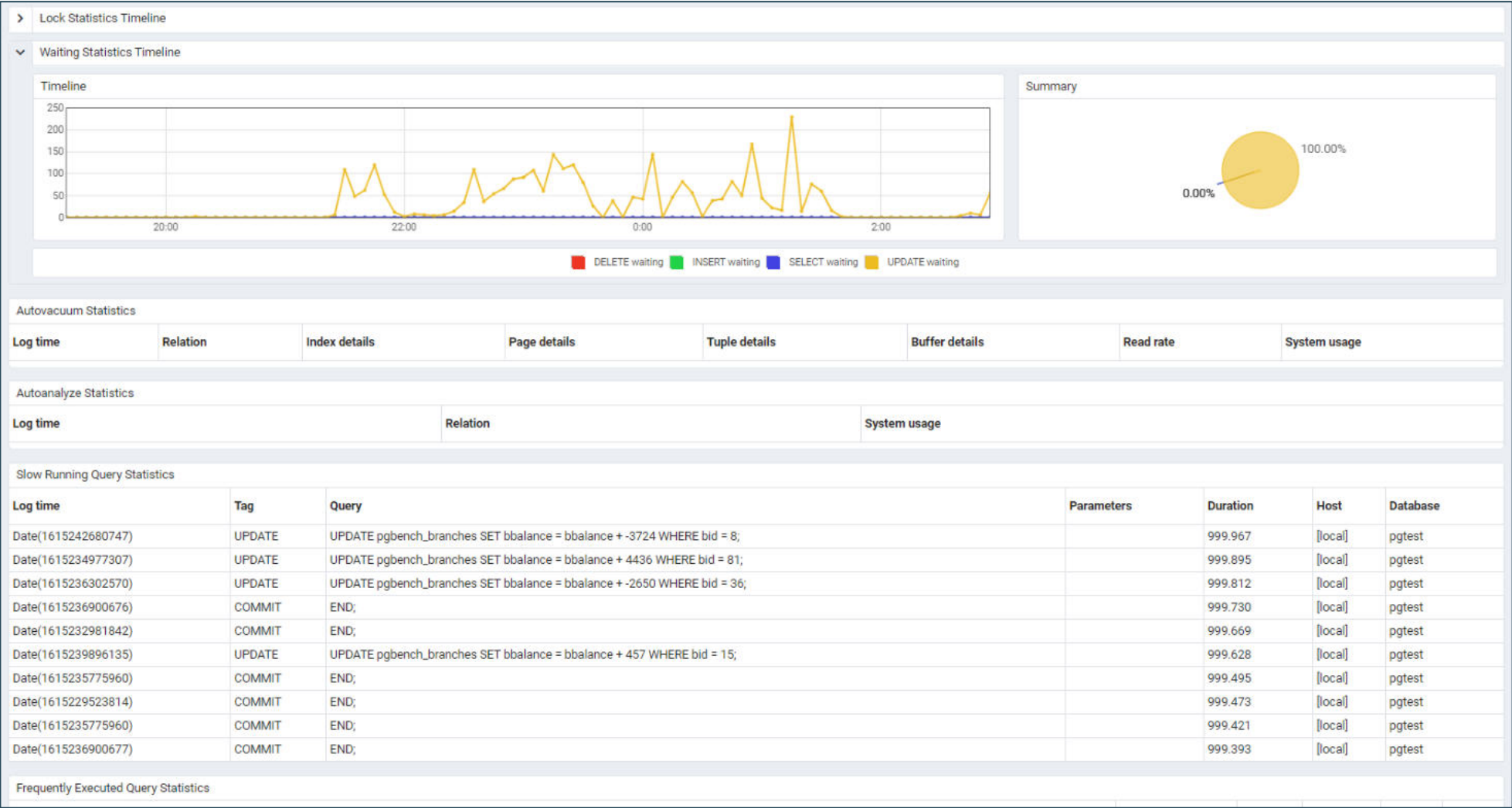
Time	Database name	Statement	Count	Min duration	Max duration	Avg duration
14/01/2025 15:00	edb	COPY	1	4243.29	4243.29	4243.29
14/01/2025 15:00	edb	SELECT	1	1929.60	1929.60	1929.60
14/01/2025 15:00	edb	UPDATE	19	500.06	5093.38	3586.51
14/01/2025 15:00	pgtest	COPY	1	4885.02	4885.02	4885.02
14/01/2025 15:00	pgtest	UPDATE	61	607.10	6812.88	1757.53



Sample Log Analysis Report

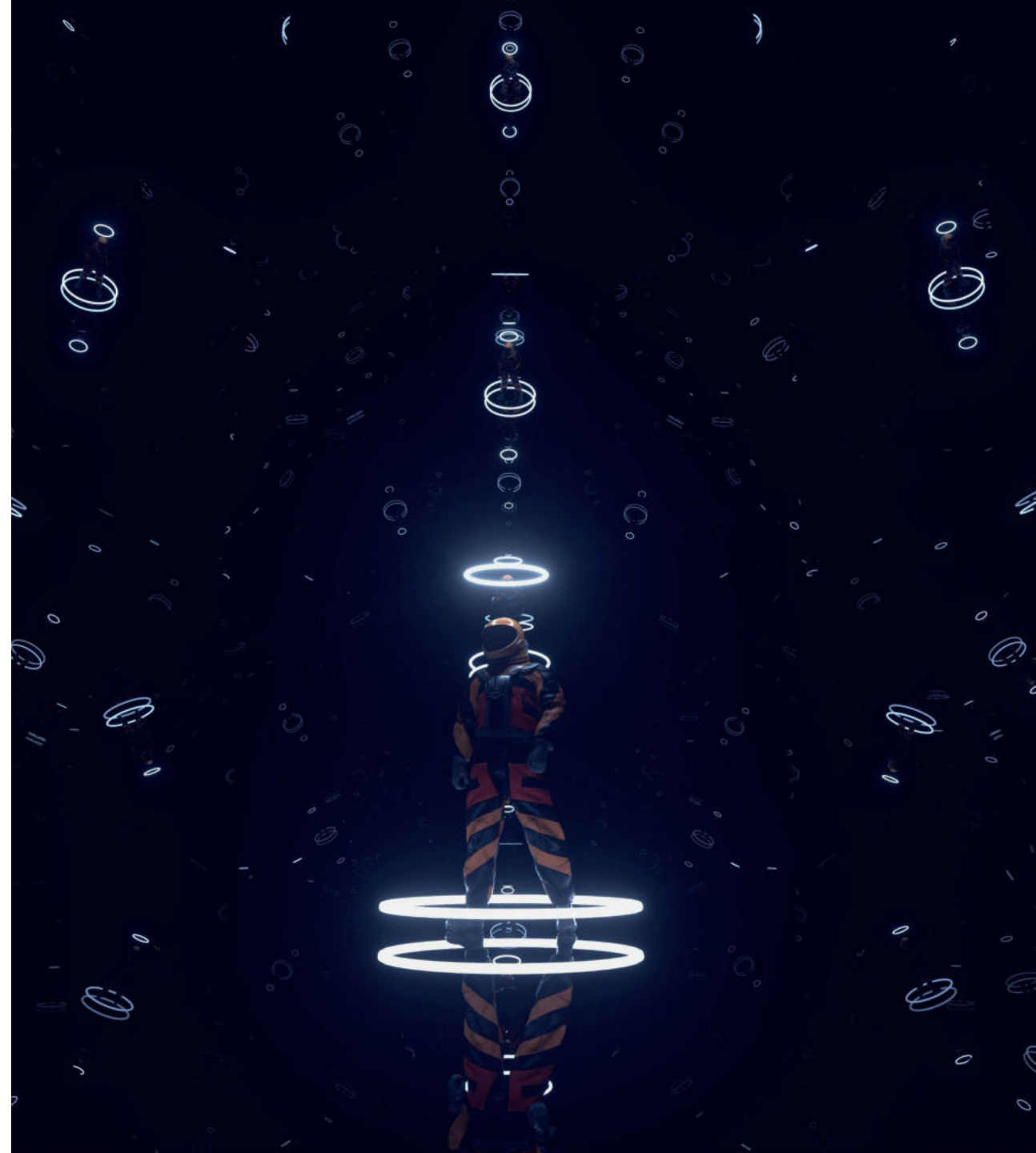


Sample Log Analysis Report



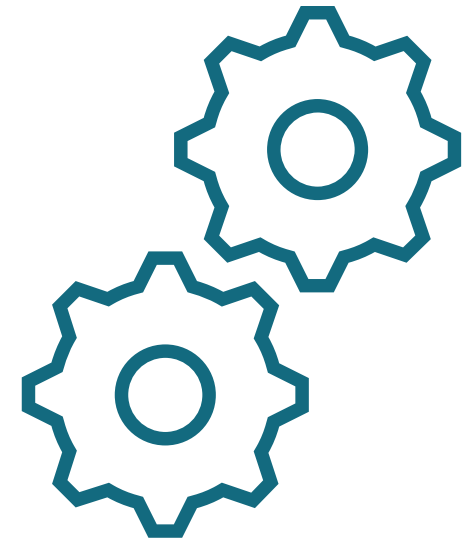
SQL Profiler

- SQL Profiler is a graphical wizard available in PEM
- It can be used to:
 - Capture workloads in a SQL trace
 - Monitor and analyze SQL
 - Diagnose slow-running queries
 - View the query execution statistics
 - Schedule a SQL trace to run during heavy workloads
 - Get advice on indexes
- SQL Profiler plug-in must be installed and configured for each Postgres database



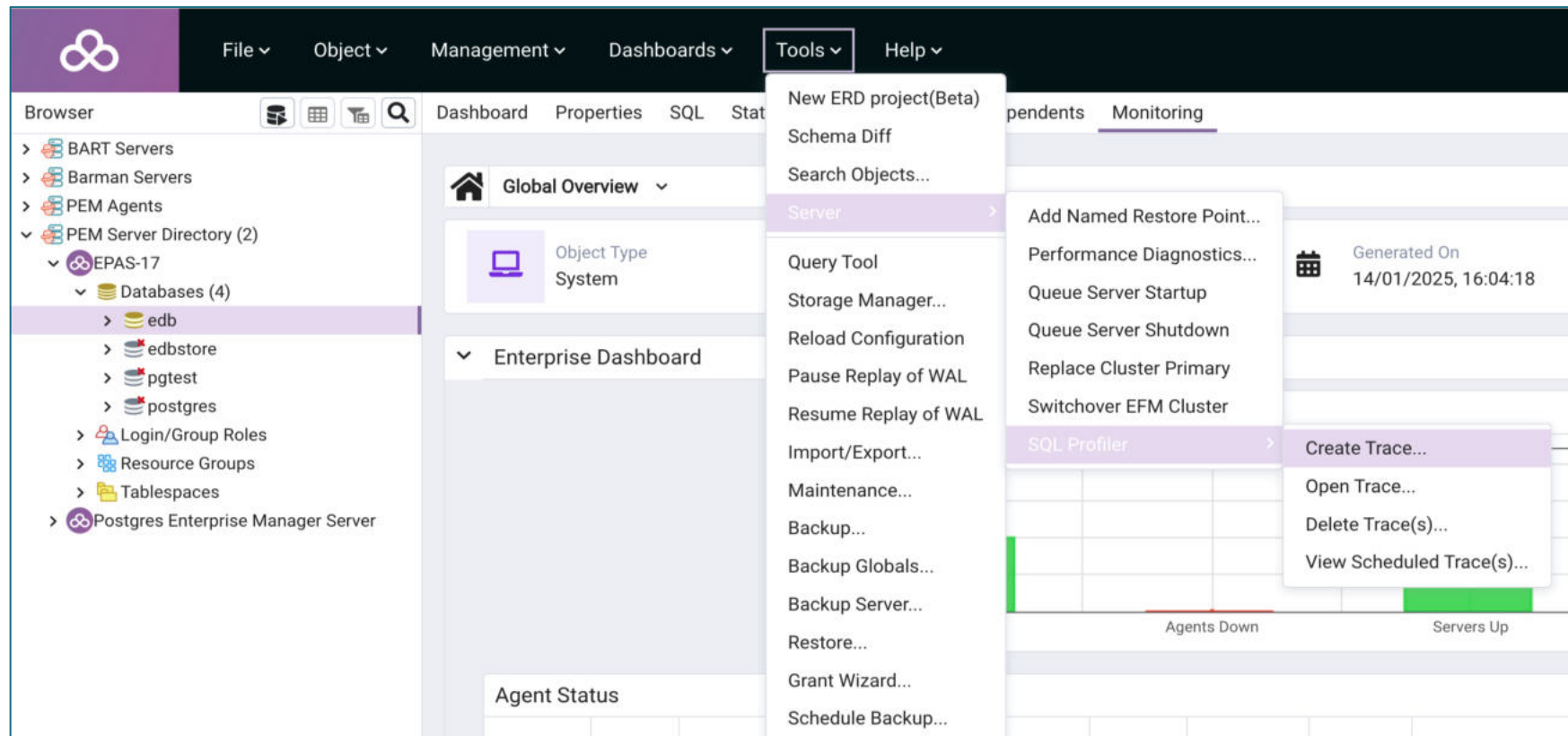
Configure SQL Profiler

- SQL Profiler can be used to trace workloads for Enterprise Postgres
- Install SQL Profiler package:
 - `sudo dnf -y install edb-as17-server-sqlprofiler`
- Required parameter change in postgresql.conf(requires restart):
 - `shared_preload_libraries = $libdir/sql-profiler`
- Enabling the extension in maintenance database:
 - `psql -p 5444 -d edb -U enterprisedb`
 - `CREATE EXTENSION sql_profiler;`



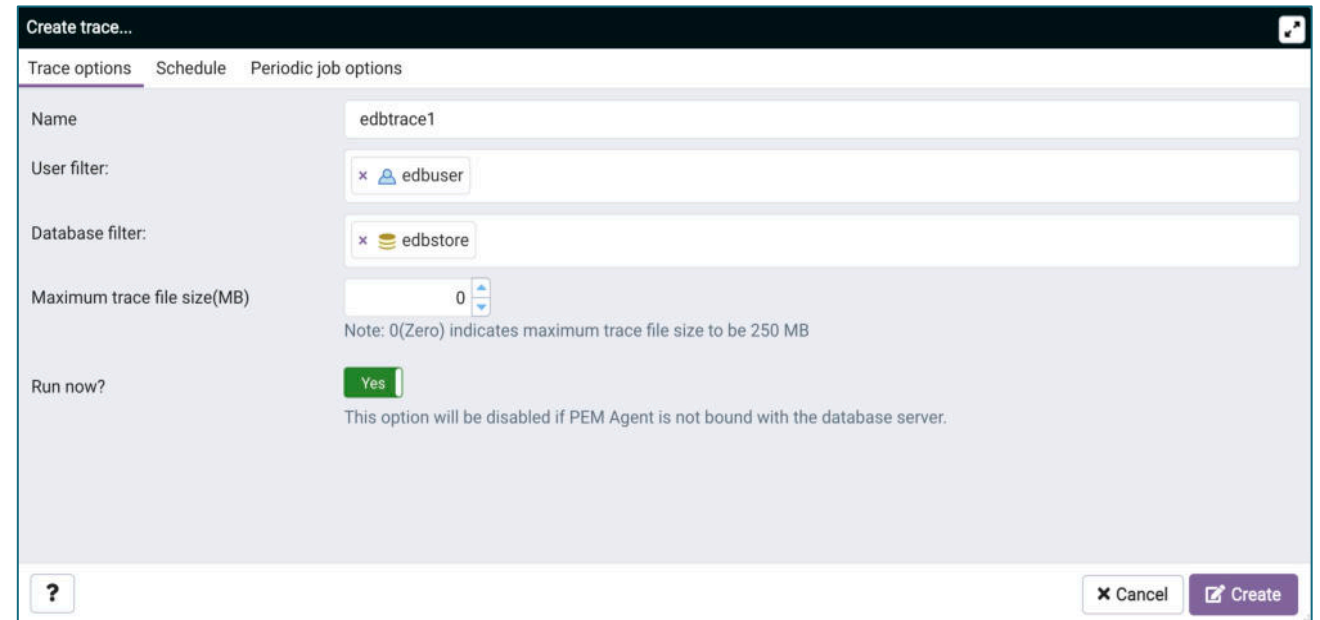
Starting SQL Profiler

- After configuration, click on the database in object browser
- Go to the Management menu, go to SQL Profiler and click “Create Trace”



Creating a New SQL Trace

- You can use the Create trace dialog to define a new SQL Trace
- Specify SQL trace setting using Trace options, Schedule and Periodic job options tabs
- You can use trace options tab for controlling:
 - Trace Name
 - Filter SQL run by one or more user
 - Filter for tracing SQL running on a specific database
 - Trace file size
 - Run now or schedule later



The screenshot shows the 'Create trace...' dialog box with three tabs: 'Trace options', 'Schedule', and 'Periodic job options'. The 'Trace options' tab is active. It contains the following fields and controls:

- Name:** A text input field containing 'edbtrace1'.
- User filter:** A dropdown menu showing 'edbuser' with a close button (x) on the left.
- Database filter:** A dropdown menu showing 'edbstore' with a close button (x) on the left.
- Maximum trace file size(MB):** A numeric input field set to '0'. Below it, a note states: 'Note: 0(Zero) indicates maximum trace file size to be 250 MB'.
- Run now?:** A green button labeled 'Yes'. Below it, a note states: 'This option will be disabled if PEM Agent is not bound with the database server.'

At the bottom of the dialog, there is a help icon (question mark), a 'Cancel' button, and a 'Create' button.



Viewing the SQL Profiler Report

DashboardPropertiesSQLStatisticsDependenciesDependentsMonitoringSQL Profiler (Trace: edbtrace1)

1 of 1

Show queries per page: 100

#	Start Time	Duration (ms)	Query	Rows Affected	User	Database	PID	File System Read	File System
1	2025-01-14 16:10:17.199591+05:30	78.517535	select * from custom...	20000	edbuser	edbstore	4529	9771546	
2	2025-01-14 16:10:34.666058+05:30	0.289911	SELECT c.relname, N...	2	edbuser	edbstore	4529	9771546	
3	2025-01-14 16:10:35.432454+05:30	0.183181	SELECT c.relname, N...	1	edbuser	edbstore	4529	9771546	
4	2025-01-14 16:10:36.612079+05:30	0.154356	SELECT n.nspname a...	1	edbuser	edbstore	4529	9771546	
5	2025-01-14 16:10:39.476097+05:30	0.11375	SELECT c.oid, n.nspn...	1	edbuser	edbstore	4529	9771546	
6	2025-01-14 16:10:39.477453+05:30	0.093739	SELECT c.relchecks, c...	1	edbuser	edbstore	4529	9771546	
7	2025-01-14 16:10:39.479778+05:30	1.6049	SELECT a.attname, pg...	20	edbuser	edbstore	4529	9771546	
8	2025-01-14 16:10:39.485241+05:30	4.067359	SELECT c2.relname, i...	2	edbuser	edbstore	4529	9771546	
9	2025-01-14 16:10:39.491894+05:30	0.057196	SELECT true as samet...	0	edbuser	edbstore	4529	9771546	

SQL QueryMetrics

SELECT c.oid::pg_catalog.regclass, c.relkind, inhdetachpending, pg_catalog.pg_get_ex
FROM pg_catalog.pg_class c, pg_catalog.pg_inherits i
WHERE c.oid = i.inhrelid AND i.inhparent = '17692'
ORDER BY pg_catalog.pg_get_expr(c.relpartbound, c.oid) = 'DEFAULT', c.oid::pg_catalo

Graphical PlanText-Based Plan

GraphicalAnalysisStatistics

pg_inherits

pg_class_oid_index

Nested Loop Inner Join

Sort

© EDB 2025 - ALL RIGHTS RESERVED.

Performance Diagnostic

- The Performance Diagnostic dashboard can be used to view and analyze wait events data
- The wait events data is gathered using EDB Wait States module
- This data includes information about locking, IO, CPU and SQL queries
- The performance data is depicted on easy-to-read graphs with various options available to easily point to sudden spike in the wait events



Prerequisites for Performance Diagnostic



PEM uses EDB Wait States Extension for data collection.



Install required package:

`edb-as<X>-server-edb-modules`



Changes required in the postgresql.conf file:

`shared_preload_libraries = '$libdir/edb_wait_states'`



Restart the database server, and then create the extension in the maintenance database:

`CREATE EXTENSION edb_wait_states;`



You must have super user privileges to access the Performance Diagnostic dashboard

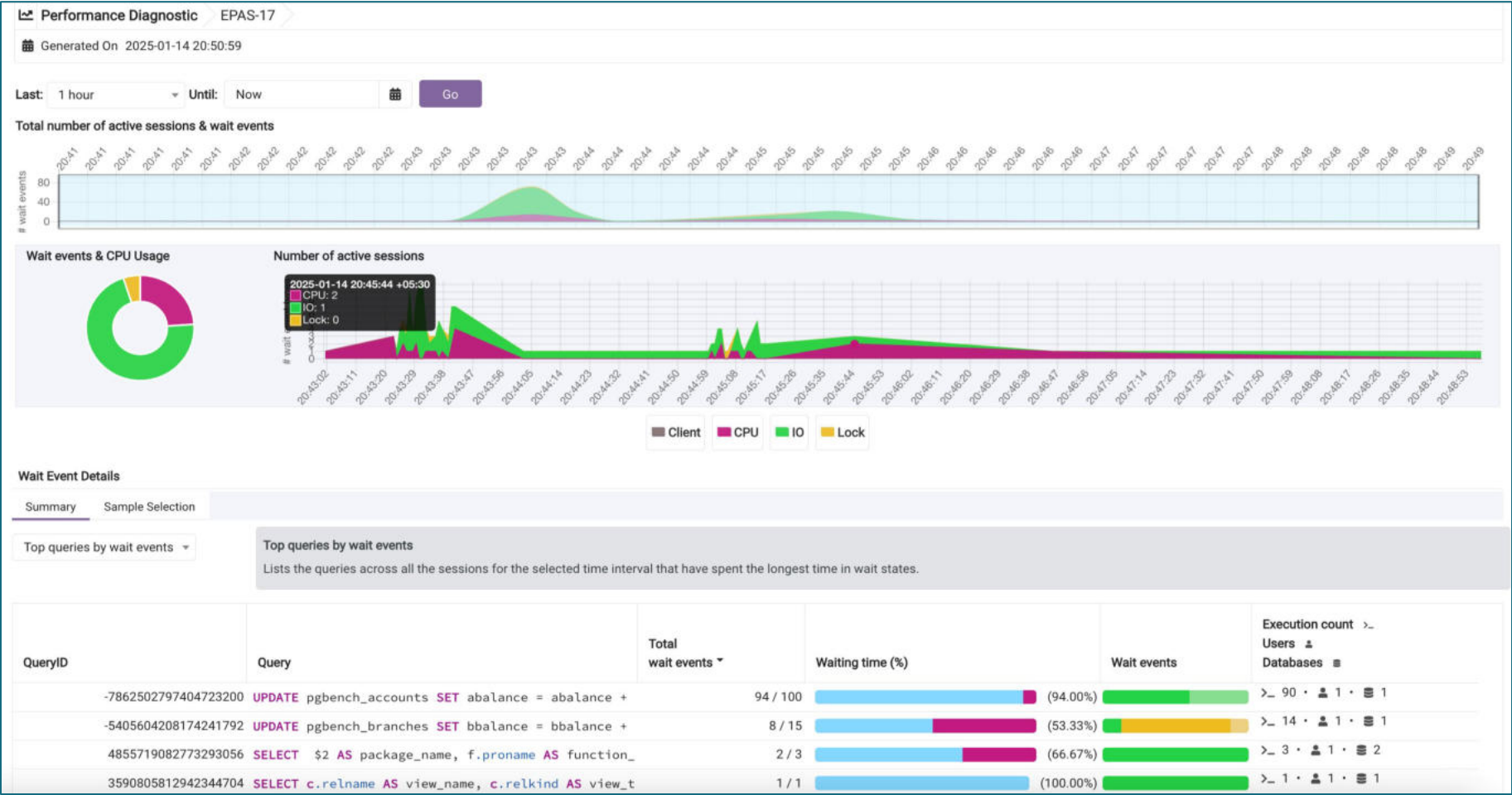


Open Performance Diagnostics Dashboard

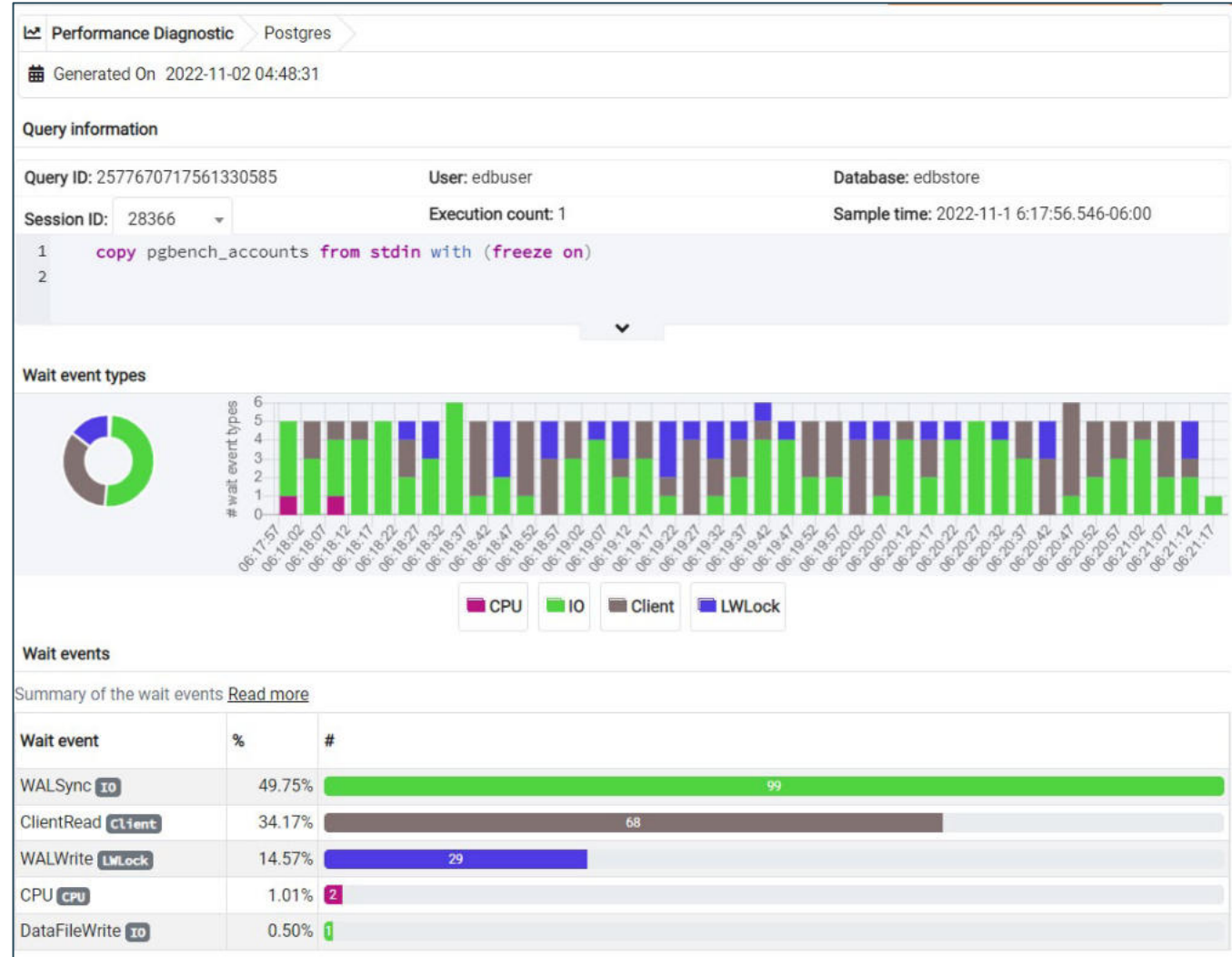
The screenshot shows the PostgreSQL Enterprise Manager interface. The top navigation bar includes a logo and menu items: File, Object, Management, Dashboards, Tools, and Help. The 'Tools' menu is open, displaying a list of options: New ERD project(Beta), Schema Diff, Search Objects..., Server (highlighted), Query Tool, Storage Manager..., Reload Configuration, Pause Replay of WAL, Resume Replay of WAL, Import/Export..., Maintenance..., Backup..., Backup Globals..., Backup Server..., Restore..., Grant Wizard..., and Schedule Backup... The 'Server' option has a submenu open, showing: Add Named Restore Point..., Performance Diagnostics... (highlighted), Queue Server Startup, Queue Server Shutdown, Replace Cluster Primary, Switchover EFM Cluster, and SQL Profiler. The main dashboard area is titled 'Performance Diagnostic' and shows a graph of 'Total number of active sessions & wait events' for the last 24 hours, with a peak around 16:25. Below the graph is a 'Wait events & CPU Usage' donut chart.



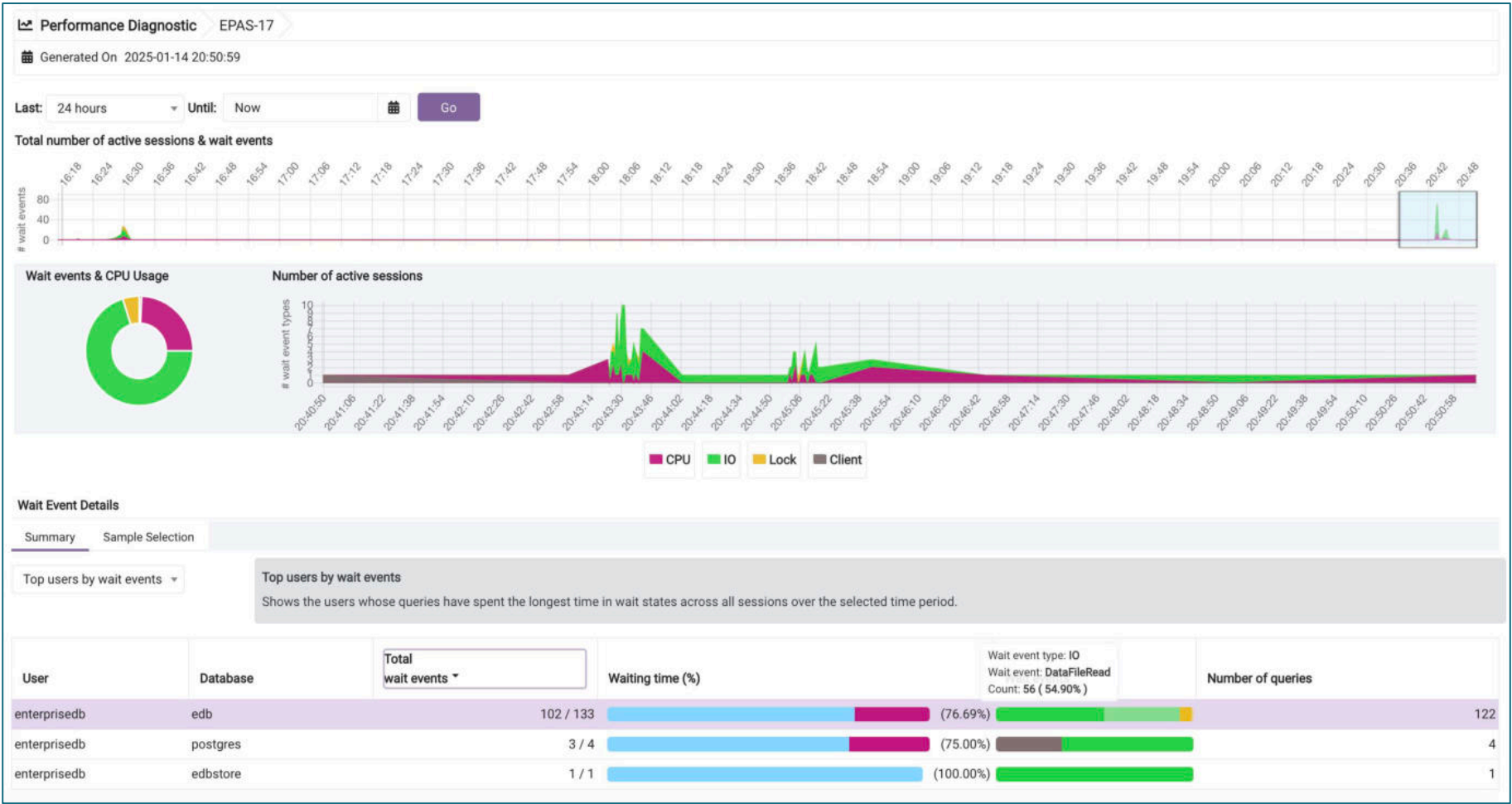
View Performance Diagnostics Dashboard



View Wait Events using Performance Diagnostics Dashboard



View Wait Events using Performance Diagnostics Dashboard



Lab Exercise 1

- You have been asked to install and configure Postgres Enterprise Manager and monitor a Postgres instance running on a Linux machine
 - Prepare a Linux machine for PEM Server installation
 - Install and configure a backend database
 - Install and configure PEM server
 - Open PEM Web Interface and logon to PEM Server
- Optional
 - Install PEM Agent on the managed host
 - Register PEM Agent with PEM Server
 - Verify PEM Agent information in PEM Web Interface



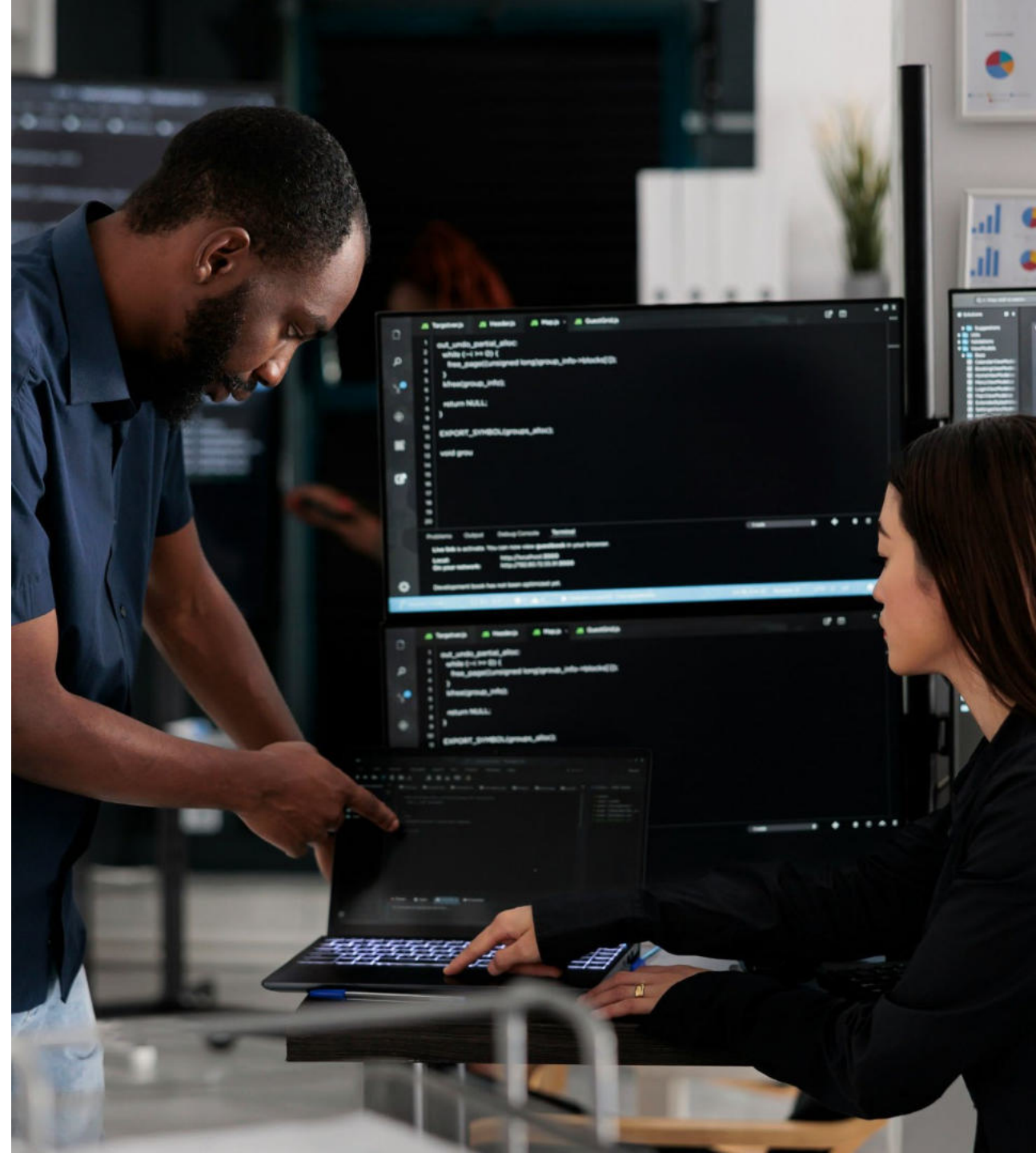


SQL Tuning

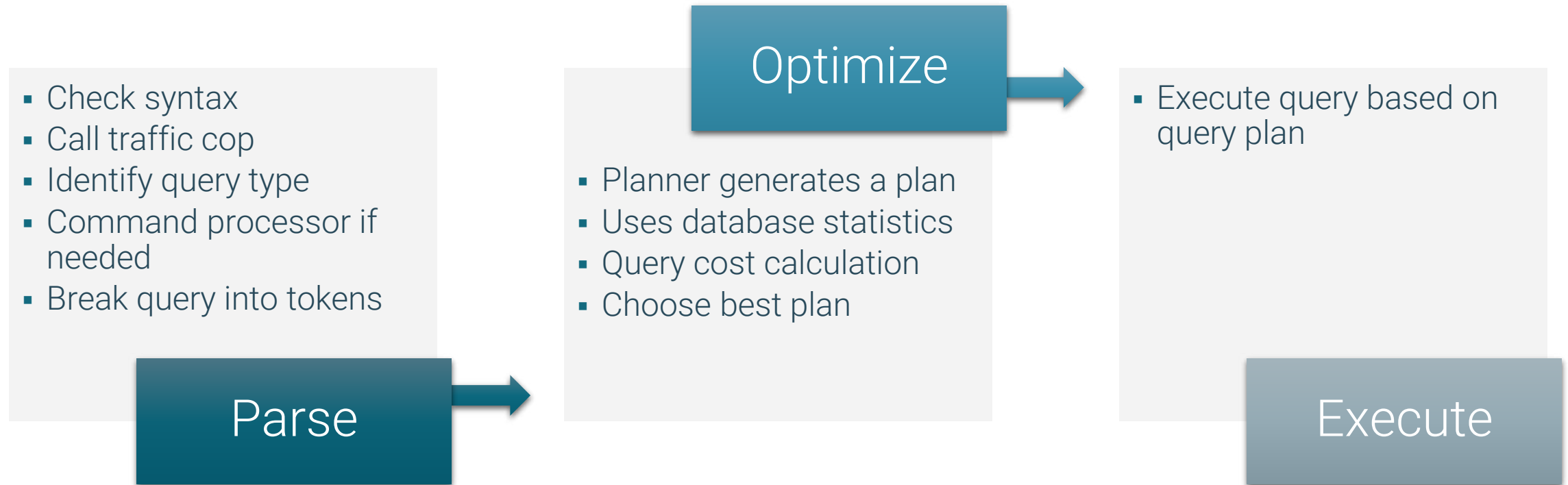


Module Objectives

- SQL queries planning and execution
- Find and tune slow running SQL
- Types of Indexes and their usage
- Writing an efficient SQL query



Statement Processing - SQL Engine



Common Query Performance Issues

Full table scans

Bad SQL

Sorts using disk

Join orders

Old or missing statistics

I/O issues

Bad connection management

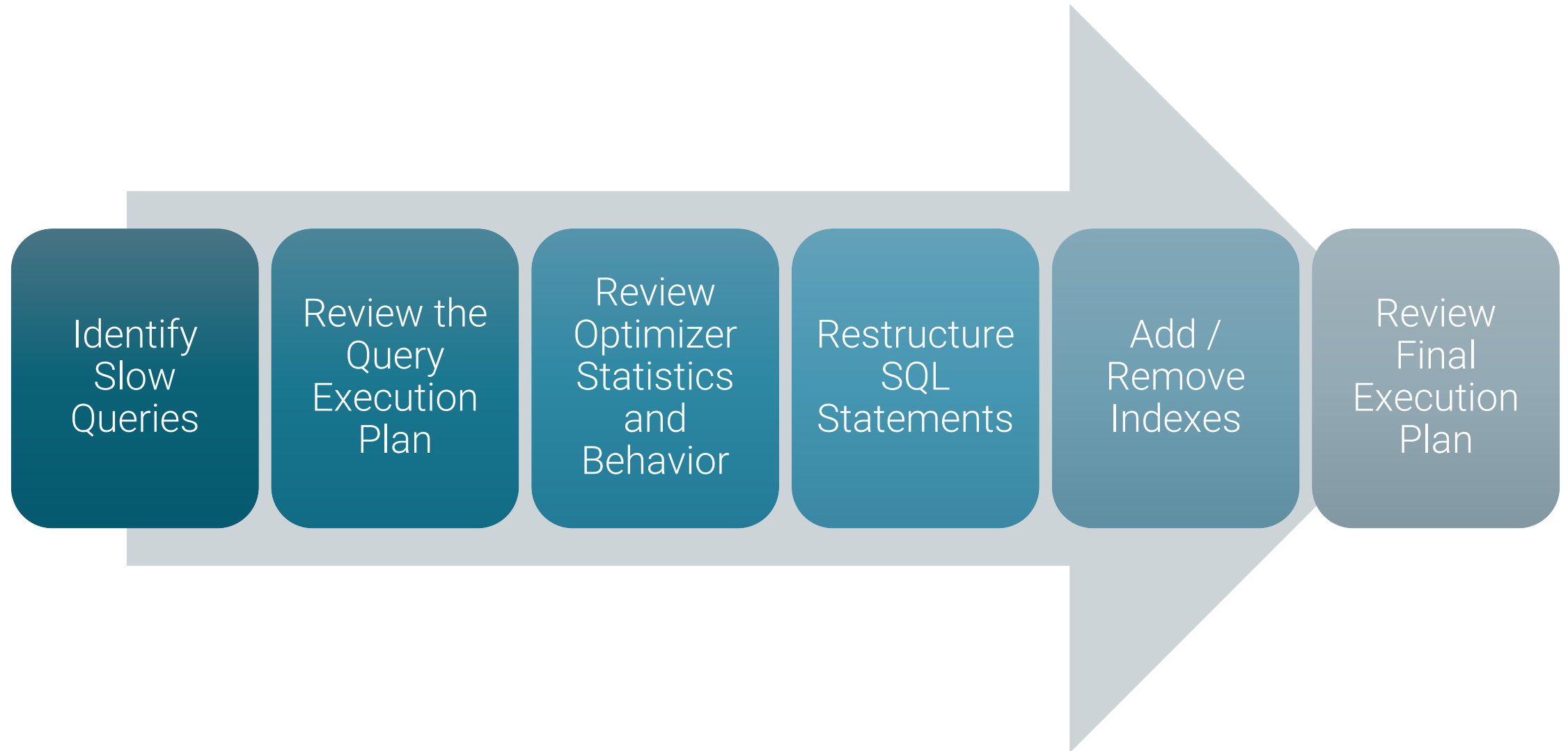


SQL Tuning Goals

Identify	Identify bad or slow SQL
Find	Find the possible performance issue in a query
Reduce	Reduce total execution time
Reduce	Reduce the resource usage of a query
Determine	Determine most efficient execution plan
Balance or parallelize	Balance or parallelize the workload



SQL Tuning Steps



Step 1- Identify Slow Queries



Tracking Slow Queries



log_min_duration_statement tracks slow running SQL



Use PEM's Log Manager Wizard to configure logging of slow queries



Use PEM's Postgres Log Analysis Expert Wizard to analyze log messages



Run a PEM's SQL Profiler trace to find, troubleshoot, and optimize slow running SQL



Step 2 - Review the Query Execution Plan



Execution Plan

- An execution plan shows the detailed steps necessary to execute a SQL statement
- Planner is responsible for generating the execution plan
- The Optimizer determines the most efficient execution plan
- Optimization is cost-based, cost is estimated resource usage for a plan
- Cost estimates rely on accurate table statistics, gathered with ANALYZE
- Costs also rely on seq_page_cost, random_page_cost, and others
- The EXPLAIN command is used to view a query plan
- EXPLAIN ANALYZE is used to run the query to get actual runtime stats



Execution Plan Components

- Syntax:

EXPLAIN [(option [, ...])] statement

EXPLAIN [ANALYZE] [VERBOSE] statement

where option can be one of:

ANALYZE [boolean]

VERBOSE [boolean]

COSTS [boolean]

SETTINGS [boolean]

GENERIC_PLAN [boolean]

BUFFERS [boolean]

SERIALIZE [{ NONE | TEXT | BINARY }]

WAL [boolean]

TIMING [boolean]

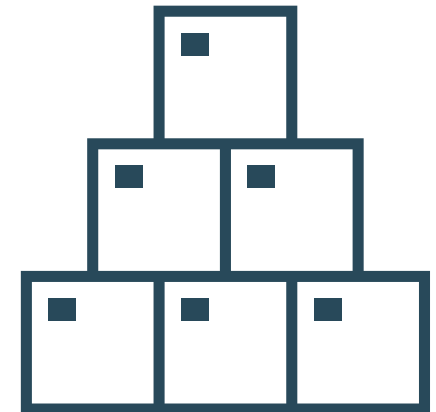
SUMMARY [boolean]

MEMORY [boolean]

FORMAT { TEXT | XML | JSON | YAML }

Execution Plan Components:

- **Cardinality** - Row Estimates
- **Access Method** - Sequential or Index
- **Join Method** - Hash, Nested Loop etc.
- **Join Type, Join Order**
- **Sort and Aggregates**



Explain Example - One Table

- Example

```
postgres=# EXPLAIN SELECT * FROM emp;
```

```
QUERY PLAN
```

```
-----  
Seq Scan on emp (cost=0.00..1.14 rows=14 width=145)
```

- The numbers that are quoted by EXPLAIN are:
 - Estimated start-up cost
 - Estimated total cost
 - Estimated number of rows output by this plan node
 - Estimated average width (in bytes) of rows output by this plan node



Explain Example - Multiple Tables

- Create the tables
 - `=# CREATE TABLE city (cityid numeric(5) primary key, cityname varchar(30));`
 - `=# CREATE TABLE office (officeid numeric(5) primary key, cityid numeric(5) references city(cityid));`
- Let's see the plan without data and updating statistics:
 - `=# EXPLAIN ANALYZE SELECT city.cityname, office.officeid, office.cityid FROM city,office WHERE office.cityid = city.cityid;`
- Output:
 - Hash Join (cost=25.30..71.16 rows=1510 width=102) (actual time=0.002..0.002 rows=0 loops=1)
 - Hash Cond: (office.cityid = city.cityid)
 - -> Seq Scan on office (cost=0.00..25.10 rows=1510 width=24) (actual time=0.001..0.001 rows=0 loops=1)
 - -> Hash (cost=16.80..16.80 rows=680 width=90) (never executed)
 - -> Seq Scan on city (cost=0.00..16.80 rows=680 width=90) (never executed)
 - Planning time: 0.456 ms
 - Execution time: 0.067 ms



Explain Example – Load and Analyze

- Load data:

- `=# INSERT INTO city
values(1,'Edmonton'), (2,'Calgary'), (3,'SherwoodPark'), (4,'STAlbert');`
- `=# INSERT INTO office VALUES(generate_series(1,100),4);`
- `=# INSERT INTO office VALUES(generate_series(101,200),3);`
- `=# INSERT INTO office VALUES(generate_series(201,300),2);`
- `=# INSERT INTO office VALUES(generate_series(301,400),1);`

- Update the statistics for city and office table:

- `=# ANALYZE city;`
- `=# ANALYZE office;`



Explain Example – Explain Analyze

- Plan:

```
=# EXPLAIN ANALYZE SELECT city.cityname, office.officeid,  
office.cityid FROM city, office WHERE office.cityid =  
city.cityid;
```

```
Hash Join  (cost=1.09..12.59 rows=400 width=20) (actual time=0.057..0.770 rows=400  
loops=1)
```

```
Hash Cond: (office.cityid = city.cityid)
```

```
-> Seq Scan on office  (cost=0.00..6.00 rows=400 width=10) (actual  
time=0.012..0.128 rows=400 loops=1)
```

```
-> Hash  (cost=1.04..1.04 rows=4 width=15) (actual time=0.014..0.014 rows=4  
loops=1)
```

```
Buckets: 1024  Batches: 1  Memory Usage: 1kB
```

```
-> Seq Scan on city  (cost=0.00..1.04 rows=4 width=15) (actual  
time=0.002..0.005 rows=4 loops=1)
```

```
Planning time: 0.577 ms
```

```
Execution time: 0.893 ms
```



PEM - Query Tool's Visual Explain

edbstore/postgres@Postgres

Query Editor Query History Scratch Pad

```
1 select firstname,lastname, city, orderid from edbuser.customers join edbuser.cust_hist using (customerid);
```

Data Output Explain Messages Notifications

Graphical Analysis Statistics

edbuser.cust_hist

Hash Inner Join

edbuser.customers

Hash

Actual Startup Time	12.501
Actual Total Time	36.167
Actual Rows	60350
Actual Loops	1
Output	customers.firstname,customers.lastname,customers.city,cust_hist.orderid
Inner Unique	true
Hash Cond	(cust_hist.customerid = customers.customerid)
Shared Hit Blocks	3
Shared Read Blocks	833
Shared Dirtied Blocks	0
Shared Written Blocks	0
Local Hit Blocks	0
Local Read Blocks	0
Local Dirtied Blocks	0
Local Written Blocks	0
Temp Read Blocks	0
Temp Written Blocks	0
_serial	1



Reviewing Explain Plans

Examine the various costs at different levels in the explain plan

Look for sequential scans on large tables

All sequential scans are not inefficient

Check whether a join type is appropriate for the number of rows returned

Check for indexes used in the explain plan

Examine the cost of sorting and aggregations

Review whether views are used efficiently



Step 3 - Review Optimizer Statistics and Behavior



Optimizer Statistics

- The Postgres Optimizer and Planner use table statistics for generating query plans
- Choice of query plans are only as good as table stats
- Table statistics
 - Stored in catalog tables like `pg_class` and `pg_stats`
 - Stores row sampling information



Updating Planner Statistics



Absolutely critical to have accurate statistics to ensure optimal query plans



Are not updated in real time, so should be manually updated after bulk operations



Can be updated using ANALYZE command or OS command vacuumdb with -Z option



Stored in pg_class and pg_statistics



You can run the ANALYZE command from psql on specific tables and just specific columns



Autovacuum will run ANALYZE on tables as configured



ANALYZE Command

- Postgres collects and updates statistics for each table using ANALYZE command
- ANALYZE command can also be run in conjunction with the VACUUM command
- Syntax for ANALYZE command:
 - `ANALYZE [options] table_name [ ( column_name [, ...] ) ]`
 - Where options can be:
 - `VERBOSE [ boolean ]`
 - `SKIP_LOCKED [ boolean ]`
 - `BUFFER_USAGE_LIMIT [ size ]`



Controlling Statistics Collection

- Postgres gathers and maintains table and column level statistics
- Statistics collection level can be controlled using:
 - `=# ALTER TABLE <table> ALTER COLUMN <column> SET`
 - `STATISTICS <number>;`
- The `<number>` can be set between 1 and 10000
- A higher `<number>` will signal the server to gather and update more statistics but may have slow autovacuum and analyze operation on stat tables. Higher numbers only useful for tables with large irregular data distributions



Extended Statistics

- Postgres stores column-specific statistics lacking awareness of inter-column relationships
- Understanding of relationships between different columns can help during query optimization
- Extended statistics created using “CREATE STATISTICS” empower Postgres to comprehend inter-column relationships
- This improves query optimization by offering additional insights to the planner
- Extended statistics are vital for addressing cross-column correlation challenges



Example - Extended Statistics

- Create a table

```
CREATE TABLE items (  
    item_id INTEGER PRIMARY KEY,  
    price NUMERIC,  
    quantity INTEGER,  
    category VARCHAR(30)  
);
```

- Insert sample data

- INSERT INTO items VALUES
(generate_series(1,1000000),18.99, 200, 'Speaker');
- INSERT INTO items VALUES
(generate_series(1000001,2000000),31.99, 100, 'TV');
- INSERT INTO items VALUES
(generate_series(2000001,3000000),29.99, 100, 'TV');
- INSERT INTO items VALUES
(generate_series(3000001,4000000),29.99, 100, 'Camera');

- Update statistics and view explain plan

```
ANALYZE items;
```

```
EXPLAIN ANALYZE SELECT * FROM items WHERE  
price > 30 AND quantity < 120;
```

```
QUERY PLAN
```

```
-----  
Seq Scan on items  (cost=0.00..87018.26  
rows=757286 width=21) (actual time=139.089..  
1192.178 rows=1000000 loops=1)
```

```
    Filter: ((price > '30'::numeric) AND  
    (quantity < 120))
```

```
    Rows Removed by Filter: 3000000
```

```
    Planning Time: 0.165 ms
```

```
    Execution Time: 1828.823 ms
```

```
(5 rows)
```



Example - Extended Statistics

- Create extended statistics

```
CREATE STATISTICS prc_qnt_rel  
ON price,quantity FROM items;
```

- Analyze the table

```
ANALYZE items;
```

```
EXPLAIN ANALYZE SELECT * FROM items  
WHERE price > 30 AND quantity < 120;
```

```
QUERY PLAN
```

```
-----Seq Scan  
on items (cost=0.00..86462.00 rows=984267  
width=21) (actual time=138.165..
```

```
1184.710 rows=1000000 loops=1)
```

```
Filter: ((price > '30'::numeric) AND  
(quantity < 120))
```

```
Rows Removed by Filter: 3000000
```

```
Planning Time: 0.166 ms
```

```
Execution Time: 1823.448 ms
```



Step 4 - Restructuring SQL Statements and Optimizer Hints



Restructure SQL Statements

Rewriting inefficient SQL is often easier than repairing it

Avoid implicit type conversion

Avoid expressions as the optimizer may ignore the indexes on such columns

Use equijoins wherever possible to improve SQL efficiency

Try changing the access path and join orders with hints

Avoid full table scans if using an index is more efficient

The join order can have a significant effect on performance

Use views or materialized views for complex queries

Use parallel query scans to improve the query performance



Optimizer Hints

- Optimizer hints are directives embedded in comment-like syntax immediately following the `DELETE`, `INSERT`, `SELECT` or `UPDATE`
- Optimizer hints can alter execution plans
- Optimizer hints are used to directly influence selection of all or part of the final plan
- Optimizer hints influence optimizer decisions



Embedding Optimizer Hints

- Optimizer hints may be given in two different formats:

```
{ DELETE | INSERT | SELECT |  
UPDATE } /*+ { hint [ comment  
] } [...] */ statement_body
```

```
{ DELETE | INSERT | SELECT |  
UPDATE } --+ { hint [ comment  
] } [...] statement_body
```



Usage Description

- Planner parameters override optimizer hints
- Hints must be placed immediately after the first statement keyword
- If the hint is misspelled in the SQL command, there will be no indication that any sort of error has occurred
- If an alias is used for a table or view name in the SQL command, then the alias name, not the original object name, must be used in the hint
- Hints apply only to the statement where it is specified
- Use `EXPLAIN` to ensure that hints are correctly used



Access Method Hints

- The following hints influence how the optimizer accesses relations to create the result set:

Hint	Description
<code>FULL (table)</code>	Perform a full sequential scan on <code>table</code>
<code>INDEX (table [index] [...])</code>	Use <code>index</code> on <code>table</code> to access the relation
<code>NO_INDEX (table [index] [...])</code>	Do not use <code>index</code> on <code>table</code> to access the relation

- The `FULL` hint is used to force a full sequential scan instead of using the index:

```
=# EXPLAIN SELECT /*+ FULL(accounts) */ * FROM  
    accounts WHERE aid = 100;
```
- The `NO_INDEX` hint also forces a sequential scan



Join Hints

- There are three possible plans that may be used to perform a join between two tables:
 - Nested Loop Join, Merge Sort Join and Hash Join

Hint	Description
USE_HASH(table [...])	Use a hash join with a hash table created from the join attributes of table
NO_USE_HASH(table [...])	Do not use a hash join created from the join attributes of table
USE_MERGE(table [...])	Use a merge sort join for table
NO_USE_MERGE(table [...])	Do not use a merge sort join for table
USE_NL(table [...])	Use a nested loop join sort for table
NO_USE_NL(table [...])	Do not use a nested loop join sort for table

- Example:

```
=# EXPLAIN SELECT /*+ USE_HASH(a) */ b.bid, a.aid, abalance FROM  
    branches b, accounts a WHERE b.bid = a.bid;
```



APPEND Optimizer Hint

- By default, Postgres will add new data into the first available free-space
- Use APPEND optimizer hint with INSERT or SELECT for adding new rows at the end of the table
- This is useful when bulk loading data
- Example:

```
=# INSERT /*+APPEND*/ INTO sales VALUES (10, 10, '01-SEP-2014', 10, 'OR');
```

```
=# INSERT INTO sales_history SELECT /*+APPEND*/ FROM sales;
```



Parallelism Hints

- Parallel scanning provides performance improvement over sequential scan
- The `PARALLEL` optimizer hint is used to force parallel scanning
- The `NO_PARALLEL` optimizer hint prevents usage of a parallel scan
- Syntax:

```
PARALLEL (table [ parallel_degree | DEFAULT ])
```

```
NO_PARALLEL (table)
```

- Example:

```
=# EXPLAIN SELECT /*+ PARALLEL(pgbench_accounts) */ * FROM pgbench_accounts;
```



Step 5 - Review Indexes



General Indexing Guidelines

- Create indexes only when needed
- Remove unused indexes
- Adding an index for one SQL can slow down other queries
- Verify index usage using the EXPLAIN command



Indexes

Indexes	Description
B-tree	Equality and range queries on data that can be sorted (Default index type)
Hash	Only simple equality comparisons
GiST	Can be used depending on the indexing strategy (the operator class)
SP-GiST	Like GiST, offer an infrastructure that supports various kinds of searches
GIN	Can handle values that contain more than one key, for example arrays
BRIN (Block Range Index)	Accelerates scanning of large tables by maintaining summary data about block ranges. Very small index size compared to B-Tree, the tradeoff being you can't select a specific row so they are not useful for all data sets



Multicolumn Indexes

- An index can be defined on more than one column of a table
- Currently, only the B-tree, GiST, and GIN index types support multicolumn indexes
- Example:

```
=> CREATE INDEX test_idx1 ON test (id_1, id_2);
```

- This index will be used when you write:

```
=> SELECT * FROM test WHERE id_1=1880 AND id_2= 4500;
```

- Multicolumn indexes should be used sparingly



Indexes and ORDER BY

- You can adjust the ordering of a B-tree index by including the options `ASC`, `DESC`, `NULLS FIRST`, and/or `NULLS LAST` when creating the index to save sorting time spent by the query
- By default, B-tree indexes store their entries in ascending order with nulls last
- Examples:

```
=# CREATE INDEX test2_info_nulls_low ON test2 (info NULLS FIRST);
```

```
=# CREATE INDEX test3_desc_index ON test3 (id DESC NULLS LAST);
```



Unique Indexes

- Indexes can also be used to enforce uniqueness of a column's value or the uniqueness of the combined values of more than one column

```
=> CREATE UNIQUE INDEX name ON table  
(column [, ...]);
```

- Currently, only B-tree indexes can be declared unique
- Postgres automatically creates a unique index when a unique constraint or primary key is defined for a table



Functional Indexes

- An index can be created on a computed value from the table columns.
 - For example:

```
=> CREATE INDEX test1_lower_col1_idx ON test1 (lower(col1));
```
- Index expressions are relatively expensive to maintain
- Indexes on expressions are useful when retrieval speed is more important than insertion and update speed
- Indexes can also be created on user defined functions
- `ALTER INDEX ALTER COLUMN SET STATISTICS` can be used to set statistic target for index columns that are defined as expression



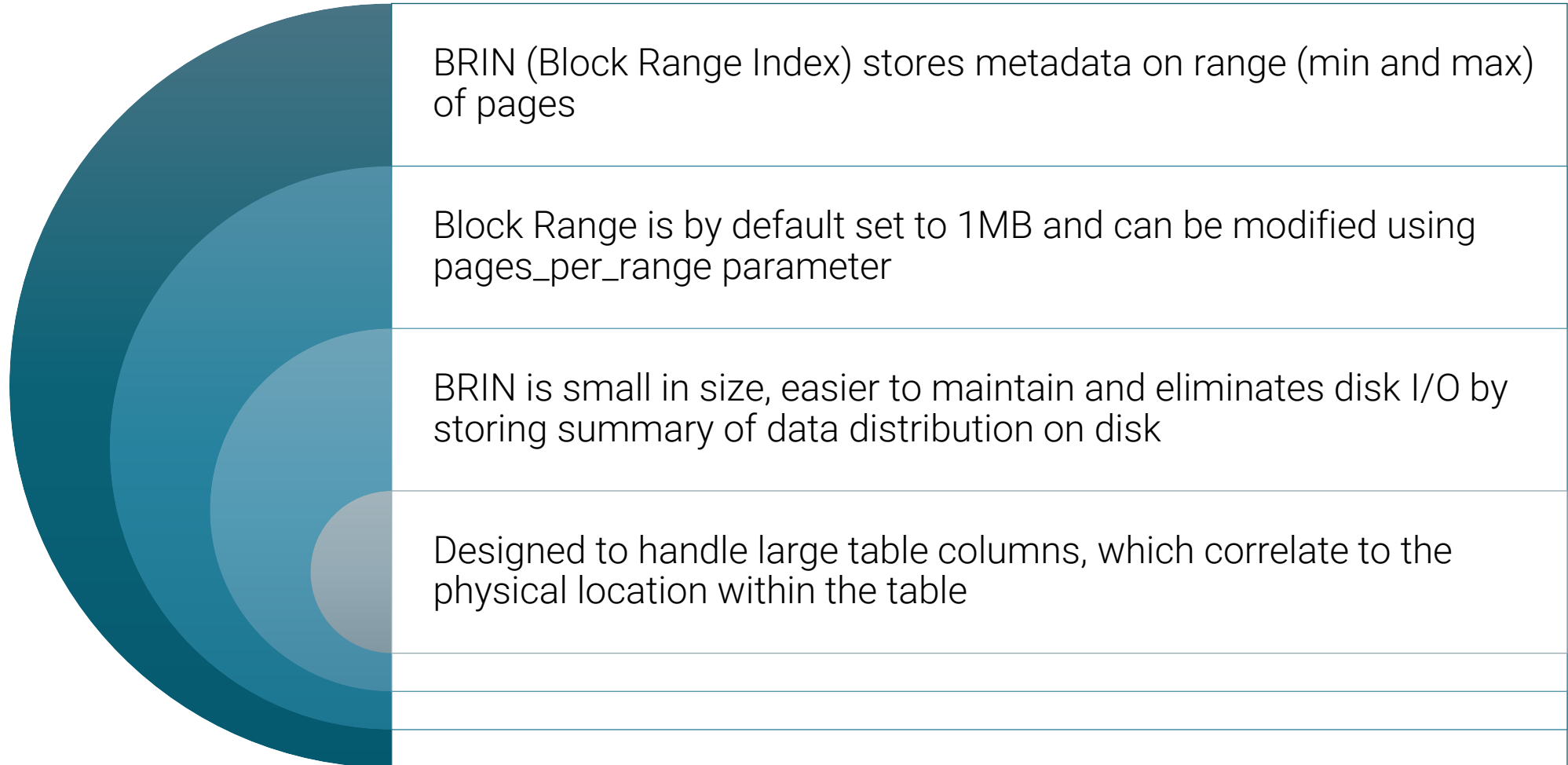
Partial Indexes

- A partial index is an index built over a subset of a table.
- The index contains entries only for those table rows that satisfy the predicate.
- Example:

```
edbstore=> CREATE TABLE citizen (id int, name varchar(50), city varchar(30),  
state varchar(20));  
CREATE TABLE  
  
edbstore=> CREATE INDEX citizen_state_idx_AR ON citizen(id) WHERE state='AR';  
CREATE INDEX
```



BRIN Indexes



Example - BRIN Indexes

```
postgres=# CREATE TABLE brin_example AS
  SELECT generate_series(1,100000000) AS id;
postgres=#
postgres=# CREATE INDEX brin_index on brin_example USING brin(id);
postgres=#
postgres=# CREATE INDEX btree_index ON brin_example(id);
postgres=#
postgres=# SELECT relname, pg_size_pretty(pg_relation_size(oid))
postgres-# FROM pg_class
postgres-# WHERE relname LIKE 'brin_%' OR relname='btree_index'
postgres-# ORDER BY relname;
```

relname	pg_size_pretty
brin_example	3457 MB
brin_index	104 kB
btree_index	2142 MB

(3 rows)



Index Only Scans

- Indexes speed up data retrieval and sorting by storing indexed column value and pointers in index pages
- If all the columns fetched by a query are indexed, PostgreSQL can perform an index only scan
- Index-only scan is a faster version of the ordinary index scan
- Any non-key column can also be included with indexed column in the index page using `INCLUDE` option of `CREATE INDEX` statement



Examining Index Usage

- It is difficult to formulate a general procedure for determining which indexes to create.
- Always run `ANALYZE` first.
- Use real data for experimentation.
- When indexes are not used, it can be useful for testing to force their use.
- The `EXPLAIN ANALYZE` command can be useful here.



Step 6 - Review Final Execution Plan



Last Step - Review the Final Plan

- Check again for missing indexes
- Check table statistics are showing correct estimates
- Check large table sequential scans
- Compare the cost of first and final plans



Lab Exercise - 1

- You are working as a DBA. Users are complaining about long running queries and high execution times. Configure your database instance to log slow queries. Any query taking more than 5 seconds must be logged.
- After logging slow queries, you find the following query taking longer than expected:

```
=> SELECT * FROM customers JOIN orders USING(customerid);
```

- View the explain plan of the above query.
- View the execution time for the above query in psql terminal.



Lab Exercise - 2

- Create a table using following queries:
 - `CREATE TABLE lab_test1 (c1 int4, c2 int4);`
 - `INSERT INTO lab_test1(c1, c2) values(generate_series(1, 100000), 1);`
 - `INSERT INTO lab_test1(c1, c2) values(generate_series(100001, 200000), 2);`
 - `INSERT INTO lab_test1(c1, c2) values(generate_series(200001, 300000), 3);`
- Create three partial indexes on lab_test1 table as following:

Index Name	Predicate
idx1_c1	c1 between 1 and 100000
idx2_c1	c1 between 100001 and 200000
idx3_c1	c1 between 200001 and 300000



Lab Exercise - 3

- Detect the index usage for all the user indexes in edbstore database.
- Reindex all the indexes.
- Manually update the statistics for all the objects in edbstore database.





Performance Tuning



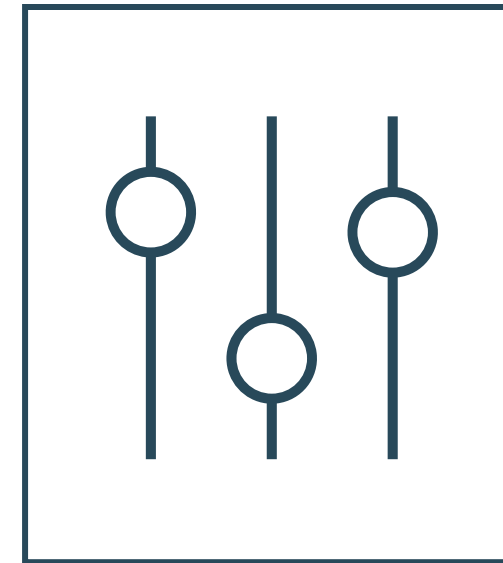
Module Objectives

- Parameter Tuning in Postgres
- Hot cache vs cold cache
- Data Loading best practices
- Tuning Postgres Using PEM



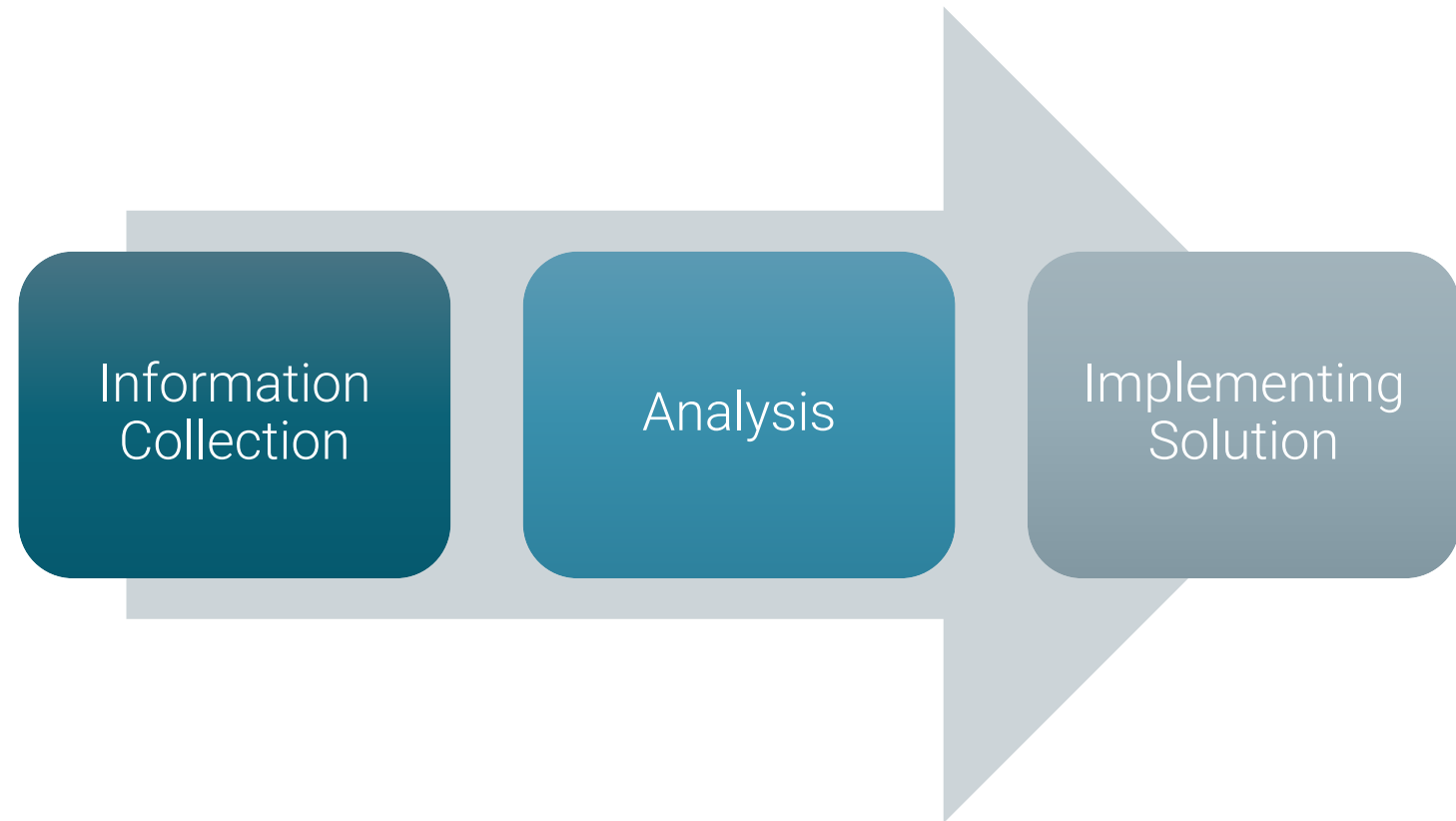
Performance Tuning - Overview

- Performance Tuning is a group of activities used to optimize a database
- Database Tuning is used to correct:
 - Poorly written SQL
 - Poor session management
 - Misconfigured database parameters
 - Operating system I/O issues
 - No Database maintenance



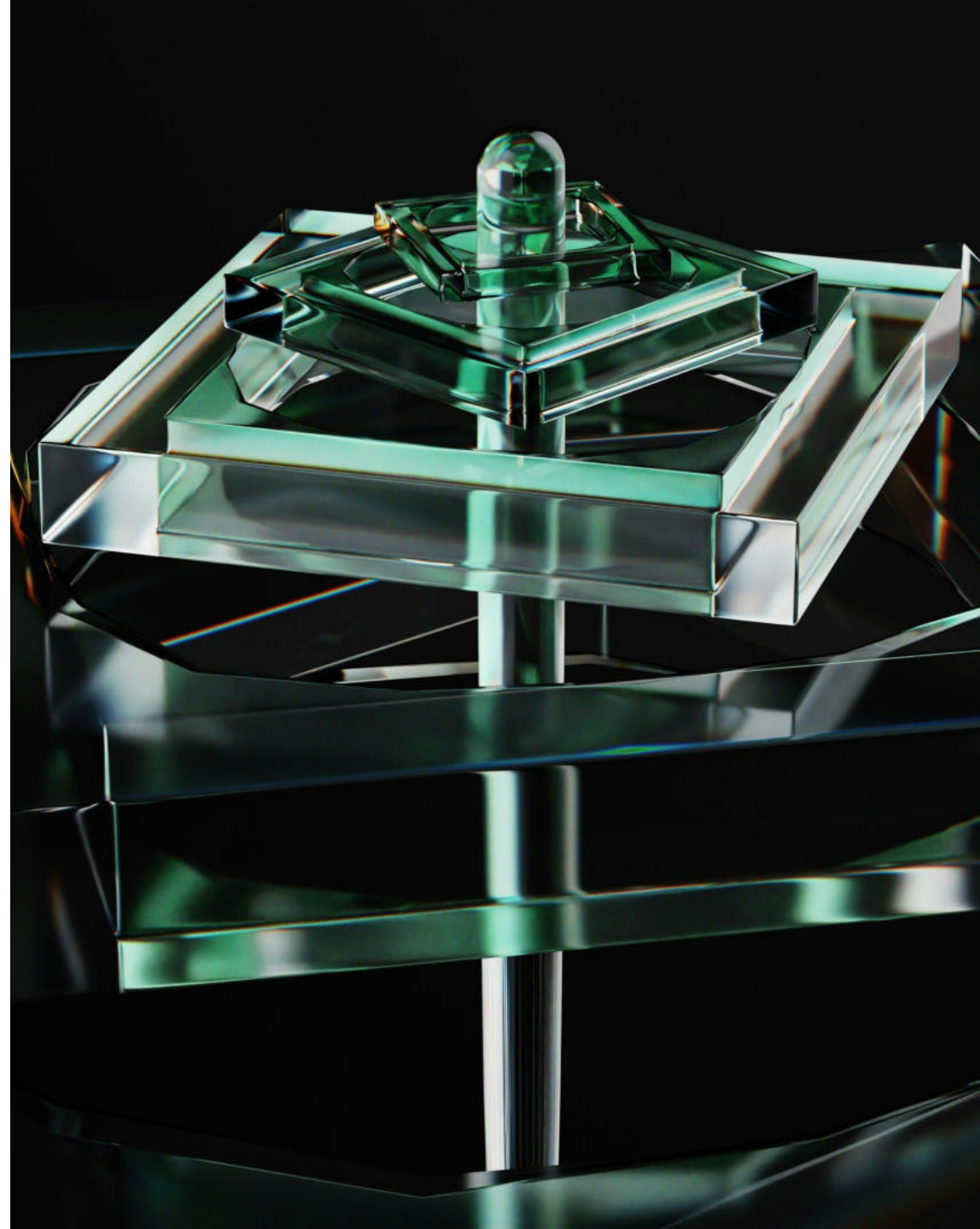
The Performance Tuning Process

- Identify the information relevant to diagnosing performance problems
- Collect that information on a regular basis
- Expert analysis is needed to understand and correlate all the relevant statistics together
- Multiple solutions for different problems
- Use your own judgment to prioritize and quantify the solutions by impact

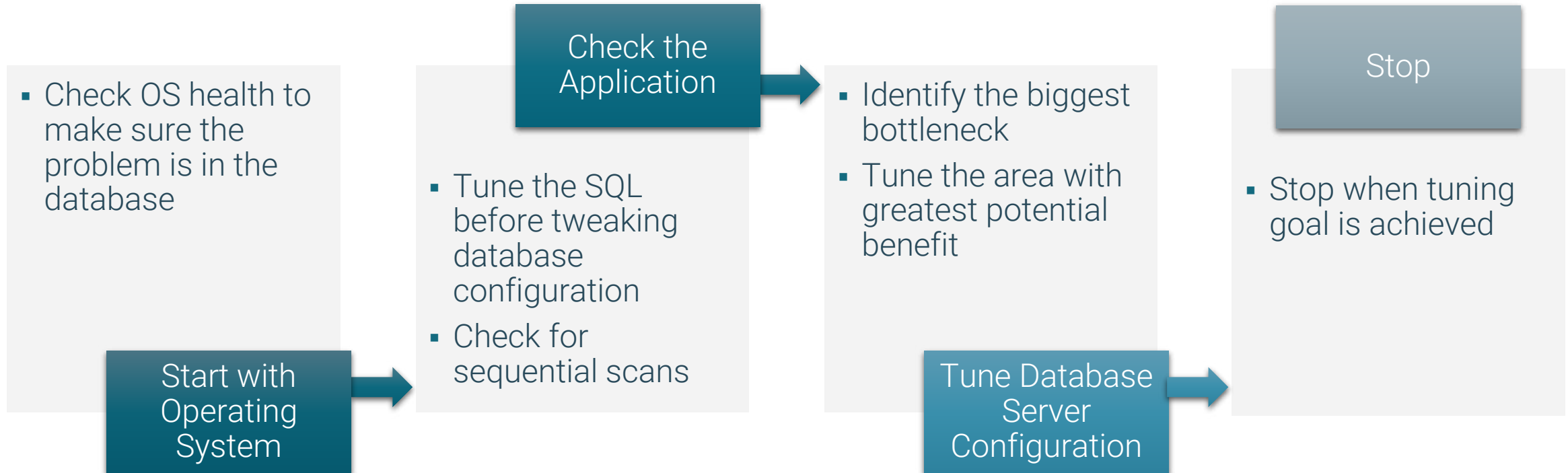


Performance Monitoring Using PEM

- Postgres Enterprise Manager (PEM) simplifies collection of performance data
- Automatic analysis of performance and diagnostics data
- Performance Dashboards - view I/O, memory usage, session activity, and wait statistics
- SQL Profiler - optimize slow SQL
- Index Advisor
- Setup alerts and thresholds



Tuning Technique





Operating System Considerations



Operating System Issues

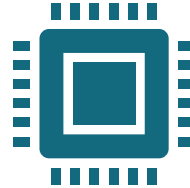


Memory

Increased memory demand may lead to 100% memory usage and swapping

Check memory usage

Solution is to reduce memory usage or increase system RAM



CPU

CPU may be the bottleneck when load or process wait is high

Check CPU usage (%) for the database connections

Solution is to reduce CPU usage (%)



Disk (I/O)

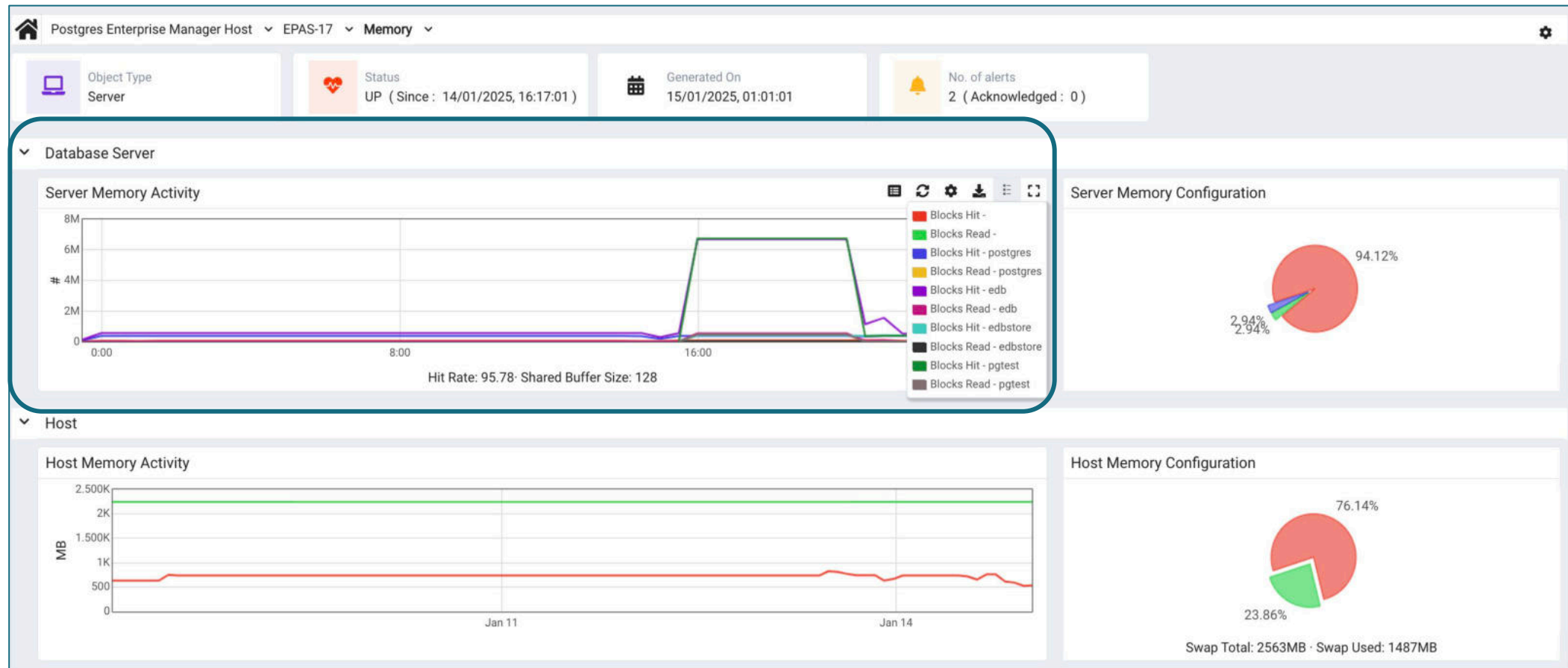
High wait times or request rates are symptoms of an I/O problem

Check for I/O spikes

Solution is to reduce demand or increase capacity



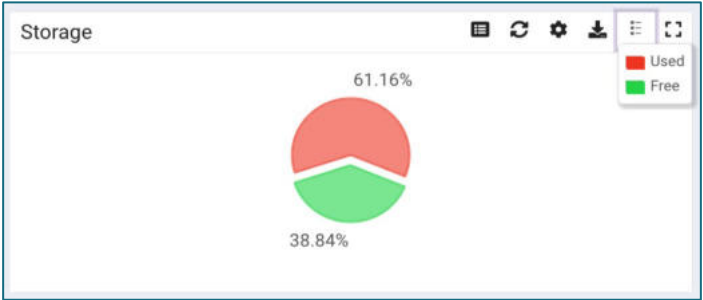
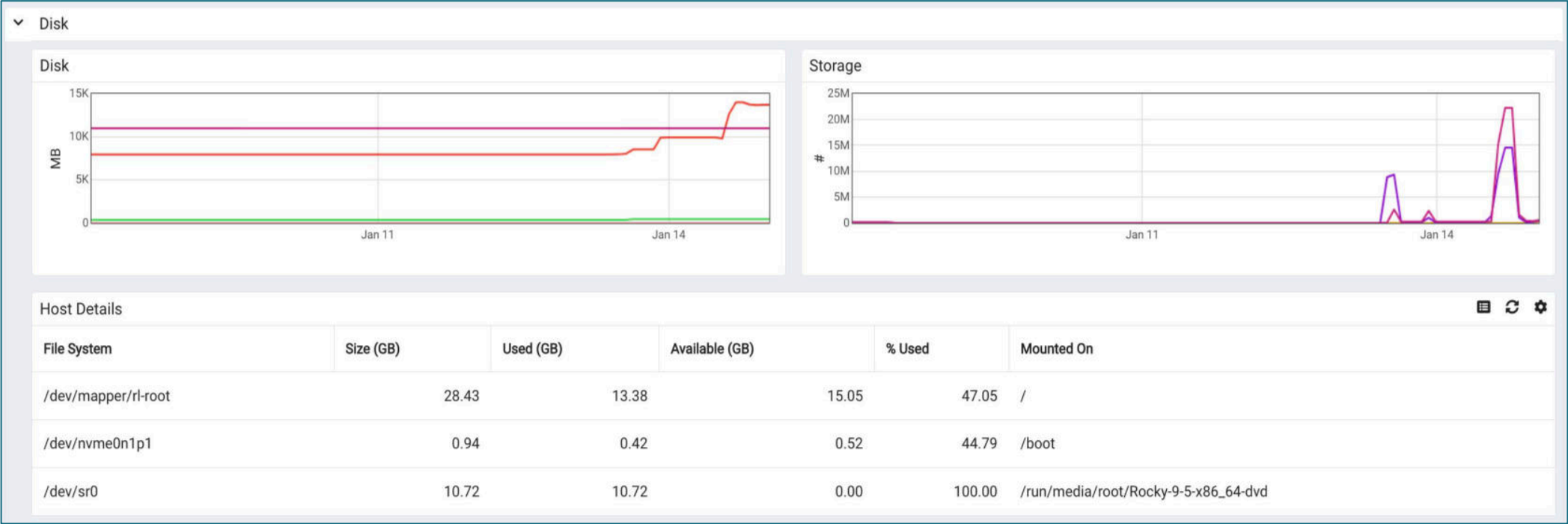
PEM - Memory Graphs and Alerts



PEM - CPU Info



PEM - Disk Info



Hardware Configuration

Focus on disk speed and RAM over CPU speed

- Like other RDBMS, Postgres is IO intensive

Separate the transaction log and indexes

- Put the database transaction log (pg_wal) on a dedicated disk resource
- Tablespaces can be used to create indexes on separate drives

Setup disks to match speed requirements, not just size requirements

- IOPS is just as important as GB when purchasing hardware
- RAID 0 + 1 is optimal for speed and redundancy
- Consider disk speed during failures (e.g. RAID-5 parity rebuild after failed disk)
- Write caches must be persistent battery backed “Write-Back”, or you will get corruption



OS Configuration

- The filesystem makes a difference
 - A journaling file system is not required for the transaction log
 - Multiple options are available for Linux:
 - EXT2 with sync enabled
 - EXT3 with write-back
 - EXT4 and XFS
 - Remote file systems are not recommended
- Choose the best based on better writes, recoverability, support from multiple vendors and reliability
- Eliminate unnecessary filesystem overhead, such as “noatime”
- Consider a virtual “ram” disk for `stats_temp_directory`



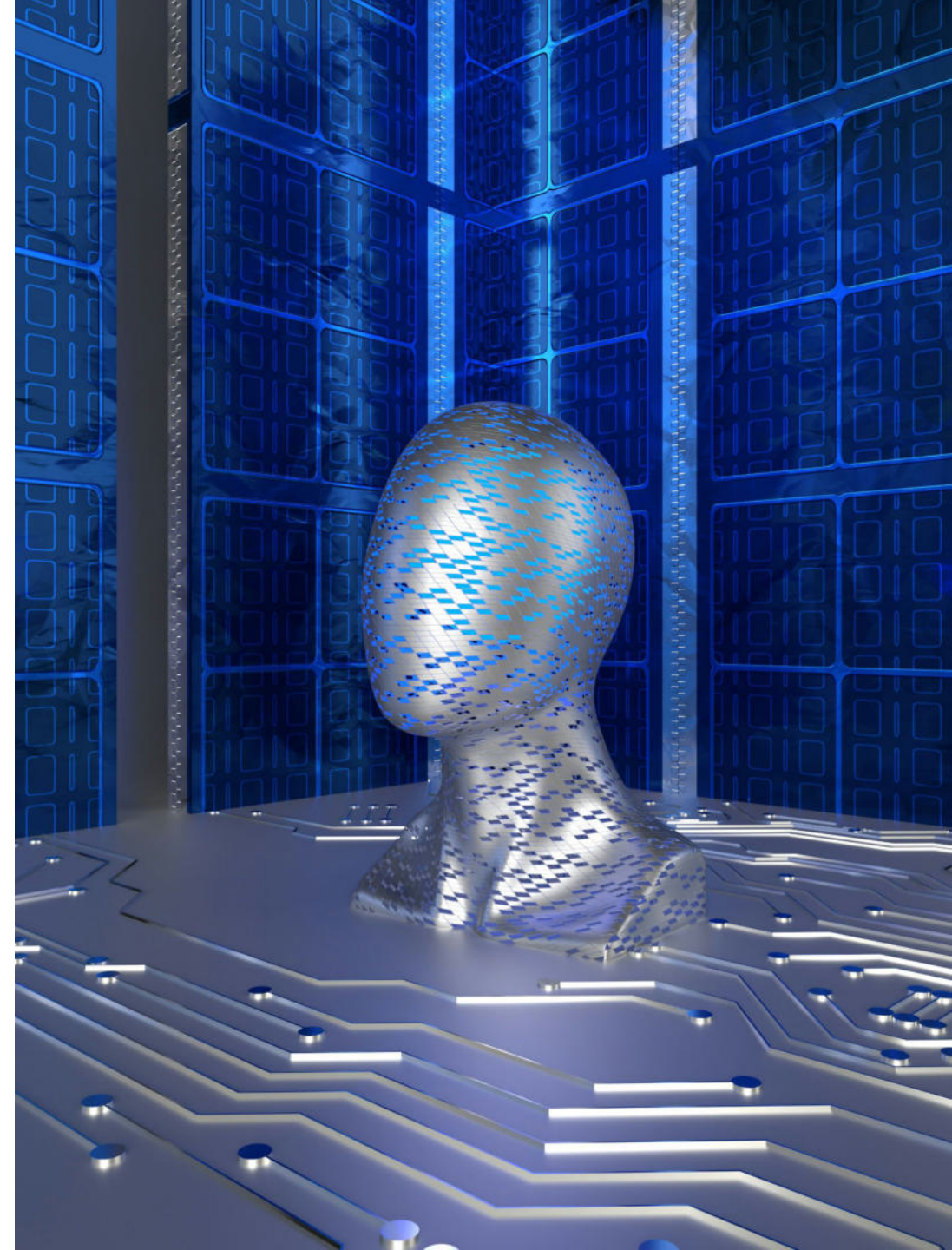
Server Parameter Tuning

- Default [postgresql.conf](#) server parameters are configured for wide compatibility
- Server parameters must be tuned to optimal values for better performance
- Parameters can be evaluated using different methods and can be set permanently in the [postgresql.conf](#) file
- Some parameters require a restart
- Basic information needed for tuning but not limited to:
 - Database size
 - Largest table size
 - Type and frequency of queries
 - Available RAM
 - Number of concurrent connections



PEM - Postgres Expert

- Postgres Expert analyzes the server configuration and reports potential performance and security issues
- Report also provides suggestions and best practices for addressing potential issues
- Postgres Expert is an advisory utility which provides advice about Performance, Security and Configuration



PEM - Postgres Expert Report

Postgres Expert Report

Generated On: 2025-01-15 01:14:08

Go to: EPAS-17

Summary

Servers Tested: 1 Rules Checked: 31 High Alerts: 2 Medium Alerts: 2 Low Alerts: 2

Server: EPAS-17 (localhost:5445)

Advisor: Configuration Expert

Rule	Database	Severity
> Check effective_cache_size	-	Medium
> Check effective_io_concurrency	-	Low
> Check reducing random_page_cost	-	Low

Advisor: Schema Expert

Rule	Database	Severity
> Check data and transaction log on same drive	-	High
> Check for missing foreign key indexes	edbstore	Medium

Advisor: Security Expert

Rule	Database	Severity
> Check TRUST authentication is disabled	-	High

Recommendations



Connection Settings

- `max_connections`
 - Sets the maximum number of concurrent connections
 - Each user connection has an associated user backend process on the server
 - User backend processes are terminated when a user logs off
 - Connection pooling can decrease the overhead on postmaster by reusing existing user backend processes



Memory Parameters - shared_buffers

- Sets the number of shared memory buffers used by the database server
- Each buffer is 8K bytes
- Minimum value must be 16 and at least $2 \times \text{max_connections}$
- 6% - 25% of available memory is a good general guideline
- You may find better results keeping the setting relatively low and using the operating system cache more instead



Memory Parameters - work_mem

- `work_mem`
 - Amount of memory in KB to be used by internal sorts and hash tables before switching to temporary disk files
 - Minimum allowed value is 64 KB
 - It is set in KB
 - Increasing the `work_mem` often helps in faster sorting
 - `work_mem` settings can also be changed on a per session basis



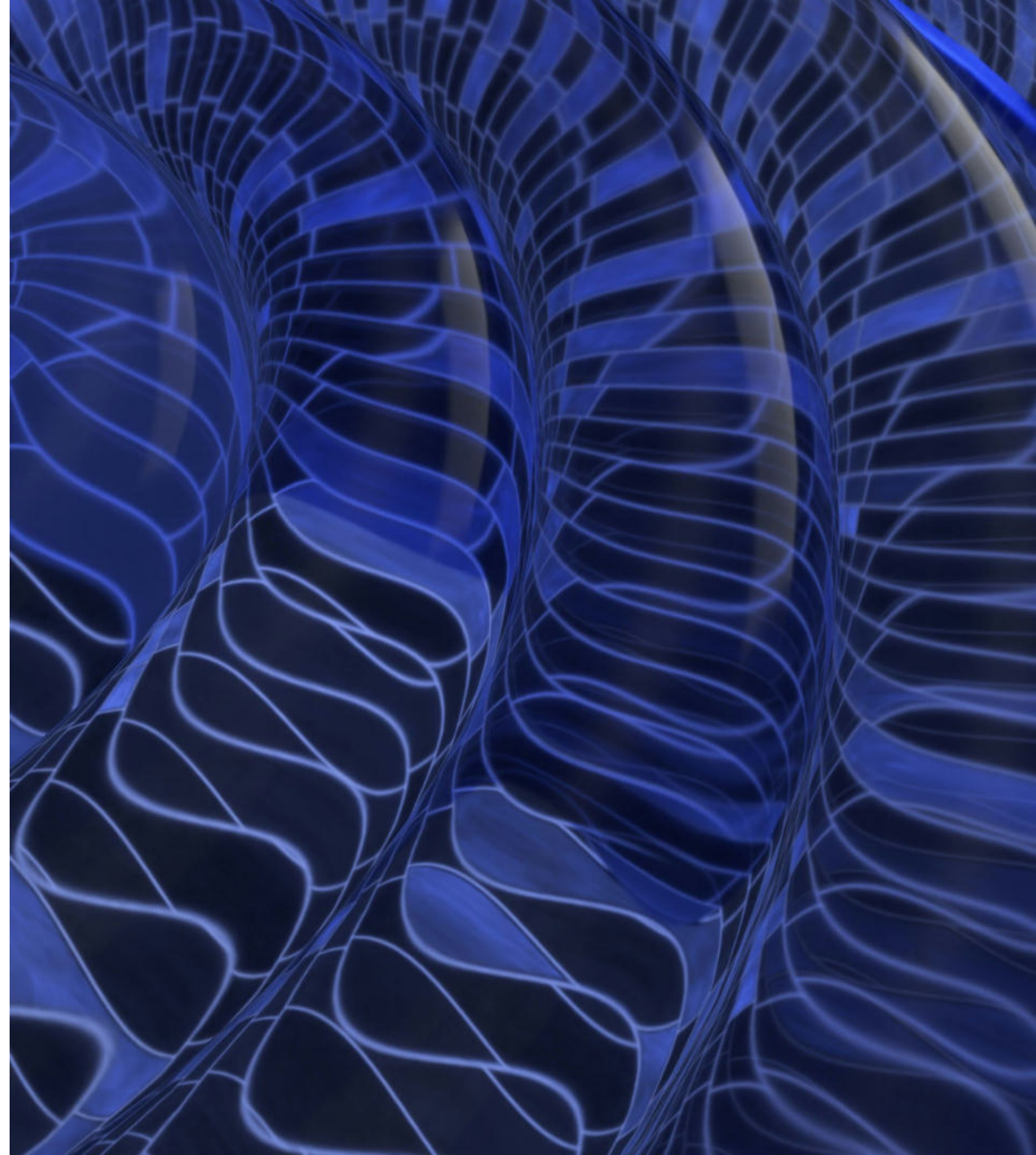
Memory Parameters - maintenance_work_mem

- `maintenance_work_mem`
 - Maximum memory in KB to be used in maintenance operations such as `VACUUM`, `CREATE INDEX`, and `ALTER TABLE ADD FOREIGN KEY`
 - Minimum allowed value is 1024 KB
 - It is set in KB
 - Performance for vacuuming and restoring database dumps can be improved by increasing this value



Memory Parameters - autovacuum_work_mem

- `autovacuum_work_mem`
 - Maximum amount of memory to be used by each autovacuum worker process
 - Default value is `-1`, indicates that `maintenance_work_mem` to be used instead



Memory Parameters - huge_pages

- `huge_pages`
 - Enables/disables the use of huge/large pages
 - Valid values are `try` (the default), `on`, and `off`
 - May help in increasing performance by using smaller page tables thus less CPU time on memory management
 - This parameter is only supported on Linux and Windows
- `huge_page_size`
 - Controls size of huge pages on Linux systems



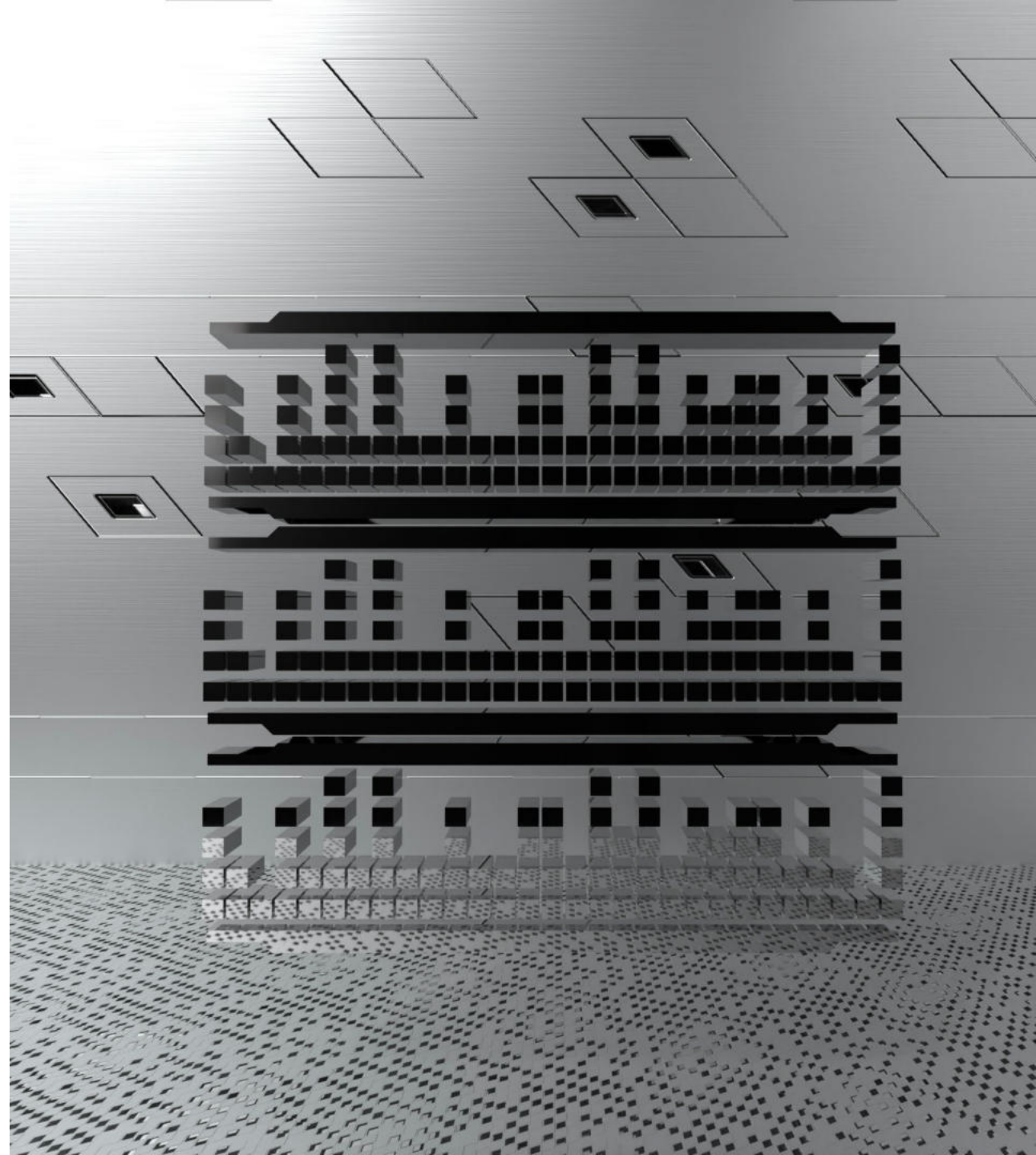
Memory Settings for Planner

- `effective_cache_size`
 - Size of memory available for disk cache that is available to a single query
 - A higher value favors index scans
 - This parameter does not reserve kernel disk cache; it is used only for estimation purposes
 - $\frac{1}{2}$ of RAM is a conservative setting
 - $\frac{3}{4}$ of RAM is more aggressive
 - Find the optimal value looking at OS stats after increasing or decreasing this parameter



Temporary Files

- `temp_file_limit`
 - Maximum amount of disk space that a session can use for temporary files
 - A transaction attempting to exceed this limit will be cancelled
 - Default is `-1` (no limit)
 - This setting constrains the total space used at any instant by all temporary files used by a given Postgres session



WAL Parameters - wal_level

- `wal_level`
 - `wal_level` determines how much information is written to the WAL
 - The default value is `replica`, adds logging required for WAL archiving as well as information required to run read-only queries on a Replica server
 - The value `logical` is used to add information required for logical decoding
 - The value `minimal`, removes all logging except the information required to recover from a crash or immediate shutdown
 - This parameter can only be set at server start



WAL Parameters - wal_buffers

- `wal_buffers`
 - Number of disk-page buffers allocated in shared memory for WAL data
 - Each buffer is 8K bytes
 - Needs to be only large enough to hold the amount of WAL data created by a typical transaction since the WAL data is flushed out to disk upon every transaction commit
 - Minimum allowed value is 4
 - Default setting is -1 (auto-tuned)



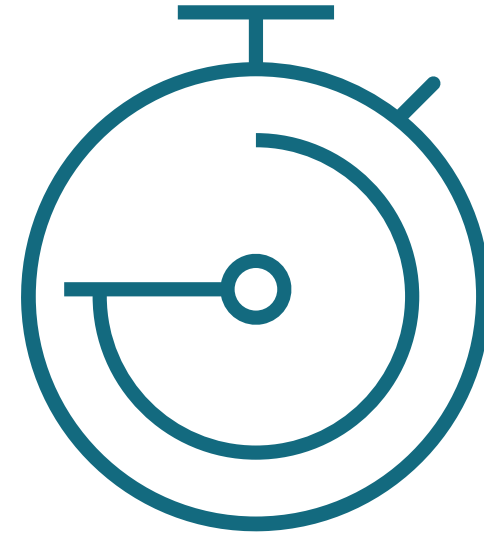
WAL Parameters – Checkpoints and max_wal_size

- Checkpoints
 - Writes the current in-memory modified pages (known as dirty pages) to the disk
 - An automatic checkpoint occurs each time the `max_wal_size` is reached
- `max_wal_size` (default: 1GB)
 - Maximum distance between automatic WAL checkpoints
 - Each log file segment is 16 megabytes and the size can be changed at the time of initialization using `--wal-segsize`
 - A checkpoint is forced when the `max_wal_size` is reached
 - `max_wal_size` is soft limit and WAL size may exceed during heavy load, failed archive command, or `high_wal_keep_size`
 - Increase in `max_wal_size` also increases mean time to recovery



WAL Parameters - checkpoint_timeout

- `checkpoint_timeout`
 - Maximum time between automatic WAL checkpoints in seconds before a checkpoint is forced
 - A larger setting results in fewer checkpoints
 - Range is 30 – 3600 seconds
 - The default is 300 seconds



WAL Parameters - fsync

- `fsync`
 - Ensures that all the WAL buffers are written to the WAL logs at each COMMIT
 - When on, `fsync()` or other `wal_sync_method` is forked
 - Turning this off will be a performance boost but there is a risk of data corruption
 - Can be turned off during initial loading of a new database cluster from a backup file
 - `synchronous_commit = off` can provide similar benefits for noncritical transactions without any risk of data corruption



pg_test_fsync Tool

- wal_sync_method is used for forcing WAL updates out to disk
- pg_test_fsync can determine the fastest wal_sync_method on your specific system
- pg_test_fsync reports average file sync operation time
- Diagnostic information for an identified I/O problem



Parallel Queries – Parallel Plans

- Parallel scans – The following types of parallel table scans are supported:
 - Parallel sequential scan – Allows multiple workers to perform a sequential scan
 - Parallel bitmap heap scan – Allows a single index scan to dispatch parallel workers to process different areas of the heap
 - Parallel index scan or parallel index-only scan – Allows B-tree index pages to be searched by separate parallel workers
- Parallel joins – Allows nested loop, hash join or merge joins to be performed in parallel
- Parallel aggregation – Allows queries with aggregations to be parallelized
- Parallel DDLs - `CREATE TABLE AS SELECT`, `CREATE INDEX`, and `CREATE MATERIALIZED VIEW`
- Parallel execution of FULL and Right joins, optimizing join operations for efficiency
- Parallel execution support for `string_agg` and `array_agg` aggregate functions



Parallel Query Scan Parameters

- Postgres supports parallel execution of the read-only queries
- Parallel scans can be enabled and configured using various configuration parameters

Parameter Name	Default Value
enable_parallel_append	on
enable_parallel_hash	on
max_parallel_maintenance_workers	2
max_parallel_workers	8
max_parallel_workers_per_gather	2
min_parallel_index_scan_size	64
min_parallel_table_scan_size	1024
parallel_setup_cost	1000
parallel_tuple_cost	0.1



Example – Parallel Query Scan

- Create table and insert data

```
edb=# create table test(id int);  
CREATE TABLE  
edb=# insert into test values (generate_series(1,100000000));  
INSERT 0 100000000  
edb=# analyze test;  
ANALYZE
```

- Disable parallel scan and check execution time:

```
edb=# set max_parallel_workers_per_gather=0;  
SET  
edb=# explain analyze select * from test where id=1;
```

QUERY PLAN

```
Seq Scan on test (cost=0.00..1692478.40 rows=1 width=4) (actual time=79.  
147..16043.048 rows=1 loops=1)  
  Filter: (id = 1)  
    Rows Removed by Filter: 99999999  
    Planning Time: 0.181 ms  
    JIT:  
      Functions: 2  
      Options: Inlining true, Optimization true, Expressions true, Deforming  
true  
      Timing: Generation 1.852 ms (Deform 0.061 ms), Inlining 9.836 ms, Optim  
ization 51.837 ms, Emission 17.222 ms, Total 80.748 ms  
      Execution Time: 16045.004 ms  
(9 rows)
```



Example – Parallel Query Scan (continued)

Enable parallel scan and check execution time:

```
edb=# set max_parallel_workers_per_gather=2;  
SET  
edb=# explain analyze select * from test where id=1;
```

QUERY PLAN

```
-----  
Gather  (cost=1000.00..964311.43 rows=1 width=4) (actual time=17.501..2913.952 rows=1 loops=1)  
  Workers Planned: 2  
  Workers Launched: 2  
    -> Parallel Seq Scan on test  (cost=0.00..963311.33 rows=1 width=4) (actual time=1888.716..2843.390 rows=0 loops=3)  
        Filter: (id = 1)  
        Rows Removed by Filter: 33333333  
Planning Time: 0.273 ms  
JIT:  
  Functions: 6  
  Options: Inlining true, Optimization true, Expressions true, Deforming true  
  Timing: Generation 1.025 ms (Deform 0.247 ms), Inlining 277.504 ms, Optimization 22.777 ms, Emission 22.524 ms, Total 323.8  
29 ms  
Execution Time: 2914.229 ms  
(12 rows)
```



Loading a Table into Memory

- `pg_prewarm`
 - Can be used to load relation data into either the operating system buffer cache or into the PostgreSQL buffer cache
 - Supports `prefetch` method for operating system buffer cache and `buffer` method for PostgreSQL buffer cache
- `pg_prewarm` can be used to automatically load shared blocks at the time of server restart
- Add `pg_prewarm` to `shared_preload_libraries` to start autoprewarm master process
- Configuration Parameters:
 - `pg_prewarm.autoprewarm` (default is on)
 - `pg_prewarm.autoprewarm_interval` (default is 300 seconds)



pg_prewarm Example

- Install Postgres contrib(installed by default with server packages):
 - `dnf install edb-asxx-server-contrib`
- Add pg_prewarm extension to a database and load table data into shared buffers:

```
pgtest=# CREATE EXTENSION pg_prewarm;
CREATE EXTENSION
pgtest=# SELECT pg_prewarm('pgbench_tellers','buffer');
 pg_prewarm
-----
          552
(1 row)

pgtest=#
```



Best Practices for Inserting Large Amounts of Data

- While inserting data using multiple inserts use `BEGIN` at the start and `COMMIT` at the end
- Use `COPY` to load all the rows in one command, instead of using a series of `INSERT` commands
- If you cannot use `COPY`, it might help to use `PREPARE` to create a prepared `INSERT` statement, and then use `EXECUTE` as many times as required
- If you are loading a freshly created table, the fastest method is to create the table, bulk load the table's data using `COPY`, then create any indexes needed for the table. EDB*Loader of Enterprise Postgres is 2x faster than `COPY`
- It might be useful to drop foreign key constraints, load the data, and then re-create the constraints



Best Practices for Inserting Large Amounts of Data (Cont.)

- Temporarily increasing the `maintenance_work_mem` and `max_wal_size` configuration variables when loading large amounts of data can lead to improved performance
- Disable WAL Archival and Streaming Replication
- Triggers and Autovacuum can also be disabled
- Certain commands run faster if `wal_level` is minimal:
 - `CREATE TABLE AS SELECT`
 - `CREATE INDEX` (and variants such as `ALTER TABLE ADD PRIMARY KEY`)
 - `ALTER TABLE SET TABLESPACE`
 - `CLUSTER`
 - `COPY FROM`, when the target table has been created or truncated earlier in the same transaction



Non-Durable Settings

- Durability guarantees the recording of committed transactions but adds significant overhead.
- Postgres can be configured to run without durability.
- Turn off `fsync` when there is no need to flush wal data to disk.
- Turn off `full_page_writes`.
- Increase `max_wal_size` and `checkpoint_timeout`; this reduces the frequency of checkpoints.
- Turn off `synchronous_commit` when there is no need to write the WAL to disk on every commit.



PEM Tuning Wizard

- The PEM Tuning Wizard reviews your PostgreSQL installation and recommends a set of configuration options to tune your server
- You must specify `Service ID` field on the `Advanced` tab of the server's `Properties` dialog
- It can make recommendations for servers that reside on the same server as their bound PEM agent
- If a value of `Yes` is specified in the `Remote monitoring` field while defining the server then it will not be displayed in Tuning Wizard tree control



PEM Tuning Wizard – Welcome and Select Servers

Tuning Wizard - Welcome (step 1 of 5)

The Tuning Wizard contains the following steps:

- 1 Welcome
- 2 Select Servers
- 3 Configuration
- 4 Tuning Changes Summary
- 5 Schedule or Run?

Welcome to the Tuning Wizard

This wizard will perform basic tuning of your Postgres installations based on the options you specify and your hardware specifications.

Note that systems under very high load or used for benchmarking may also require further manual tuning for the specific workload to obtain maximum performance.

? Cancel Back Next Finish

Tuning Wizard - Select Servers (step 2 of 5)

The Tuning Wizard contains the following steps:

- 1 Welcome ✓
- 2 Select Servers
- 3 Configuration
- 4 Tuning Changes Summary
- 5 Schedule or Run?

Select the server(s) you wish to tune

- ✓ Servers
 - ✓ EPAS-17 (localhost:5445)
 - ✓ Postgres Enterprise Manager Server (127.0.0.1:5444)

? Cancel Back Next Finish



PEM Tuning Wizard – Configuration and Tuning Changes Summary

Tuning Wizard - Configuration (step 3 of 5)

The Tuning Wizard contains the following steps:

- 1 Welcome ✓
- 2 Select Servers ✓
- 3 Configuration
- 4 Tuning Changes Summary
- 5 Schedule or Run?

Machine utilization: ☒ Dedicated Mixed use Developer workstation

This machine is dedicated to run Postgres and will use available memory to optimize performance.

Workload selection: ☒ OLTP Mixed Data warehouse

The running application is transaction-intensive application.

Buttons: ? Cancel Back Next Finish

Tuning Wizard - Tuning Changes Summary (step 4 of 5)

The Tuning Wizard contains the following steps:

- 1 Welcome ✓
- 2 Select Servers ✓
- 3 Configuration ✓
- 4 Tuning Changes Summary
- 5 Schedule or Run?

Please review the suggested changes to tune your selected servers.

- ✓ Servers
 - ✓ EPAS-17 (localhost:5445)
 - ✓ effective_cache_size = 841MB
 - ✓ maintenance_work_mem = 58MB
 - ✓ max_wal_size = 1GB
 - ✓ min_wal_size = 80MB
 - ✓ random_page_cost = 2
 - ✓ shared_buffers = 448MB *
 - ✓ wal_buffers = 16MB *
 - ✓ work_mem = 6MB
 - ✓ Postgres Enterprise Manager Server (127.0.0.1:5444)
 - ✓ effective_cache_size = 841MB
 - ✓ maintenance_work_mem = 58MB
 - ✓ max_wal_size = 1GB
 - ✓ min_wal_size = 80MB
 - ✓ random_page_cost = 2
 - ✓ shared_buffers = 448MB *
 - ✓ wal_buffers = 16MB *
 - ✓ work_mem = 6MB

(*) Indicates parameter requires server restart.

Buttons: ? Cancel Back Next Finish



PEM Tuning Wizard – Schedule/Run and Generate Report

Tuning Wizard - Schedule or Run? (step 5 of 5)

The Tuning Wizard contains the following steps:

- 1 Welcome ✓
- 2 Select Servers ✓
- 3 Configuration ✓
- 4 Tuning Changes Summary ✓
- 5 Schedule or Run? >

Schedule/Generate Tuning Changes

☒ Schedule changes ☐ Generate report

Configure now? ☒ Yes ☐ No

Time?

NOTE: The selected servers will be restarted to complete tuning operation.

? Cancel Back Next Finish

Tuning Wizard - Schedule or Run? (step 5 of 5)

The Tuning Wizard contains the following steps:

- 1 Welcome ✓
- 2 Select Servers ✓
- 3 Configuration ✓
- 4 Tuning Changes Summary ✓
- 5 Schedule or Run? >

Schedule/Generate Tuning Changes

☐ Schedule changes ☒ Generate report

View report now? ☒ Yes ☐ No

Save the report to file

Note: Filename can only be latin-1 compatible characters.

? Cancel Back Next Finish



PEM Tuning Wizard Report

Tuning Wizard Report

📅 Generated On: 2025-01-15 01:40:32

Go to: EPAS-17

Summary

☰

 Number of servers selected: 2

🕒

 Machine utilization: Dedicated

📄

 Workload profile: OLTP

▼ Server: EPAS-17 (localhost:5445)

GUC Parameter	Original Value	Recommended Value
effective_cache_size	4096MB	841MB
maintenance_work_mem	64MB	58MB
random_page_cost	4	2
shared_buffers	128MB	448MB
wal_buffers	4MB	16MB
work_mem	4MB	6MB



SQL Profiler

- SQL Profiler is a graphical wizard available in PEM
- PEM SQL Profiler capabilities are very similar to Microsoft SQL Server's Profiler
- It can be used to:
 - Capture workloads in a SQL trace
 - Monitor and analyze SQL
 - Diagnose slow-running queries
 - View the query execution statistics
 - Schedule a SQL trace to run during heavy workloads
 - Get advice on indexes
- SQL Profiler plug-in must be installed and configured for each Postgres database



Capacity Planning using Capacity Manager

- PEM contains built-in capabilities for performing database capacity planning
- Capacity planning helps by providing answers to questions like:
 - How much storage will my database need 6 months from now?
 - How fast is my database growing?
 - What objects are responsible for the growth in my database?
 - Will my server be able to support another database instance?
 - Is the performance of my database getting better, staying the same, or getting worse?

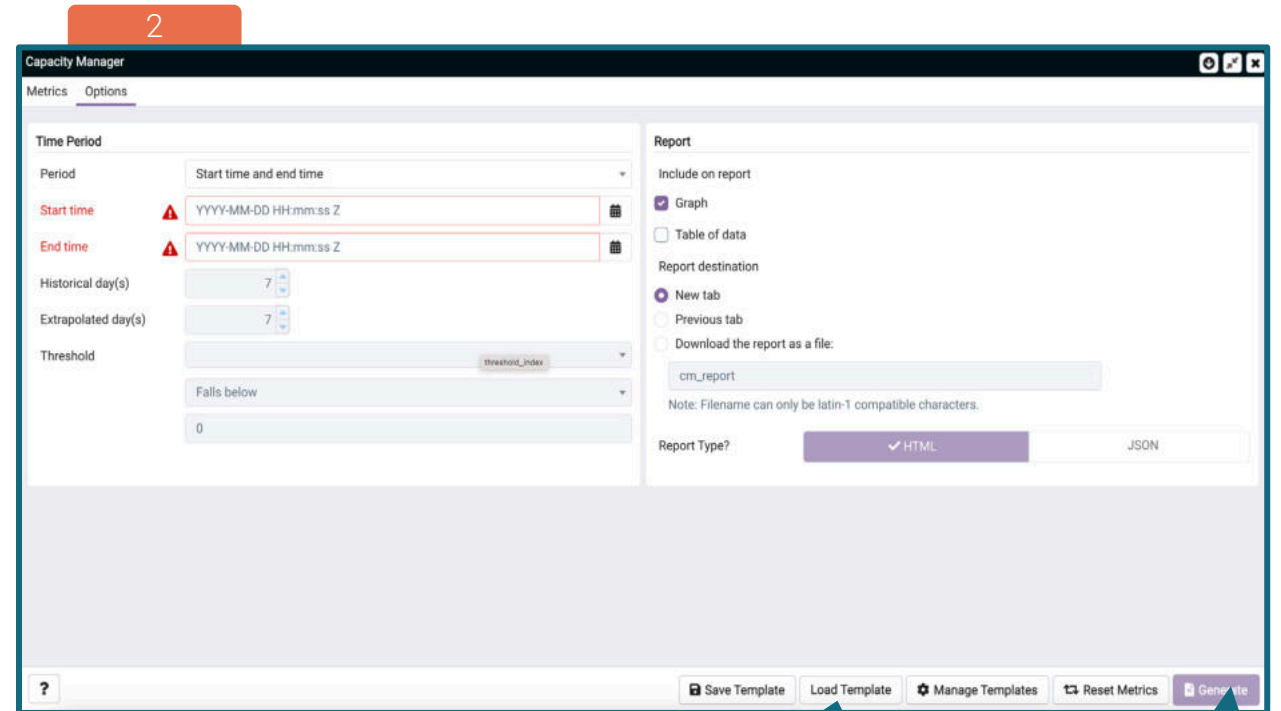
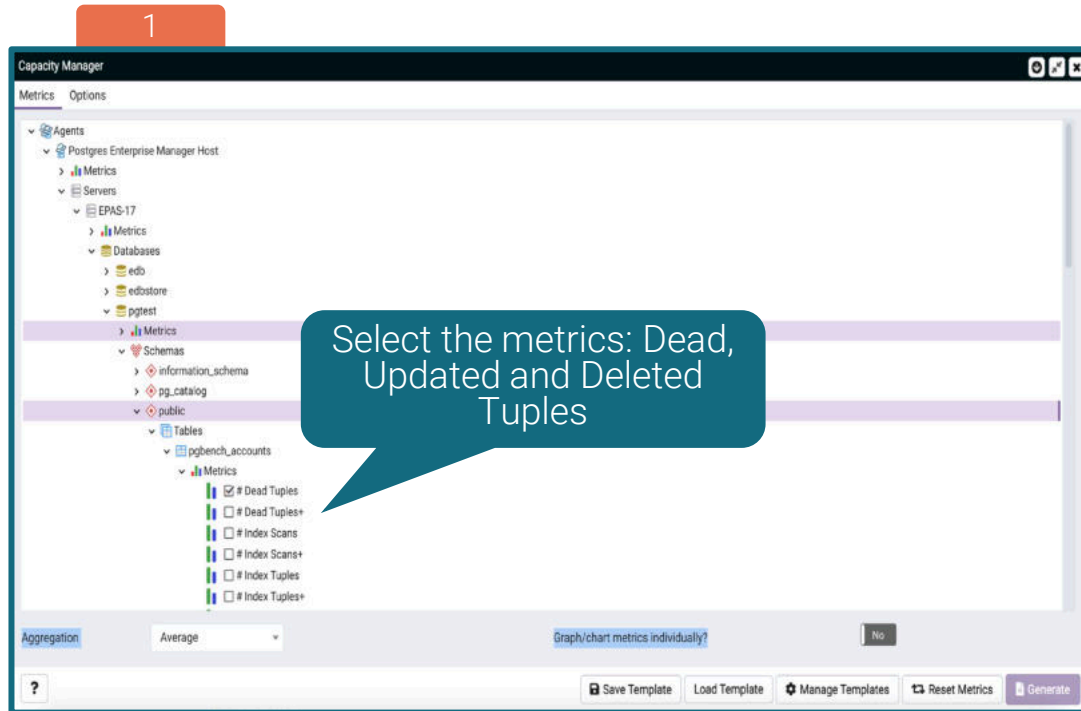


Capacity Manager

- Analyzes collected statistics to generate a graph or table
- Displays the historical usage statistics of an object
- Can project the anticipated usage statistics for an object
- Analyze metrics for a specific:
 - Host/operating system
 - Enterprise Postgres or PostgreSQL server
 - Database
 - Database object (table, index, function etc.)
- You can choose a specific metric (or metrics) to include in the report
- You can also specify a start and end date for the Capacity Manager report



Example - Capacity Manager

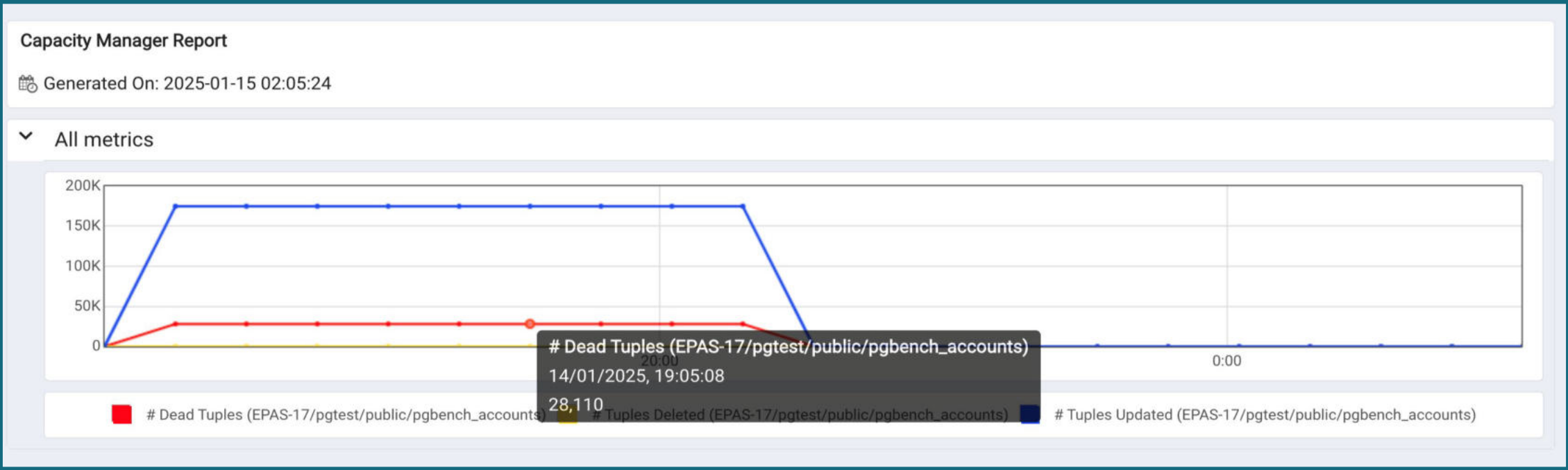


Buttons for Saving and Loading Templates

Click Generate for Report



Sample Capacity Manager Report





DRITA



DRITA

- Dynamic Runtime Instrumentation Tools Architecture (DRITA):
 - Allows DBAs to query catalogs
 - Determines the wait events affecting performance
 - DRITA records wait event information while consuming minimal resources
 - Compares snapshots to evaluate the performance of a system
 - Snapshots identified by a unique ID number
 - Functions available for creating, viewing and comparing snapshots
 - DRITA consumes minimal system resources



DRITA Snapshots

- A snapshot is a saved set of system performance data at a given point in time
- A unique ID number identifies each snapshot
- Snapshot ID numbers are used with DRITA reporting functions to return system performance statistics

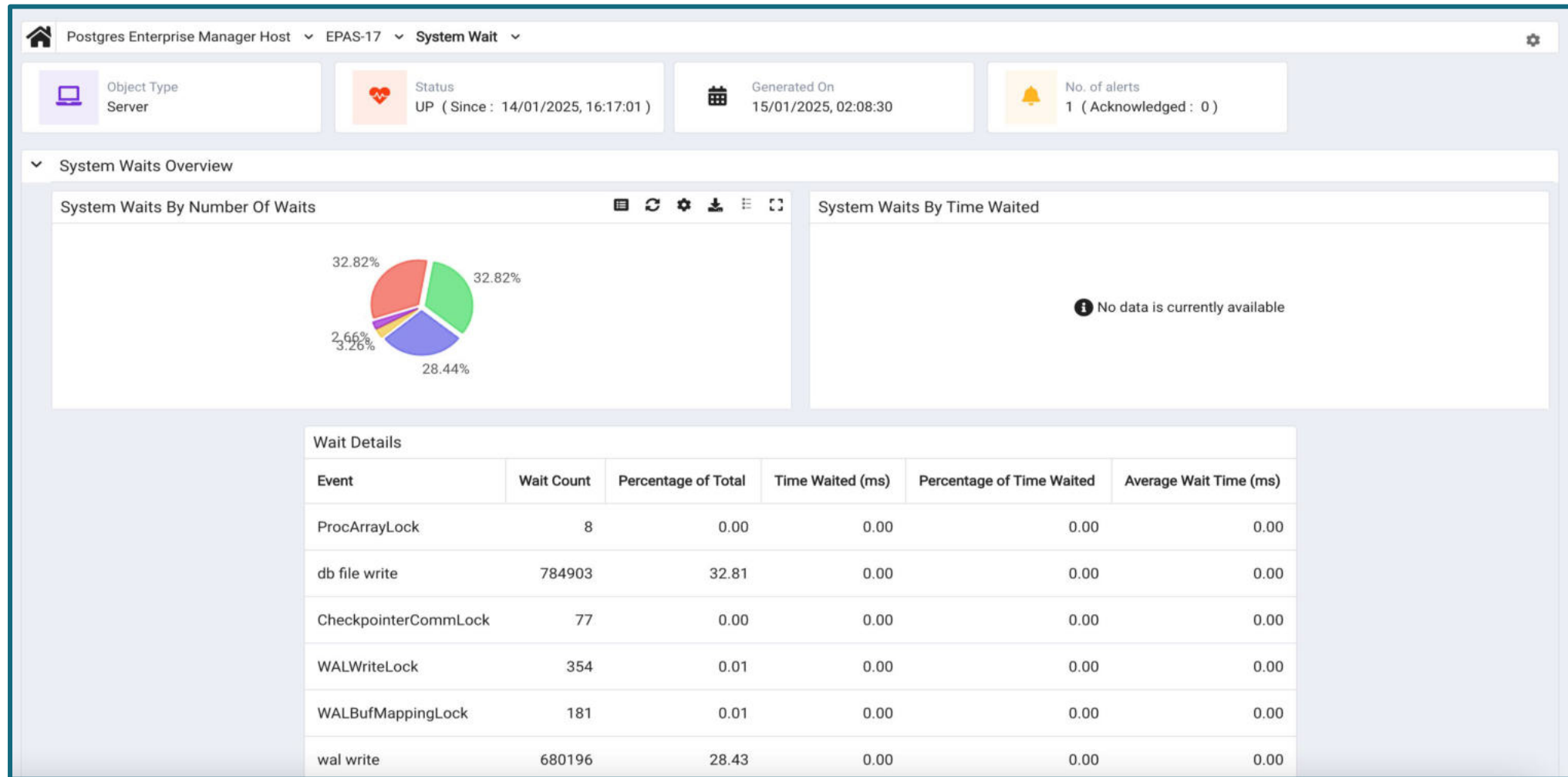


Using DRITA Reports for Performance Tuning

- Review the top five events in a given report
- Look for any event that takes a disproportionately large percentage of resources
- Waits should be evaluated in the context of CPU usage and total time
- When evaluating events, watch for:
 - Checkpoint waits
 - WAL-related waits
 - SQL parse waits
 - db file random reads
 - db file random writes
 - btree random lock acquires



PEM – System Wait Analysis Dashboard



Lab Exercise - 1

- Users are complaining about slower than normal performance on `edbstore` database
- Tune `postgresql.conf` parameters for optimal performance based on the `edbstore` database size, largest table, and other necessary information collected from `edbstore` database
- Change maximum concurrent connections on the cluster to 50



Lab Exercise - 2

- Write a statement to load `customers` table from `edbstore` database to the PostgreSQL buffer cache



Lab Exercise - 3

- `pg_test_fsync` can determine the fastest `wal_sync_method` on a specific system. Run this tool on your local machine and determine the best `wal_sync_method` settings for your system.





Performance Benchmarking



Module Objectives

- Database Benchmarking
- Benchmarking Tools
- pgbench
- Preparing a database for pgbench
- Running a pgbench test
- Default Script



Database Benchmarking

- Measures the performance of database features essential to applications
- Helps in preparing the system
- Helps in detecting a performance shortfall
- Can evaluate database parameter changes and help in calculating optimal parameter values
- Identifies problematic areas
- Can be used for comparing different databases
- Helps make file system choice for data directory



Benchmarking Tools

- Various open-source database performance benchmarking tools can be used with PostgreSQL:
 - pgbench
 - pgbench-tools
 - BenchmarkSQL
 - HammerDB



pgbench

- Simple program for running benchmark tests on PostgreSQL
- pgbench executable is by default located in `/usr/edb/asXX/bin/`
- Based on TPC-B
- Bank, Accounts, and Tellers model is used
- Derived from JDBC Bench eliminating Java overhead



pgbench Working

- Connect multiple database sessions and run the same SQL multiple times
- Calculates the average transaction rate (transactions per second)
- Default workload involves five `SELECT`, `UPDATE`, and `INSERT` commands per transaction
- TPC-B-like test requires specific tables in a database



Preparing a Database for pgbench - Invoke

- Invoke pgbench with `-i` to create and populate required tables

```
pgbench -i [ other-options ] dbname
```

```
[postgres@pgsrv1 ~]$ pgbench -i -s 50 pgtest  
dropping old tables...  
creating tables...  
generating data (client-side)...  
5000000 of 5000000 tuples (100%) done (elapsed 24.29 s, remaining 0.00 s)  
vacuuming...  
creating primary keys...  
done in 39.76 s (drop tables 0.03 s, create tables 0.01 s, client-side generate 24.69  
s, vacuum 3.41 s, primary keys 11.61 s).
```

- pgbench can run a custom script and will not require initialization



Preparing a Database for pgbench - Verify

- `pgbench -i` creates four tables
- Options available with `pgbench` during initialization:
 - `-s` : scale factor to control the number of rows
 - `-F` : fill factor (default 100)
 - `--unlogged-tables`: create all tables as unlogged
 - `--tablespace`
 - `--index-tablespace`



Running a pgbench Test

- Run your benchmark with the pgbench command without using `-i`

```
pgbench [ options ] dbname
```

- Options:

-c clients	Number of concurrent database sessions. Default is 1
-C	Establish a new connection for each transaction
-f filename	Read transaction script from filename
-j threads	Number of worker threads within pgbench
-S	Perform select-only transactions instead of a TPC-B like test
-t	Number of transactions each client runs. Default is 10
-T seconds	Run the test for this many seconds
--log-prefix	Prefix for transaction time log file (default; "pgbench_log")
-?	Help



Default Script

```
BEGIN;  
UPDATE pgbench_accounts SET abalance = abalance + :delta WHERE aid = :aid;  
SELECT abalance FROM pgbench_accounts WHERE aid = :aid;  
UPDATE pgbench_tellers SET tbalance = tbalance + :delta WHERE tid = :tid;  
UPDATE pgbench_branches SET bbalance = bbalance + :delta WHERE bid = :bid;  
INSERT INTO pgbench_history (tid, bid, aid, delta, mtime) VALUES (:tid, :bid,  
:aid, :delta, CURRENT_TIMESTAMP);  
END;
```



Example - pgbench

```
[postgres@pgcentos1 ~]$ pgbench -c 50 -T 60 testdb
Password:
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 1
query mode: simple
number of clients: 50
number of threads: 1
duration: 60 s
number of transactions actually processed: 54038
latency average = 55.672 ms
tps = 898.125237 (including connections establishing)
tps = 898.250706 (excluding connections establishing)
[postgres@pgcentos1 ~]$
```





Extensions



Module Objectives

- What are Extension modules?
- Installing Extension Modules
- Add Extension to a Database
- Extensions List



What are Extension Modules?

- Additional Supplied Modules found in the “extension” directory provides additional functionalities
- Extensions are not included in the core database and may be still under development
- Extensions are also tied to that particular version of Postgres
- They are part of contrib package and can be installed using
 - `dnf install edb-asxx-server-contrib`
- Package installer places extensions in directory:
`/usr/edb/asxx/share/extension/`



Add Extension to a Database

- You need to register the new objects in the database system by running the `CREATE EXTENSION` command
- Alternatively, run it in database template1 so that the module will be copied into subsequently-created databases by default
- Syntax:

```
=# CREATE EXTENSION module_name;
```

- This command must be run by a database superuser



Available Extensions

- `adminpack` - File and log manipulation routines, used by pgAdmin
- `btree_gist` - Support for emulating B-Tree indexing in GiST
- `cube` - Multidimensional-cube datatype (GiST indexing example)
- `dblink` - Allows remote query execution
- `earthdistance` - Operator for computing Earth distance for two points
- `fuzzystrmatch` - Levenshtein, metaphone, and soundex fuzzy string matching
- `hstore` - Module for storing (key,value) pairs
- `pg_visibility` – To examine the visibility map and page-level visibility info
- `bloom` – Provides an index access method based on bloom filters



Available Extensions(Continued)

- `intagg` - Integer aggregator
- `intarray` - Index support for arrays of int4, using GiST
- `isn` - Postgres type extensions for ISBN, ISSN, ISMN, EAN13 product numbers
- `lo` - Large Object maintenance
- `ltree` – Adds ltree data type for storing hierarchical tree-like structure
- `pg_buffercache` - Real time queries on the shared buffer cache
- `pg_freemap` - Displays the contents of the free space map (FSM)
- `pgcrypto` - Cryptographic functions



Available Extensions (Continued)

- [pg_trgm](#) - Functions for determining the similarity of text based on trigram matching
- [pgrowlocks](#) - A function to return row locking information
- [pgstattuple](#) - A function to return statistics about "dead" tuples and free space within a table
- [seg](#) - Confidence-interval datatype (GiST indexing example)
- [spi](#) - Various trigger functions, examples for using SPI
- [sslinfo](#) - Functions to get information about SSL certificate
- [uuid-oss](#) - Provide functions to generate UUIDs
- [edb_job_scheduler](#) – EDB Job is an extension that runs Scheduler the job scheduler as a background process for the DBMS_SCHEDULER and DBMS_JOB packages.



EDB Job Scheduler

- EDB Job Scheduler eliminates the requirement of pgAgent for `DBMS_SCHEDULER` and `DBMS_JOB`
- EDB Job Scheduler is an extension that runs as a background process residing in the `contrib/dbms_scheduler_ext` subdirectory
- Components:
 - Launcher process
 - Scheduler process

Configure and Setup Job Scheduler

Parameters changes in postgresql.conf:

```
edb_job_scheduler.database_list = 'edb'  
  
shared_preload_libraries='$libdir/edb_job_scheduler'
```

Restart the database cluster

Connect with edb database and execute:

```
CREATE EXTENSION edb_job_scheduler;
```

Catalog Table for Monitoring:

```
sys.jobs
```



Example

```
edb=# CREATE EXTENSION lo;
CREATE EXTENSION
edb=# CREATE TABLE images (img_id NUMBER,img LO);
CREATE TABLE
edb=# INSERT INTO images values(1,lo_import('/home/enterprisedb/logo.png'));
INSERT 0 1
edb=# TABLE images;
  img_id |  img
-----+-----
        1 | 65596
(1 row)

edb=# SELECT lo_export(img,'/home/enterprisedb/logo.png') FROM images WHERE img_id=1;
 lo_export
-----
        1
(1 row)
```



Lab Exercise - 1

- In this lab, you will be working with a Postgres extension to enhance the functionality of your database.
 1. Install and configure extension named hstore in edbstore database
 2. Perform a basic test or use case that demonstrates the functionality provided by the extension



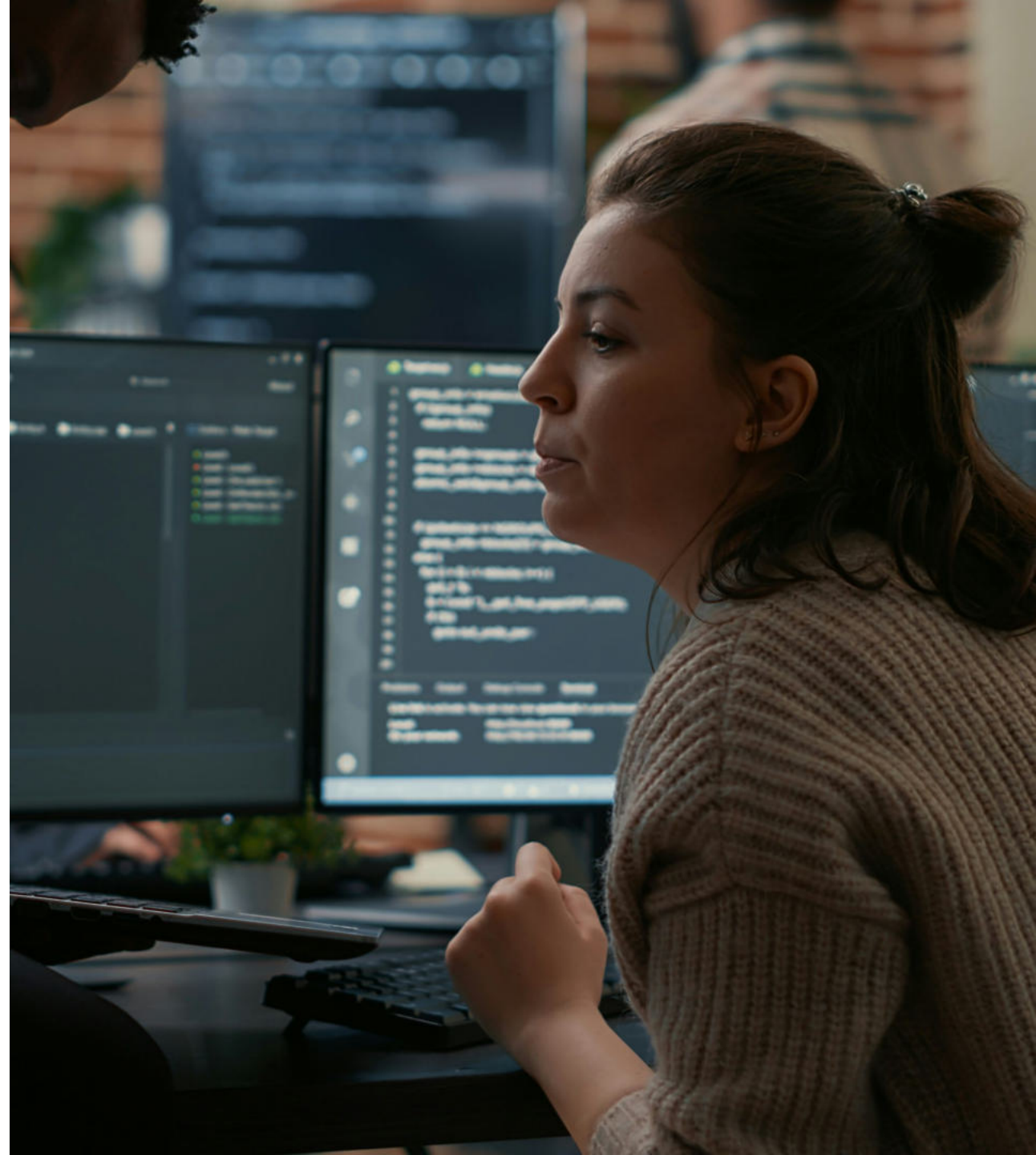


Table Partitioning



Module Objectives

- Partitioning Benefits
- Partitioning Methods and Syntax
- Partition Pruning
- Adding New Partitioned Table
- Partitioning an Existing Table
- Moving and Dropping a Partition
- Handling Stray Values
- Information Views



Partitioning

- Partitioning refers to splitting what is logically one large table into smaller physical pieces
- Query performance can be improved dramatically for certain kinds of queries
- Improved UPDATE performance. When an index no longer fits easily in memory, both read and write operations on the index take progressively more disk accesses



Partitioning Benefits

- Partitioning is useful for large tables especially when a table is bigger than physical memory
- Can be used when different storage devices must be used for single table
- Seldom-used data can be migrated to cheaper and slower storage media
- Bulk loads or deletes may be accomplished by simply adding or removing one of the partitions
- Partitioning avoids VACUUM overhead caused by a bulk DELETE
- Dramatic performance improvements for certain kinds of queries
- Improved UPDATE performance
- Index operations takes less time



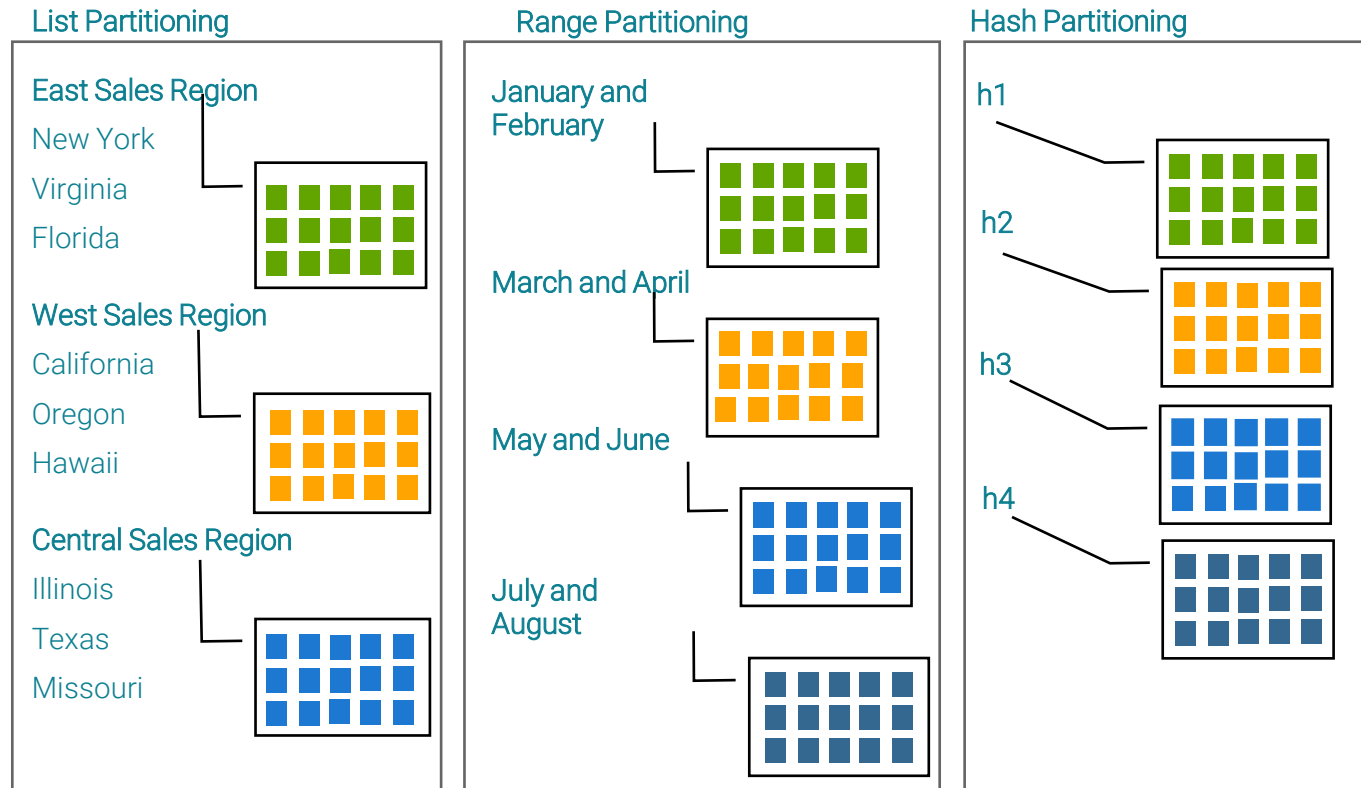
When to Partition - Suggestions

- Here are some suggestions for when to partition:
 - When a large table begins having performance issues
 - Size of the table has exceeded the physical memory
 - Tables greater than 20GB may be considered for table partitioning
 - Tables containing historical data where the newest data can be added to a new partition
 - A typical example is where 1 month of data is frequently updated and 11 months of data is read only
 - When the content of a table needs to be distributed on different storage devices depending on the usage of data
 - When queries only need to access a subset of data



Partitioning Methods

- Enterprise Postgres supports three types of partitioning methods:



Partitioning Syntax

```
=# CREATE TABLE [schema.]table_name
    table_definition
    PARTITION BY LIST|RANGE|HASH(column)
        [INTERVAL (constant | expression)] - ONLY FOR RANGE PARTITION
    [SUBPARTITION BY {RANGE|LIST|HASH(column[, column ]...)}
    (list_partition_definition[, list_partition_definition]...);
```

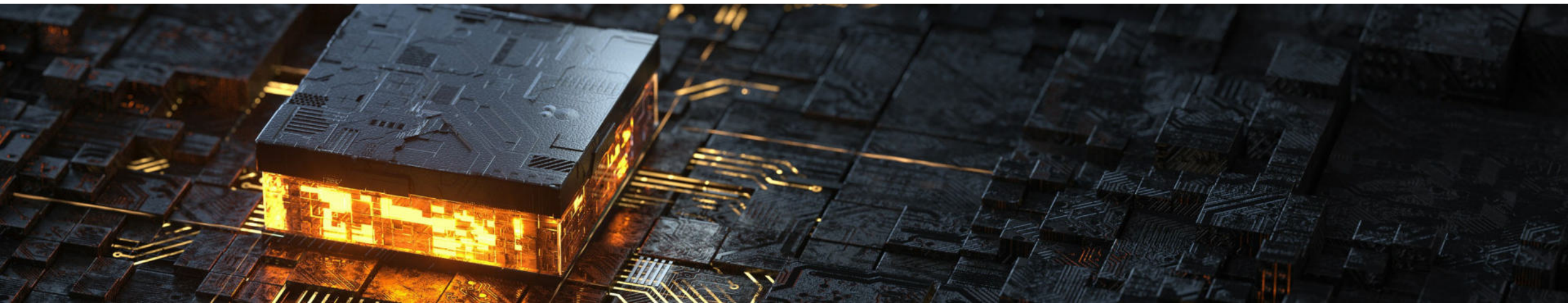
Where list_partition_definition is:

```
PARTITION [partition_name]
    VALUES (value[, value]...) -NO VALUES FOR HASH PARTITION
    [TABLESPACE tablespace_name]
    [(subpartition, ...)]
```



Attach and Detach Partitions

- A table can be added or attached as a partition to a partitioned table that exists
- An existing partition can be converted into a standalone table
- Example:
 - `ALTER TABLE users ATTACH PARTITION users_3 from (101) to (150);`
 - `ALTER TABLE users DETACH PARTITION users_3;`



Partition Pruning

- Query planner uses partition pruning to compute an efficient plan
- Query filter must use a partitioned column
- In a `LIST` partitioned table partition pruning is effective when the `WHERE` clause has `equal(=)` or `AND` operator
- In a `RANGE` partitioned table partition pruning is effective when the `WHERE` clause has `=`, `<` or `>` operators
- In a `HASH` partitioned table partition pruning is effective when the `WHERE` clause has `equal(=)` operator
- Partition pruning on Boolean columns



Fast Pruning

- Fast pruning occurs early in the planning process
- Only works on queries with WHERE or JOIN clauses
- Controlled by a parameter named `edb_enable_pruning`
- Reduces the number of partitions that the planner must consider
- Doesn't work on sub-partitioned tables or RANGE partitions partitioned on more than one column



Constraint Exclusion

- Constraint exclusion is like partition pruning except that it uses each table's check constraint
- Constraints defined for each partition are examined
- Controlled by the `constraint_exclusion` parameter which must be set to either `partition` or `on`
- Constraint exclusion only works when the query's WHERE clause contains constants or parameters
- It does not work if where clause is using any non-immutable function e.g `current_timestamp()`
- All constraints on all children of the parent table are examined
- Exclusion constraints support to prevent conflicting data in different partitions



Listing Partitioned Tables

- ALL_TAB_PARTITIONS
 - Provides partition-level partitioning information for current user
- ALL_TAB_SUBPARTITIONS
 - Describes sub-partitions accessible to the current user for all tables
- \dP meta command to list partitioned tables and indexes

```
edb=# \d ALL_TAB_PARTITIONS
```

View "sys.all_tab_partitions"	
Column	Type
table_owner	text
schema_name	text
table_name	text
composite	text
partition_name	text
subpartition_count	bigint
high_value	text
high_value_length	integer
partition_position	integer
tablespace_name	text
pct_free	numeric
pct_used	numeric
ini_trans	numeric
max_trans	numeric
initial_extent	numeric

```
edb=# \d ALL_TAB_SUBPARTITIONS
```

View "sys.all_tab_subpartitions"	
Column	Type
table_owner	text
schema_name	text
table_name	text
partition_name	text
subpartition_name	text
high_value	text
high_value_length	integer
subpartition_position	integer
tablespace_name	text
pct_free	numeric
pct_used	numeric
ini_trans	numeric
max_trans	numeric
initial_extent	numeric
next_extent	numeric



Adding A New Partitioned Table

- Use the `ALTER TABLE... ADD PARTITION` command to add a partition to an existing partitioned table
- The new partition must be of the same type
- Syntax:

```
=# ALTER TABLE table_name ADD PARTITION partition_definition;
```

Where `partition_definition` is

```
{list_partition | range_partition }
```

- Example:

```
=# ALTER TABLE sales ADD PARTITION east_asia VALUES ('CHINA', 'KOREA');
```



Partitioning An Existing Table

- The `ALTER TABLE...EXCHANGE PARTITION` command swaps an existing table with a partition or sub-partition
- Syntax of the `EXCHANGE PARTITION` command:

```
=# ALTER TABLE target_table  
    EXCHANGE PARTITION|SUBPARTITION target_partition|  
    target_subpartition  
    WITH TABLE source_table [(WITH | WITHOUT) VALIDATION];
```

- Preparing a new partition is resource intensive
- Load the data into a staging table and use `EXCHANGE PARTITION`



Move A Partition

- Use the `ALTER TABLE... MOVE PARTITION` command to move a partition or sub-partition to a different tablespace
- Syntax:

```
=# ALTER TABLE table_name  
  
    MOVE PARTITION partition_name  
  
    TABLESPACE tablespace_name;
```



Drop A Partition

- Use the `ALTER TABLE... DROP PARTITION` command to delete a partition and the data stored in that partition
- Syntax:

```
ALTER TABLE table_name DROP PARTITION partition_name;
```

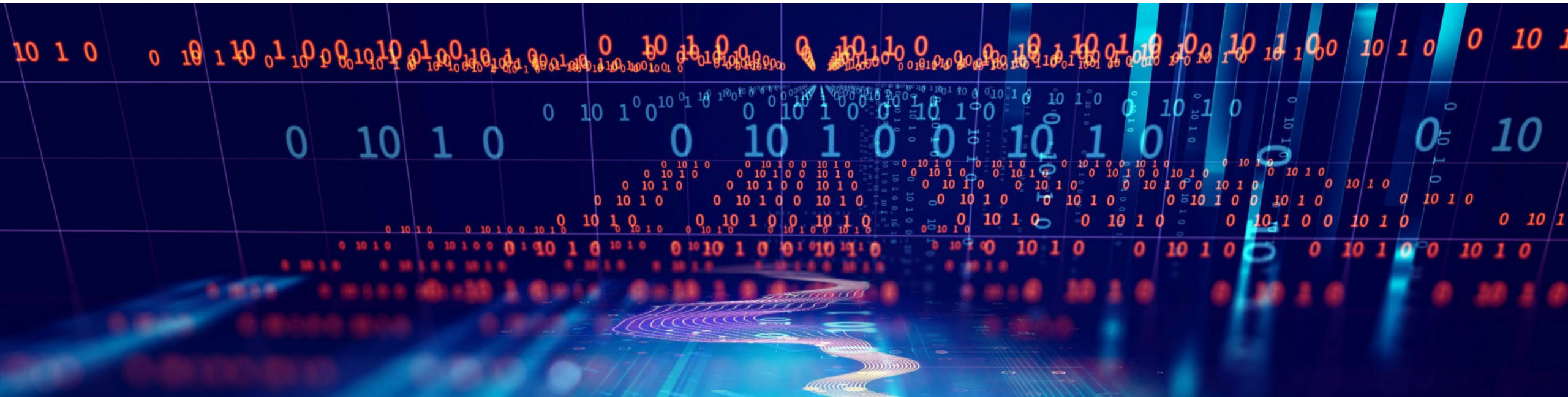
- The following example deletes the partition, `americas` from the `sales` table:

```
ALTER TABLE sales DROP PARTITION americas;
```



Dropping and Truncating

- A DROP or TRUNCATE on base table will DROP or TRUNCATE all the partitions
- Use the ALTER TABLE... DROP PARTITION command instead
- Use ALTER TABLE... TRUNCATE PARTITION command instead



Handling Stray Values

- Stray values are rows that do not meet any partitioning rules
- A DEFAULT or MAXVALUE partition or sub-partition can be used to capture such rows
- The syntax of a DEFAULT rule is:

```
PARTITION [partition_name] VALUES (DEFAULT)
```

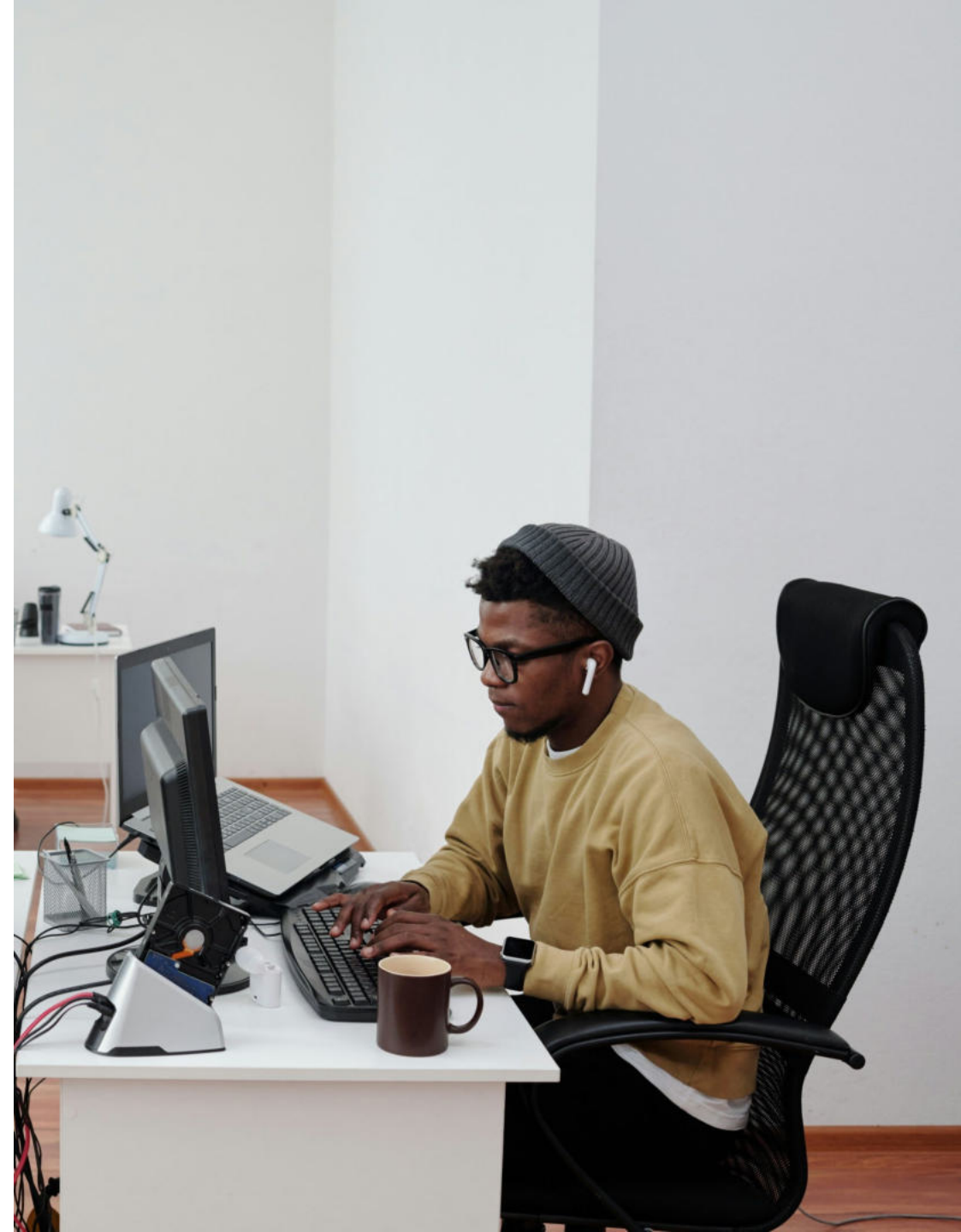
- The syntax of a MAXVALUE rule is:

```
PARTITION [partition_name] VALUES LESS THAN (MAXVALUE)
```



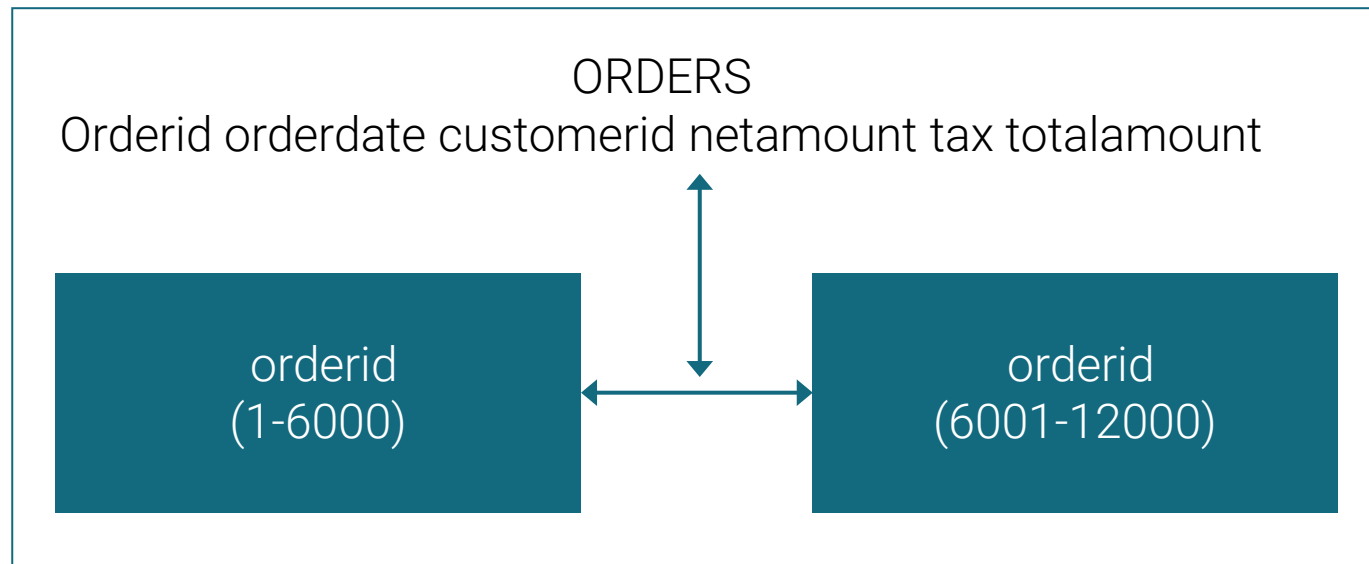
Information Views

- `ALL_PART_TABLES`
- `ALL_TAB_PARTITIONS`
- `ALL_TAB_SUBPARTITIONS`
- `ALL_PART_KEY_COLUMNS`
- `ALL_SUBPART_KEY_COLUMNS`



Lab Exercise - 1

- View the structure of orders table in edbstore database
- Partition existing orders table as shown in following diagram:



Lab Exercise - 2

- Create primary key on `orderid` column of each partitioned table
- View the execution plan of following query

```
SELECT * FROM orders WHERE orderid=3000;
```

- Turn off the `constraint_exclusion` and view execution plan of following query

```
SELECT * FROM orders WHERE orderid=3000;
```





Connection Pooling



Module Objectives

- Connection Pooling Overview
- pgpool-II - Features
- pgpool II - Installation and Configuration
- Starting/Stopping pgpool-II
- pgbouncer - Features
- pgbouncer - Installation and Configuration
- Starting/Stopping pgbouncer



Connection Pooling

- Each database client acquires a database connection from the database
- The Postmaster is responsible for listening and providing database connections
- Connection processes are destroyed once the clients log off
- Connection pooling:
 - Provides a mechanism to reuse the existing connection processes once the clients log off
 - Lowers connection overhead on the postmaster
 - Improves application database access performance
 - Spreads connection cost over repeated clients



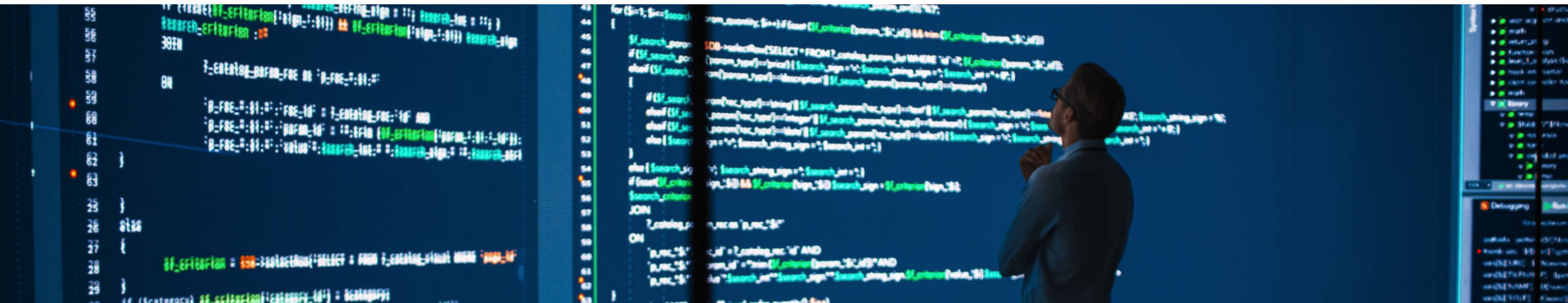
pgpool-II

- Open source
- Middleware between PostgreSQL servers and a PostgreSQL database client
- pgpool-II talks to PostgreSQL backend and frontend protocol, and relays a connection between them
- Features:
 - Connection Pooling
 - Replication
 - Load Balancing
 - Automatic Failover
 - Parallel Query



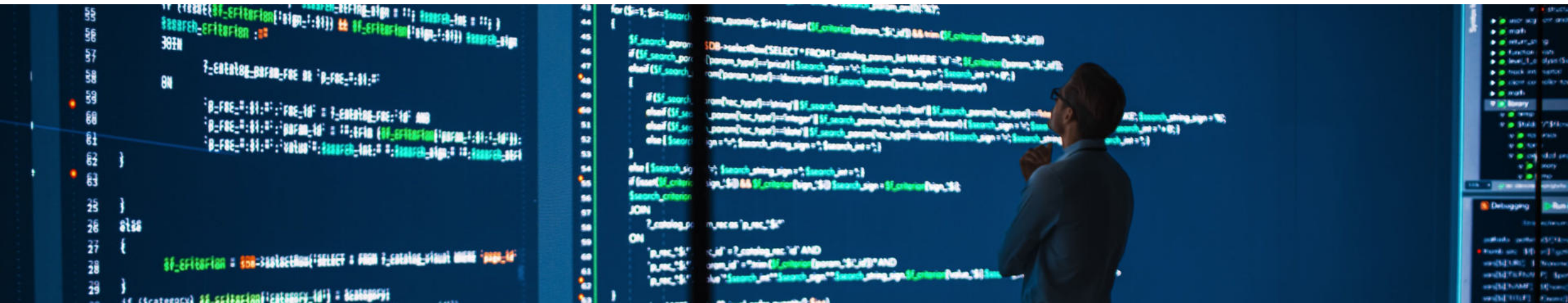
pgpool-II Connection Pooling

- It saves connections to the PostgreSQL servers
- It will reuse connections whenever a new connection request with the same properties (i.e. username, database, protocol version) is received
- It reduces connection overhead, and improves system's overall throughput
- Provides cluster-based connection pooling



Load Balancing Reads

- If a database is replicated, executing a `SELECT` query on any server will return the same result
- pgpool-II automatically detects `SELECT` and can load balance reads among multiple servers, improving the system's overall throughput
- It works best in a situation where there are a lot of users executing many queries at the same time



pgpool-II Installation

- It works on Linux, Solaris, FreeBSD, and most of the UNIX-like architectures
- root or sudo user access is required for installation
- It can be installed using packages available from repos.enterprisedb.com

Install

- `dnf install -y edb-pgpool45`

Check Version

- `pgpool -version`

Config Files

- `/etc/sysconfig/edb/pgpool4.5/`
- `pgpool.conf`
- `pcp.conf`

pgpool Service

- `edb-pgpool-4.5`



Configuring pgpool-II

- Each operation mode has specific configuration parameters in [pgpool.conf](#)
- After installing pgpool-II, `pgpool.conf.sample` is created

```
$ cp /etc/sysconfig/edb/pgpool4.5/pgpool.conf.sample  
/etc/sysconfig/edb/pgpool4.5/pgpool.conf
```
- There are additional sample [pgpool.conf](#) file for each mode

`pgpool.conf.sample-replication`

`pgpool.conf.sample-master-slave`

`pgpool.conf.sample-stream`



pgpool.conf parameters

Connections

listen_addresses, port, pcp_port

Pools

num_init_children, child_life_time, child_max_connections, client_idle_limit, enable_pool_hba, pool_passwd

Logs

log_destination, log_connections, log_statement

File Location

pid_file_name, logdir

Failovers

failover_command, failback_command, follow_master_command

Load Balancing

replication_mode, master_slave_sub, load_balance_mode, replicate_select

Backends

backend_hostname, backend_port, backend_weight, backend_data_directory, backend_flag



Configuring pcp.conf

- pgpool-II provides a control interface for its administration
- pcp.conf is the user/password file for authentication with this interface
- After installing pgpool-II, `/etc/sysconfig/edb/pgpool4.5/pcp.conf.sample` is created

```
$ cp /etc/sysconfig/edb/pgpool4.5/pcp.conf.sample  
/etc/sysconfig/edb/pgpool4.5/pcp.conf
```

- Password can be produced with the `/usr/edb/pgpool4.5/bin/pg_md5` command:

```
$ /usr/edb/pgpool4.5/bin/pg_md5 -p
```



Starting/Stopping pgpool-II

- All the backend database clusters must be started before starting pgpool-II
- `edb-pgpool-4.5` service is created to start / stop or check status of pgpool-II
- Syntax:

```
$ systemctl [ start | stop | status ] edb-pgpool-4.5
```



Connecting via pgpool-II

- pgpool-II relay connections between client and Postgres server
- Client connects to pgpool-II as if it is connecting to Postgres
- Connections are only possible to a database of a configured database cluster
- Connection using psql example:

```
[enterprisedb@localhost ~]$ psql -p 9999 -d edb -U enterprisedb
```



pgbouncer - "Bouncer is always right"

- Lightweight connection pooler for Enterprise Postgres
- Any application can connect to [pgbouncer](#) as it connects with Enterprise Postgres
- It can help to reduce the connection impact on the Enterprise Postgres
- It provides connection pooling(reuse) of the existing connections



pgbouncer Features

- It has low memory requirements (2k per connection by default)
- It is a database-based pooler that can connect to a database from different clusters
- Not tied to one backend server
- Supports online restart/upgrade without dropping client connections
- Supports Windows and Linux platforms
- Bundled in StackBuilder Plus for Windows and also available as package from repos.enterprisedb.com



Types of Connections

- pgbouncer supports several types of pooling when rotating connections:
 - Session pooling
 - A server connection is assigned to the client application for the life of the client connection
 - This is the default
 - Transaction pooling
 - A server connection is assigned to the client application for the duration of a transaction
 - Statement pooling
 - A server connection is assigned to the client application for each statement



How Connections are Established

- An application connects to pgbouncer as if it were a Postgres database
- pgbouncer then creates a connection to the actual database server, or it reuses one of the existing connections from the pool
 - **Step 1** - The client application attempts to connect to Postgres on the port where pgbouncer is running
 - **Step 2** - The database name supplied by the client application must match with the list in [pgbouncer.ini](#)
 - **Step 3** - The username and password supplied must match with the list in [users.txt](#)
 - **Step 4** - If a connection with the same settings is available in the pool it will be assigned to the client otherwise a new connection object will be created
 - **Step 5** - Once the client logs off the connection object returns back to the pool



Setting Up pgbouncer

- Create a `pgbouncer.ini` file:

```
[databases] db1 = host=127.0.0.1 port=5444 dbname=db1

[pgbouncer] listen_port = 6432, listen_addr = 127.0.0.1, auth_type =
md5, auth_file = users.txt, logfile = pgbouncer.log, pidfile =
pgbouncer.pid, admin_users = someuser
```

- Create a `users.txt` file:

```
"someuser" "same_password_as_in_server"
```

- Launch pgbouncer:

```
$ pgbouncer -d pgbouncer.ini
```

- Have your application (or the psql client) connect to pgbouncer instead of directly to the Postgres server:

```
$ psql -p 6432 -U someuser template1
```



Manage pgbouncer

- Management commands can be used: `Show stats, servers, clients, pools, lists, databases, fds`
- Manage pgbouncer by connecting to the special administration database pgbouncer and issuing `show help`;

```
$ psql -p 6432 -U someuser pgbouncer
```

```
pgbouncer=# show help;
```

```
NOTICE: Console usage
```

```
DETAIL: SHOW
```

```
[HELP | CONFIG | DATABASES | FDS | POOLS | CLIENTS | SERVERS | SOCKETS | LISTS | VERSION]
```



pgbouncer Example – Installation

- This example will show how to install and configure pgbouncer to provide connection pooling for Postgres Cluster running on port 5444
- Steps:
 1. Open a terminal and connect in as sudo or root user
 2. Execute “dnf install edb-pgbouncer123” command
 3. Verify pgbouncer version using “/usr/edb/pgbouncer1.23/bin/pgbouncer --version” command
 4. Configuration files are located in /etc/edb/pgbouncer1.23/ directory



pgbouncer Example – Configure a Database

- This example will show the configuration of pgbouncer to provide connection pooling for edbstore database to edbuser user

- Steps:

5. Open pgbouncer.ini file and add the entry for edbstore

```
$ vi /etc/edb/pgbouncer1.23/edb-pgbouncer-1.23.ini  
edbstore = host=127.0.0.1 port=5444 user=edbuser dbname=edbstore
```

Verify following entries:

```
listen_addr = *  
listen_port = 6432  
auth_file = /etc/edb/pgbouncer1.23/userlist.txt  
pool_mode = session
```



pgbouncer Example – Connect via pgbouncer

6. Add the edbuser user in the user authentication file

```
$ vi /etc/edb/pgbouncer1.23/userlist.txt  
"edbuser" "edbuser"
```

7. Enable pgbouncer service

```
# systemctl enable edb-pgbouncer-1.23
```

8. Start pgbouncer service

```
# systemctl start edb-pgbouncer-1.23
```

9. Connect as enterprisedb user in the terminal and connect to the edbstore database using edb-psql via pgbouncer port

```
# su - enterprisedb  
$ psql -p 6432 -h localhost -d edbstore -U edbuser
```





Database Cloning



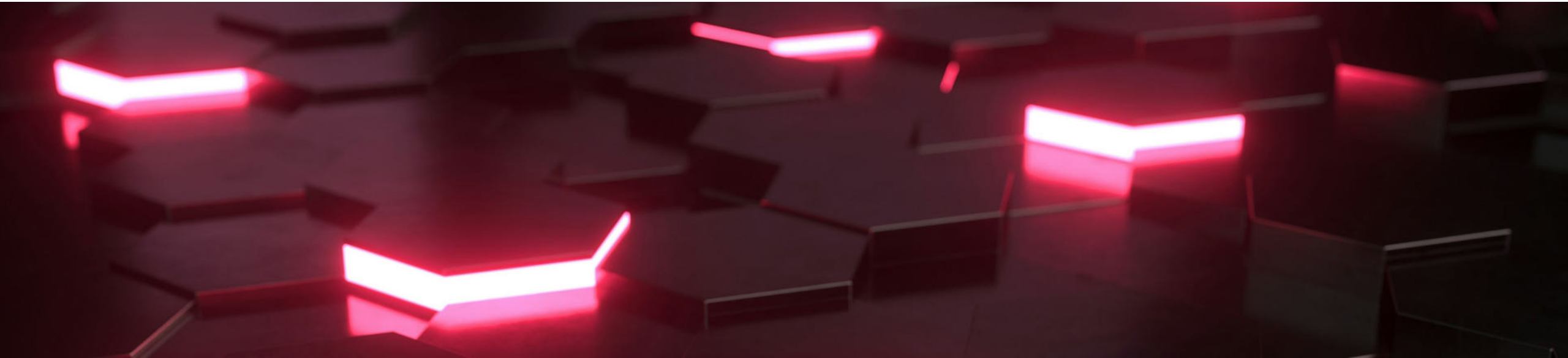
Module Objectives

- Database Cloning
- Creating a Clone Database
- How to Clone a Database using pg_dump
- Clone Schema
- Setup Clone Schema
- Install Clone Schema
- Clone the Schema



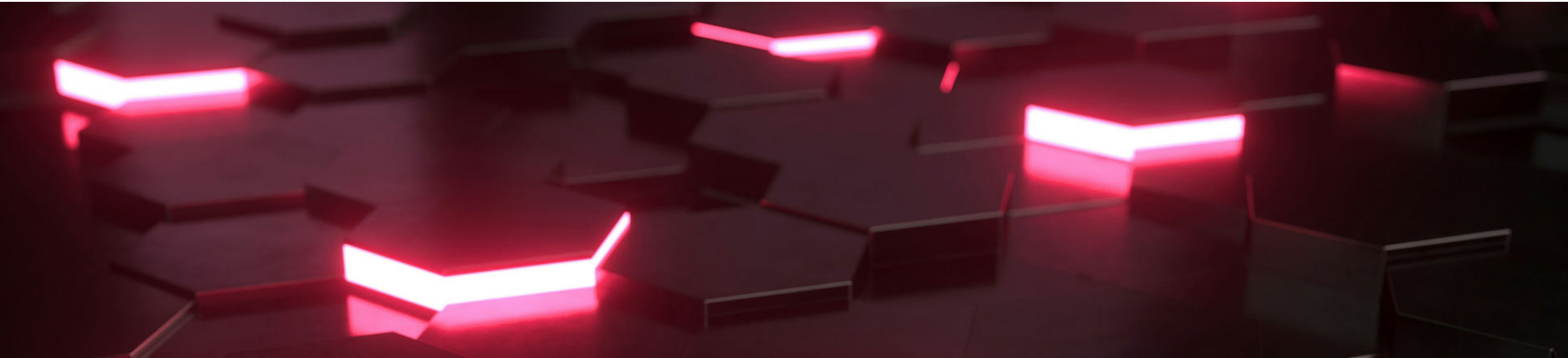
Database Cloning

- Cloning creates a snapshot/copy of an existing database
- All the objects and data is copied over to a new database
- Cloning is different from replication or backup
- Enterprise Postgres provides rapid and space efficient cloning of a database



Steps for Creating a Clone of a Database

- Identify the database to be cloned
- Identify a name for the new clone
- Verify no one is connected to the database to be cloned
- Run the `CREATE DATABASE...TEMPLATE` command to create a clone



Cloning a Database Using pg_dump

- `pg_dump` can also clone a database between clusters
- All options of `pg_dump` can be used for customization and speed
- Need psql access to the database
- Create a new blank database before running `pg_dump`
- No need to remove connections from the to-be cloned database



Clone Schema

- Enterprise Postgres includes Clone Schema extension that gives the ability to make a copy of a schema with its database objects
- Schema can be copied into another database or into same database with different name
- Cloning of a schema can be done in an online as well as in non-blocking, background job mode
- Multiple background worker process can be used to shorten the time of operation



Setup Clone Schema – Install Extensions

- It requires PL/Perl language installed along with the following extensions on the database:
 - `postgres_fdw`
 - `dblink`
 - `edb_job_scheduler`
 - `DBMS_JOB`
- Step 1 – Install the extensions by following commands:
 - `CREATE EXTENSION postgres_fdw;`
 - `CREATE EXTENSION dblink SCHEMA public;`
 - `CREATE EXTENSION edb_job_scheduler;`
 - `CREATE EXTENSION DBMS_JOB;`



Setup Clone Schema – Install and Configure PL/Perl

- Step 2 – Append `$libdir/parallel_clone` to the `shared_preload_libraries` in [postgresql.conf](#)
 - `shared_preload_libraries='$libdir/parallel_clone'`
- Step 3 – Install Perl Procedural Language (PL/Perl) by downloading `edb-as17-server-plperl` if not installed
 - Connect to the database as a superuser and execute:
 - `CREATE TRUSTED LANGUAGE plperl;`
- Step 4 – Restart the server
 - `systemctl restart edb-as-17`



Install Clone Schema

- These steps must be performed on any database to be used as the source or target database by an Clone Schema function:
- Step 1 – Install the extension with following commands:
 - `CREATE EXTENSION parallel_clone;`
 - `CREATE EXTENSION edb_cloneschema;`
- Step 2 – Create the foreign server and user mapping inside the database of the local server in which cloning is to occur
 - `CREATE SERVER local_server FOREIGN DATA WRAPPER postgres_fdw OPTIONS ( host 'localhost', port '5444', dbname 'edbstore');`
 - `CREATE USER MAPPING FOR enterprisedb SERVER local_server OPTIONS ( user 'enterprisedb',password 'password' );`



Example- Install Clone Schema

- Execute steps 1 and 2 on previous slide in database edbstore. In this example edbuser schema will be cloned to edbcopy schema in database edbstore

```
edbstore=# create extension parallel_clone;  
CREATE EXTENSION  
edbstore=# create extension edb_cloneschema;  
CREATE EXTENSION  
edbstore=# create server local_server foreign data wrapper postgres_fdw  
edbstore-# options (host 'localhost', port '5444', dbname 'edbstore');  
CREATE SERVER  
edbstore=# create user mapping for enterprisedb server local_server  
edbstore-# options (user 'enterprisedb', password 'edb');  
CREATE USER MAPPING
```



Example - Clone the Schema (Continued)

- Clone `edbuser` schema to `edbcopy` schema in database `edbstore`

```
edbstore=> set search_path to "$user",public,edb_util;  
SET  
edbstore=> show search_path;  
      search_path  
-----  
"$user", public, edb_util  
(1 row)  
  
edbstore=> select edb_util.localcopyschema('local_server','edbuser','edbcopy','edbuser_log',true,true,true,4);  
localcopyschema  
-----  
t  
(1 row)
```



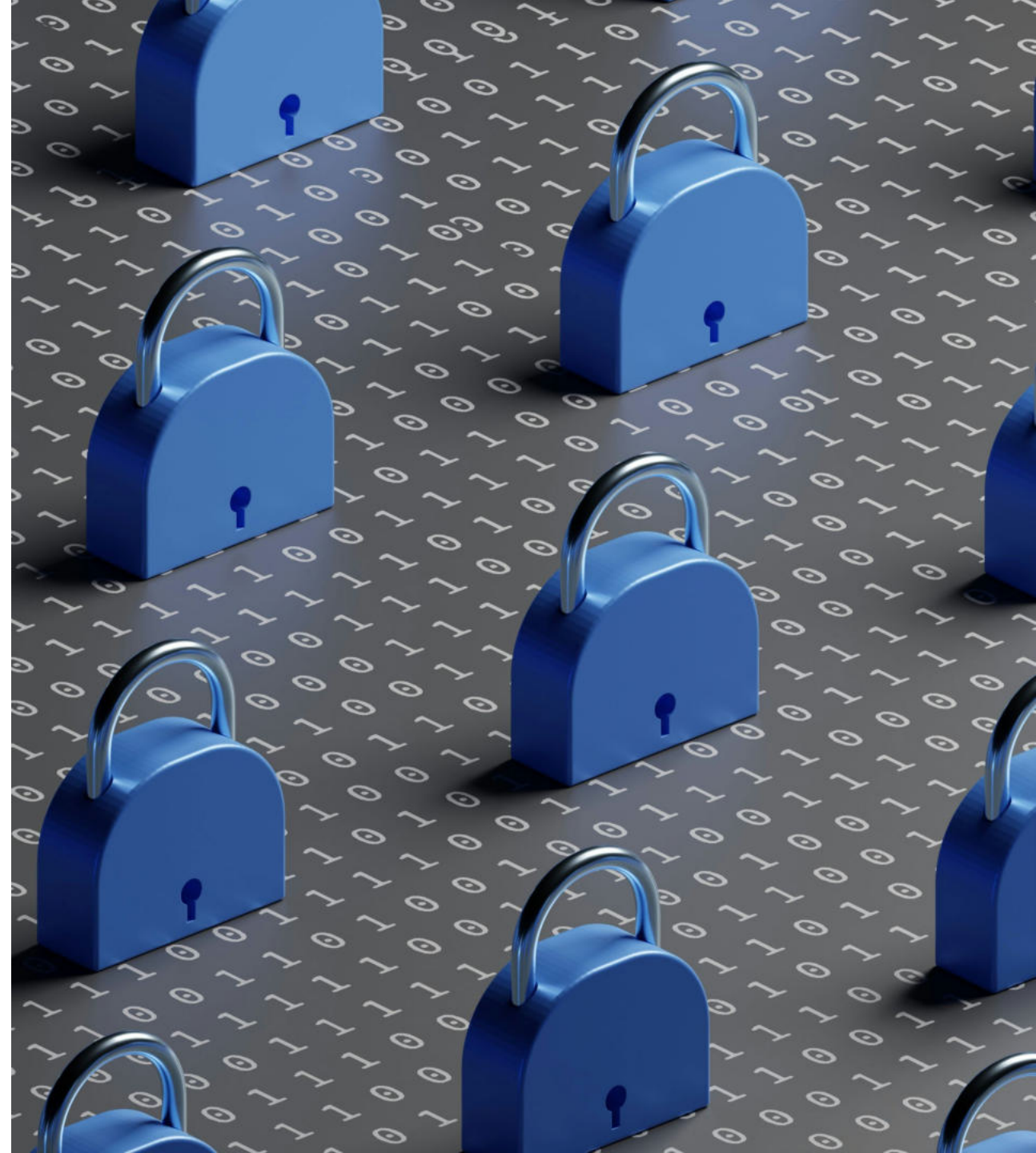


Transparent Data Encryption



Module Objectives

- Data Security Levels
- Data encryption options available with Enterprise Postgres
- Overview of Transparent Data Encryption (TDE)
- Data Encryption Key Management
- Encrypt Database Cluster using TDE



Data Security Levels



Server and Application

SSL Connections
Host Based Access
Control



Database

User/Password
Connect Privilege
Schema Permissions
Database Firewall -
SQL Protect



Object

Table and Column
Level Privileges
Grant/Revoke



Data Encryption

TDE
EDB*Wrap
DBMS_ENCRYPT
DBMS_REDACT



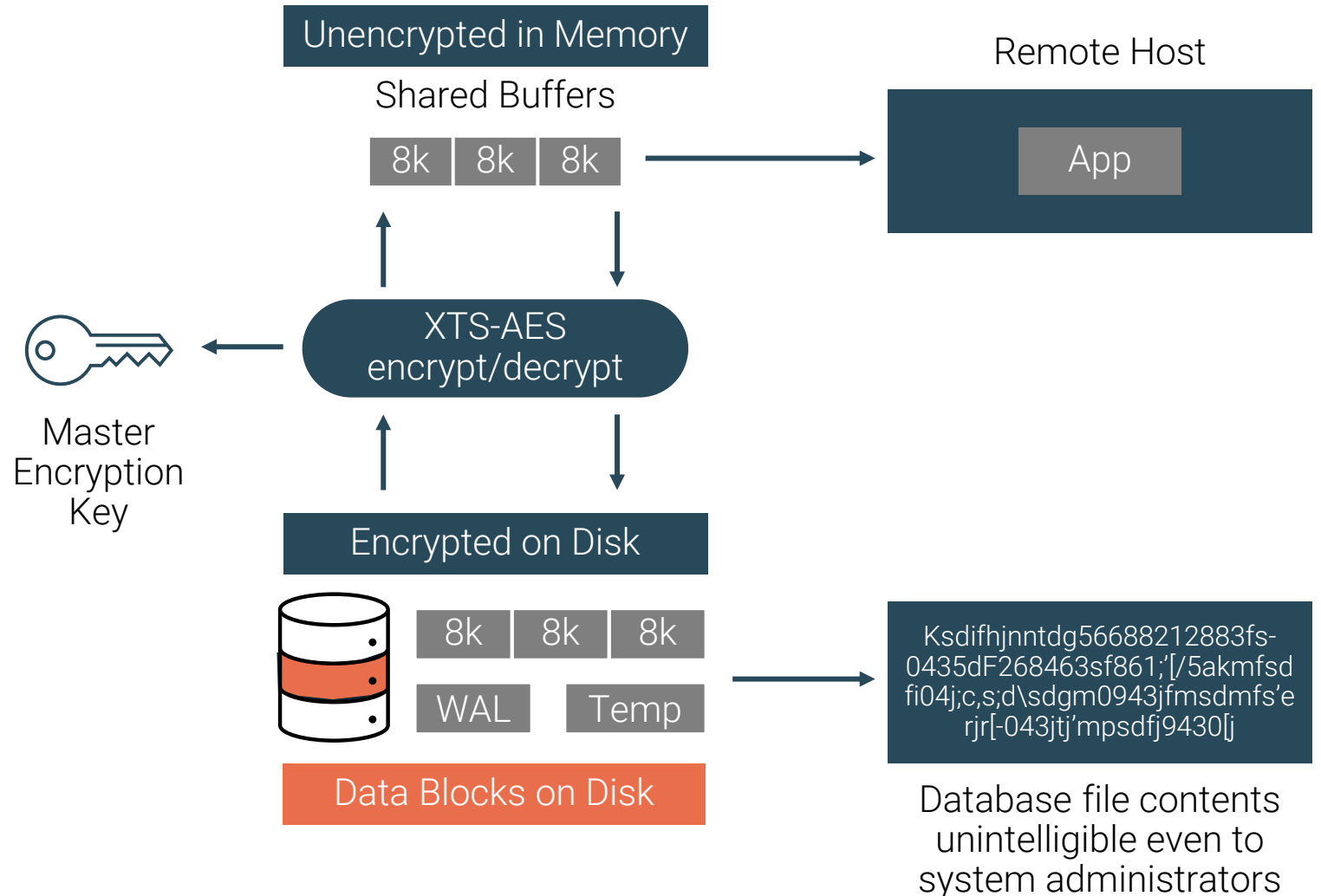
Data Encryptions options

- Enterprise Postgres offer different levels and granularity of protection depending on the needs of the application
 - Oracle-compatible wrapper around pgcrypto - DBMS_CRYPTO
 - Encrypts data on disk using Transparent Data Encryption
 - Support for OS based encrypted file system
 - Data at-rest encryption using Thales CipherTrust Transparent Encryption
 - AES-256 encryption for TDE : Transparent data encryption (TDE) now supports the AES key length 256 option to TDE



Transparent Data Encryption

- It encrypts any user data stored in the database system
- Encryption is transparent to the user
- User data includes:
 - Actual data stored in tables and other objects
 - System catalog data such as object names



Encryption Coverage

Encrypted:



- Data Files - AES-128-XTS
- Write-Ahead Logs (WAL) - AES-128-CTR
- Temporary files - AES-128-XTS and AES-128-CBC
- Base backups

Not Encrypted:



- Transaction status metadata
- Data Directory File Names
- Foreign Tables
- Error logs and config files



Data Encryption Key (Data Key)

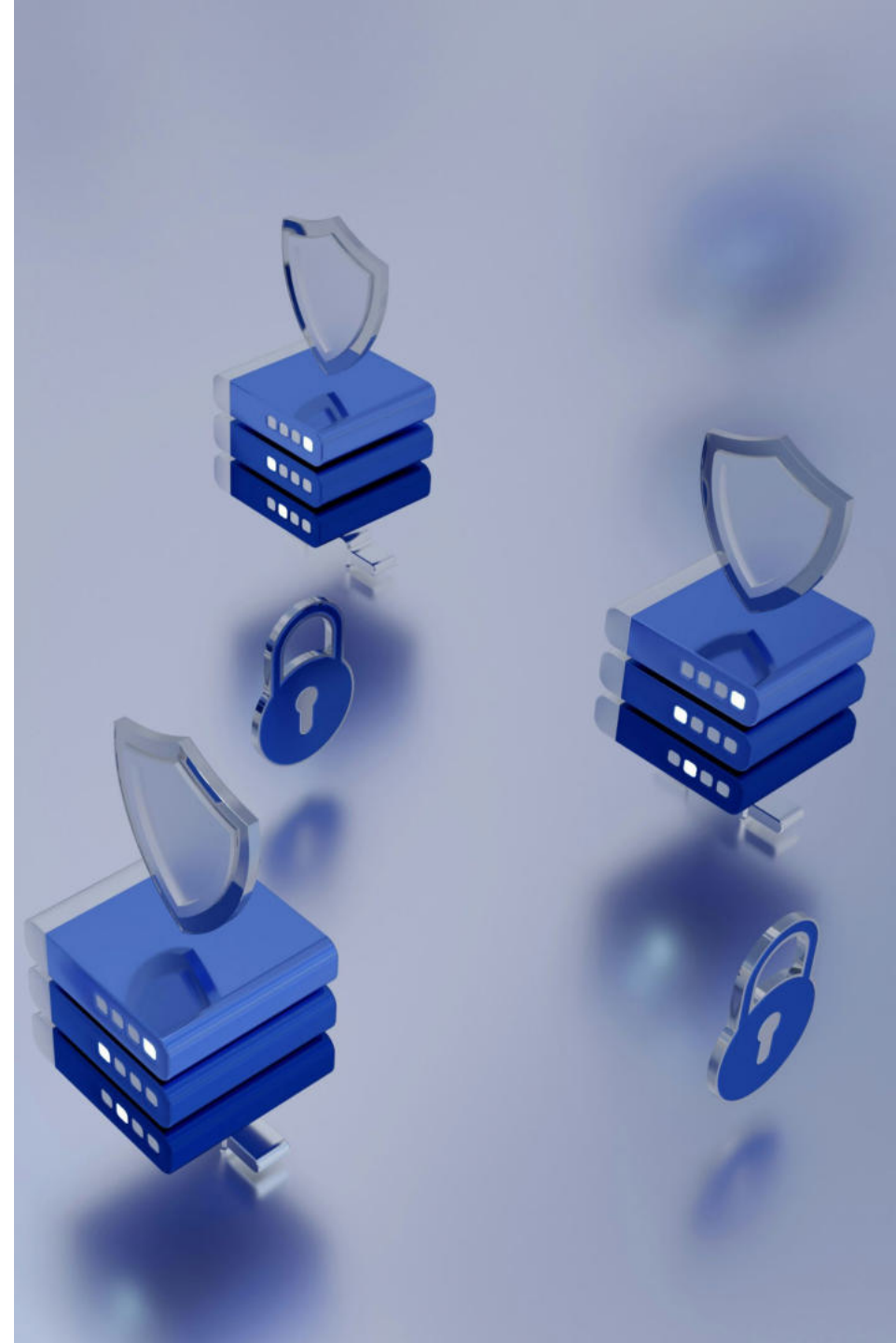
- The data key is generated by `initdb` and stored in a file “`pg_encryption/key.bin`”
- Encryption key is external to the database allowing for simple passphrase encrypt/decrypt
- Integration with various enterprise key management solutions:
 - Amazon AWS Key Management Service (KMS)
 - Google Cloud - Cloud Key Management Service
 - HashiCorp Vault
 - Microsoft Azure Key Vault
 - Thales CipherTrust Manager



Data Encryption Key (Data Key)

Data Key Protection Options:

- Passphrase Protection:
 - Protect the data key by encrypting it with a wrapping key derived from a passphrase.
- Key Management System (Key Store):
 - Store the wrapping key (key-wrapping key or master key) in a separate key management system.



Enabling TDE and Encrypting Data Key

- Enable TDE during database initialization using `-y` or `--data-encryption` option
- initdb provides data key wrapping options:

`--key-wrap-command`

`--key-unwrap-command`

- Disable key wrapping using `--no-key-wrap` option
- Optional environmental variables:
 - PGDATAKEYWRAPCMD
 - PGDATAKEYUNWRAPCMD



Configuration Parameters

- Postgres configuration parameters are automatically modified based on `initdb` options:
 - `data_encryption_key_unwrap_command`
 - `data_encryption_key_unwrap_command`



Example: Passphrase Protection

- Initialize the cluster using `initdb` with TDE:

```
initdb -D datadir1 -y --key-wrap-command='openssl enc -e -aes-128-cbc -pass pass:edb123 -out "%p"' --key-unwrap-command='openssl enc -d -aes-128-cbc -pass pass:edb123 -in "%p"'
```

- Data key is protected(wrap) using a passphrase
- Data key unwrap command is stored in `postgresql.conf`:

```
data_encryption_key_unwrap_command = 'openssl enc -d -aes-128-cbc -pass pass:edb123 -in "%p"'
```

- Password can also be prompted during `initdb` and start-up using `systemd-ask-password`



Example: Key Management System (AWS)

Generate a new master key and assign an alias:

- `aws kms create-key`
- `aws kms create-alias --alias-name alias/epas-master-tde1 --target-key-id "6c7188fd-5d62-4548-87d9-905de1c3835c"`

Encryption/Decryption commands for the TDE data key:

- `export PGDATAKEYWRAPCMD='aws kms encrypt --key-id alias/epas-master-tde1 --plaintext fileb:///dev/stdin --output text --query CiphertextBlob | base64 -d > "%p"'`
- `export PGDATAKEYUNWRAPCMD='aws kms decrypt --key-id alias/epas-master-tde1 --ciphertext-blob fileb://"%p" --output text --query Plaintext | base64 -d'`



Example: Key Management System (AWS)

- Initialize the cluster using initdb with TDE:

```
initdb -D datadir2 -y
```

- Data key unwrap command is stored in postgresql.conf:

```
data_encryption_key_unwrap_command = 'aws kms decrypt --key-id  
alias/epas-master-tde1 --ciphertext-blob fileb://" %p" --output text --query  
Plaintext | base64 -d'
```





Upgrading Best Practices



Module Objectives

- Version Change and Upgrade
- Need to Upgrade
- Upgrade Plan
- Upgrade Using pg_upgrade
- Upgrading Best Practices



Version Change and Upgrade

- Minor Version Upgrade

- Storage format not changed
- Simply replace executable
- Start server with upgraded executable and old data directory

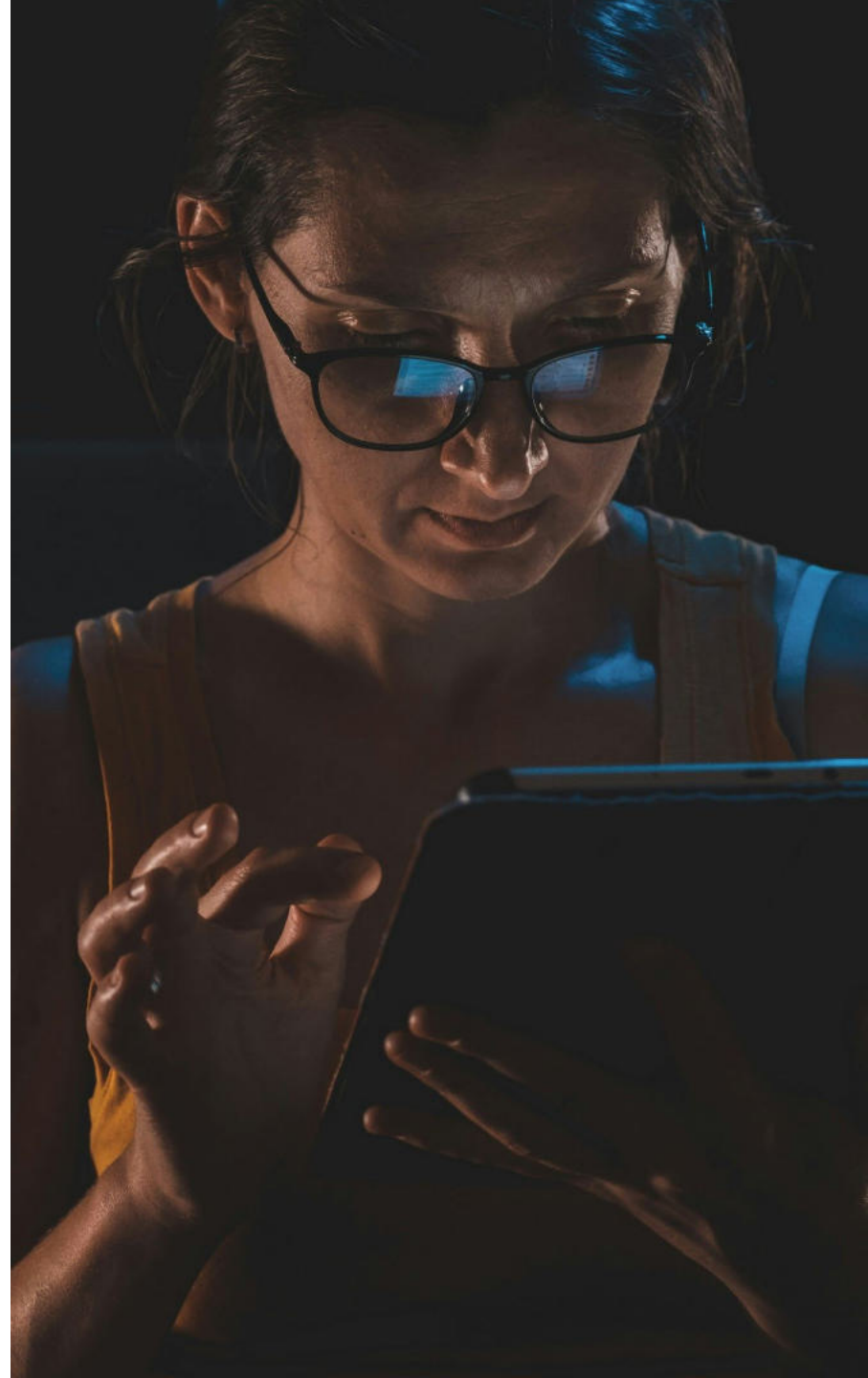
- Major Version Upgrade

- Storage format may change
- Upgrade is complicated and can be performed using:
 - `pg_dumpall`
 - `pg_upgrade`
 - Replication



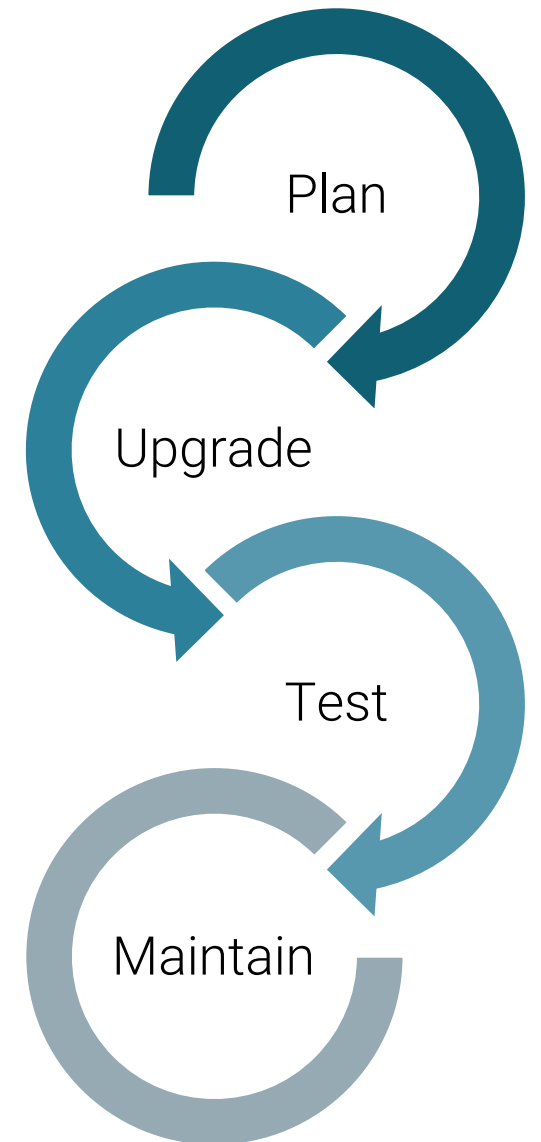
Need to Upgrade

- Enterprise Postgres is an actively developed database
- Major releases include new features, capabilities and performance enhancements
- Minor releases include security and data corruption bug fixes
- EDB Postgres® AI recommends all users to run the latest available minor release
- Not upgrading is riskier than upgrading
- EDB Postgres® AI aims to fully support a major release for five years



Upgrade Plan

1. Read the release notes and check for user-visible incompatibilities :
https://www.enterprisedb.com/docs/epas/latest/epas_rel_notes/epas17_2_rel_notes/
2. Application must be tested on new version before complete switch
3. It is recommended to setup concurrent installations of old and new Postgres versions
4. Plan for the worst case – Going back to old version
5. Decide the cutoff time
6. Communicate the goal and timing to the technical support team



Upgrade Using pg_upgrade

Enterprise Postgres

- Version 9.0 or later
- Data directory
- Old binaries

pg_upgrade

- Runs on new server
- Old and new cluster info
- Can be run in link mode
- Support for Parallel load

Enterprise Postgres

- Latest Version
- New Features
- Better Performance



Example - pg_upgrade

- Scenario: Upgrade Enterprise Postgres 16 data cluster to Enterprise Postgres 17
- Solution:
 - Step 1 - Verify the source version, objects and data directory

```
[enterprisedb@training ~]$ psql -p 5442 edb
edb=# select version();
PostgreSQL 16.6 (EnterpriseDB EDB Postgres
Advanced Server 16.6.0) on x86_64-pc-linux-
gnu, compiled by gcc (GCC) 11.5.0 20240719
(Red Hat
11.5.0-2), 64-bit
edb=# show data_directory;
data_directory
-----
/var/lib/edb/as16/data
```

object_name	object_type
emp_comp	Function
emp_query	Function
emp_query_caller	Function
emp_sal_trig_emp	Function
hire_clerk	Function
hire_salesman	Function
list_emp	Function
new_empno	Function
select_emp	Function
user_audit_trig_emp	Function
next_empno	Sequence
dept	Table
emp	Table
jobhist	Table
salesemp	Table
salesemp	View



Example - Install and Create Cluster

- Step 2 - Install Enterprise Postgres 16
- Step 3 - Initialize a new 17 data cluster and change default port

```
[enterprisedb@training ~]$ initdb --version  
initdb (EnterpriseDB) 17.2.0  
  
[enterprisedb@training ~]$ initdb -D /home/enterprisedb/data17 -W  
  
[enterprisedb@training ~]$ vi data17/postgresql.conf  
  
port=5445
```



Example - Stop Cluster and Perform Consistency Check

- Step 4 - Stop source 16 data cluster

```
[enterprisedb@training ~]$ exit  
[centos@training ~]$ sudo systemctl stop edb-as-16  
[centos@training ~]$ sudo su - enterprisedb
```

- Step 5 – Perform a consistency check

```
[enterprisedb@training ~]$ pg_upgrade -d /var/lib/edb/as16/data/  
-D /home/enterprisedb/data17 -p 5442 -P 5445 -b  
/usr/edb/as16/bin/ -B /usr/edb/as17/bin/ --check  
Performing Consistency Checks  
-----  
Checking cluster versions  
.....  
  
*Clusters are compatible*
```



Example – Run pg_upgrade and Start Cluster

- Step 6 - Run pg_upgrade

```
[enterprisedb@training ~]$ pg_upgrade -d /var/lib/edb/as16/data/ -D  
/home/enterprisedb/data17 -p 5442 -P 5445 -b /usr/edb/as16/bin/ -B /usr/edb/as17/bin/  
Upgrade Complete  
-----  
Optimizer statistics are not transferred by pg_upgrade.  
Once you start the new server, consider running:  
    /usr/edb/as17/bin/vacuumdb --all --analyze-in-stages  
  
Running this script will delete the old cluster's data files:  
    ./delete_old_cluster.sh
```

- Step 7 - Start newly upgraded cluster after changing port to 5445

```
[enterprisedb@training ~]$ pg_ctl --version  
pg_ctl (EnterpriseDB) 17.2.0  
[enterprisedb@training ~]$ pg_ctl -D /home/enterprisedb/data17/ start
```



Example - Connect and Verify

- Step 8 - Connect and verify upgrade

```
[enterprisedb@training ~]$ psql -p 5445 edb

edb=# select version();
PostgreSQL 17.2 (EnterpriseDB EDB Postgres
Advanced Server 17.2.0) on x86_64-pc-linux-gnu,
compiled by gcc (GCC) 11.5.0 20240719 (Red Hat
11.5.0-2), 64-bit

edb=# show data_directory;
-----
      /home/enterprisedb/data17
(1 row)
```

object name	object type
emp_comp	Function
emp_query	Function
emp_query_caller	Function
emp_sal_trig_emp	Function
hire_clerk	Function
hire_salesman	Function
list_emp	Function
new_empno	Function
select_emp	Function
user_audit_trig_em p	Function
next_empno	Sequence
dept	Table
emp	Table
jobhist	Table
salesemp	Table
salesemp	View



Upgrading Best Practices

- Enterprise Postgres upgrade is not automatic
- Run `pg_upgrade` with `--check` option for any upgrade issues
- Take a full backup before and after upgrade
- If `--link` option is used with `pg_upgrade`, it's unsafe to start old cluster
- Check for failed rebuild, and reindex cases reported by `pg_upgrade`
- Watch for post-upgrade warnings generated by `pg_upgrade` and run the generated script on new database as Superuser
- Do not stop or abort the upgrade process once started
- In a streaming replication environment, replica server also needs upgrade
- Once verified, remove old cluster footprints



Module Summary

- Version Change and Upgrade
- Need to Upgrade
- Upgrade Plan
- Upgrade Using pg_upgrade
- Upgrading Best Practices





Conclusion



Course Summary

- Introduction
- Database Monitoring
- SQL Tuning
- Performance Tuning
- Benchmarking
- Extensions
- Table Partitioning
- Connection Pooling
- Cloning
- Transparent Data Encryption
- Upgrading Best Practices





Thank you!

Please visit our Training Portal for
more courses and workshops!